

QuartzHub Handbook

Turning a folder of notes into a website

As of November 14, 2026

Contents

QuartzHub 4

1 – Getting started 6
- 1.1 – What QuartzHub is 7
- 1.2 – Installing and first run 9
- 1.3 – Reading the window 12
- 1.4 – Your first site 15

2 – Projects 18
- 2.1 – The Projects window 19
- 2.2 – Creating a new project 23
- 2.3 – Opening an existing project 26
- 2.4 – Duplicating a project 27
- 2.5 – The settings 29

3 – Overview 33
- 3.1 – The overview 34

4 – Setup 38
- 4.1 – Site 39
- 4.2 – The content folder 42
- 4.3 – The project image 44
- 4.4 – Translations 45
- 4.5 – Plugins 47
- 4.6 – Navigations 50
- 4.7 – Additional content 53
- 4.8 – Languages 56
- 4.9 – Forms 58
- 4.10 – Marketplace 62

5 – Design 64
- 5.1 – The layers 65
- 5.2 – Layout overview 67
- 5.3 – Components 70
- 5.4 – Frames 73
- 5.5 – Styles ▸ Basics 76
- 5.6 – Community themes 79
- 5.7 – Variables 81
- 5.8 – Custom CSS 83

5.9 – Templates 85

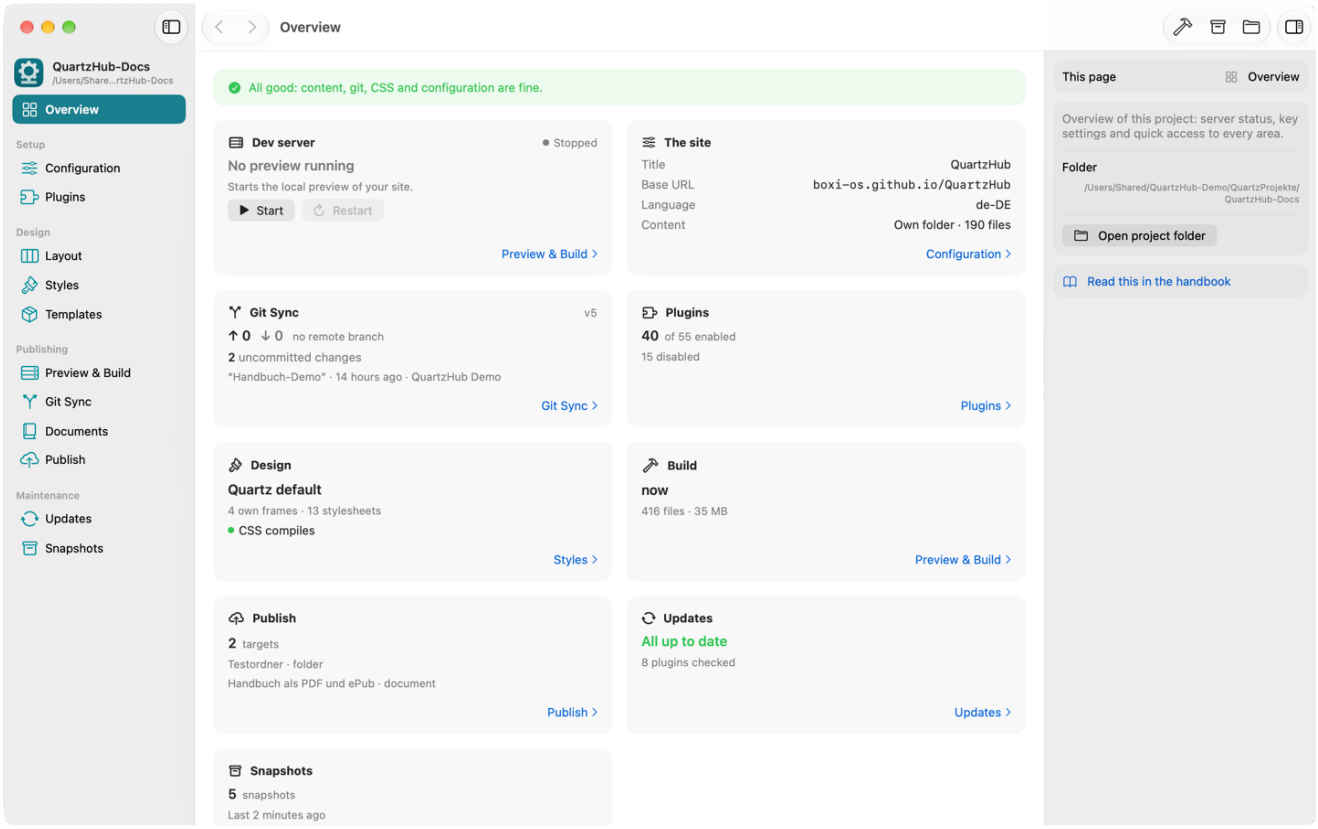
6 – Publishing 88
6.1 – The preview 89
6.2 – The build 91
6.3 – Reading the output 92
6.4 – Other servers 94
6.5 – Git Sync 95
6.6 – Documents 98
6.7 – Connections and targets 100
6.8 – GitHub Pages 103
6.9 – To a web space 104
6.10 – To a folder 106
6.11 – Webhook 107
6.12 – What gets uploaded 108

7 – Maintenance 110
7.1 – Updates 111
7.2 – Snapshots 113
7.3 – Restoring 116
7.4 – Cleaning up 118

8 – Reference 119
8.1 – Menus and shortcuts 120
8.2 – Where the data lives 122
8.3 – Connections to the network 124
8.4 – Known limits 126
8.5 – Glossary 127

QuartzHub

QuartzHub is a Mac app for **Quartz**, a static site generator. Without it, you run Quartz from the command line and edit its configuration in a text editor; with it, you click. This handbook explains every window and every page of QuartzHub: what it does, when you need it and what is better left alone there.



A project's overview: preview, build, publishing, snapshots and updates at a glance.

What QuartzHub can do

Notes become a website

An Obsidian vault or any folder becomes the content folder, **linked** if you want to keep writing in Obsidian, copied if not. Switching later is possible without losing a note. → 4.2

Designing without writing CSS

Colours and fonts, community themes, the site's CSS variables separately for light and dark, stylesheets of your own. The page grid is made in an editor with the mouse, per screen width. → 5.1, 5.4

Plugins from a marketplace

Browse, install, set up, with a form instead of YAML. Navigations, additional content boxes, languages and forms have editors of their own with a preview next to them. → 4.5

Looking before anyone else does

The preview runs in the window and rebuilds on every change, at the width of a desktop, tablet or phone; Quartz's output is below it. → 6.1

Books from the site

A selection of pages, in the order you want, as PDF or ePub, with a table of contents and formulas. This handbook exists that way too. → 6.6

Publishing and coming back

GitHub Pages, a web space over SFTP, rsync or FTP, a folder, a webhook, with a preview of what will be transferred. And before larger changes a snapshot to go back to. → 6.7, 7.2

Where to start

- **New here:** 1.1 – What QuartzHub is, then 1.4 – Your first site.
- **Looking something up:** Menus and shortcuts, Glossary.

How to *write notes* so they make good pages is in the **Example handbook**.

1 – Getting started

This part assumes nothing. Anyone who writes notes and wants a website made of them is in the right place, even without ever having heard of Quartz, git or a static site generator.

1. **What QuartzHub is**: what it takes off your hands and what it deliberately leaves alone.
2. **Installing and first run**: what the Mac needs and what happens the first time QuartzHub opens.
3. **Reading the window**: sidebar, inspector, history, saving. It works the same on every page; once you know this page, you know them all.
4. **Your first site**: from an empty project to the preview.

1.1 – What QuartzHub is

QuartzHub is a Mac app for **Quartz**. Without it, you run Quartz from the command line and edit its configuration file in a text editor; with it, you click.

What it takes off your hands

A Quartz project consists of a configuration file, a folder full of notes, a few stylesheets and a list of installed **plugins**. All of it is text, and all of it could be edited by hand. QuartzHub gives each of these a page:

- **Create a new project** without running git and npm yourself.
- **Edit the configuration** without writing YAML, with an explanation next to every field.
- **Link the content folder**, also straight to an Obsidian **vault**, and switch between **Copy** and **Link** without losing a note.
- **Install and set up plugins** from a searchable **marketplace**, with a form instead of YAML.
- **Change colours, fonts and spacing**, in layers that sit side by side in the order in which they override each other.
- **Build the page grid** in an editor with the mouse instead of in CSS.
- **Start the preview and build the site**, with Quartz's output in the same window.
- **Make books**: a selection of pages as PDF or ePub.
- **Publish** in several ways, with a preview of what will be transferred.
- **Get back** when something went wrong: before larger changes QuartzHub takes a **snapshot** by itself.

What it does not do

It is not an editor. Notes are written elsewhere, in Obsidian, in a text editor, anywhere. QuartzHub does not touch what is in the content folder; it builds a website from it.

It does not replace Quartz. What appears on the finished site is up to Quartz and its plugins. Where this handbook stops at "that is how Quartz does it", the **Quartz documentation** carries on.

It does not hide the files. Every change lands in exactly the file it would have landed in had you made it by hand. A project set up with QuartzHub can be carried on by hand at any time, and the other way round.

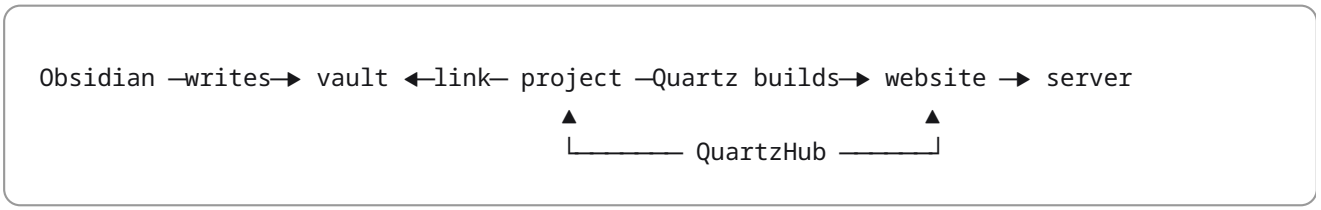
The tools behind it

Working with QuartzHub involves four programs. Only one of them needs operating.

Program	Role	Do you operate it?

Obsidian or another editor	Where notes are written, as Markdown files in a folder.	Yes, to write. Obsidian is not required, but Quartz understands its [[en/8-reference/05-glossary#Wikilink
Quartz	Turns the notes into a website. This is called a [[en/8-reference/05-glossary#Build	build]].
Node and npm	The runtime Quartz works in, and its package manager.	No, QuartzHub brings both.
git	Keeps the history of files.	Only under **Git Sync** and when publishing to a [[en/8-reference/05-glossary#Branch

This is how they fit together:



The vault need not live elsewhere. Without one, let QuartzHub create an empty content folder in the project and write there.

Who it is for

For anyone who writes notes and wants a website made of them. No prior knowledge of Quartz is needed, nor of git. Those who already use Quartz will find the same files, but with a preview next to every switch and a way back when an update breaks something.

1.2 – Installing and first run

System requirements

- **macOS 26 or later.**
- **A Mac with Apple silicon** (M1 or later). There is no version for Macs with an Intel processor.
- **Internet** when creating a project (Quartz comes from GitHub), when installing plugins and when publishing. Editing, preview and build work without.

You do **not** need to install Quartz, Node or npm: QuartzHub brings Node and npm, and Quartz lives in each project. QuartzHub takes git from the system when one answers there, so your own `.gitconfig` and credential helpers keep working; otherwise it uses the one it brings.

Download and install

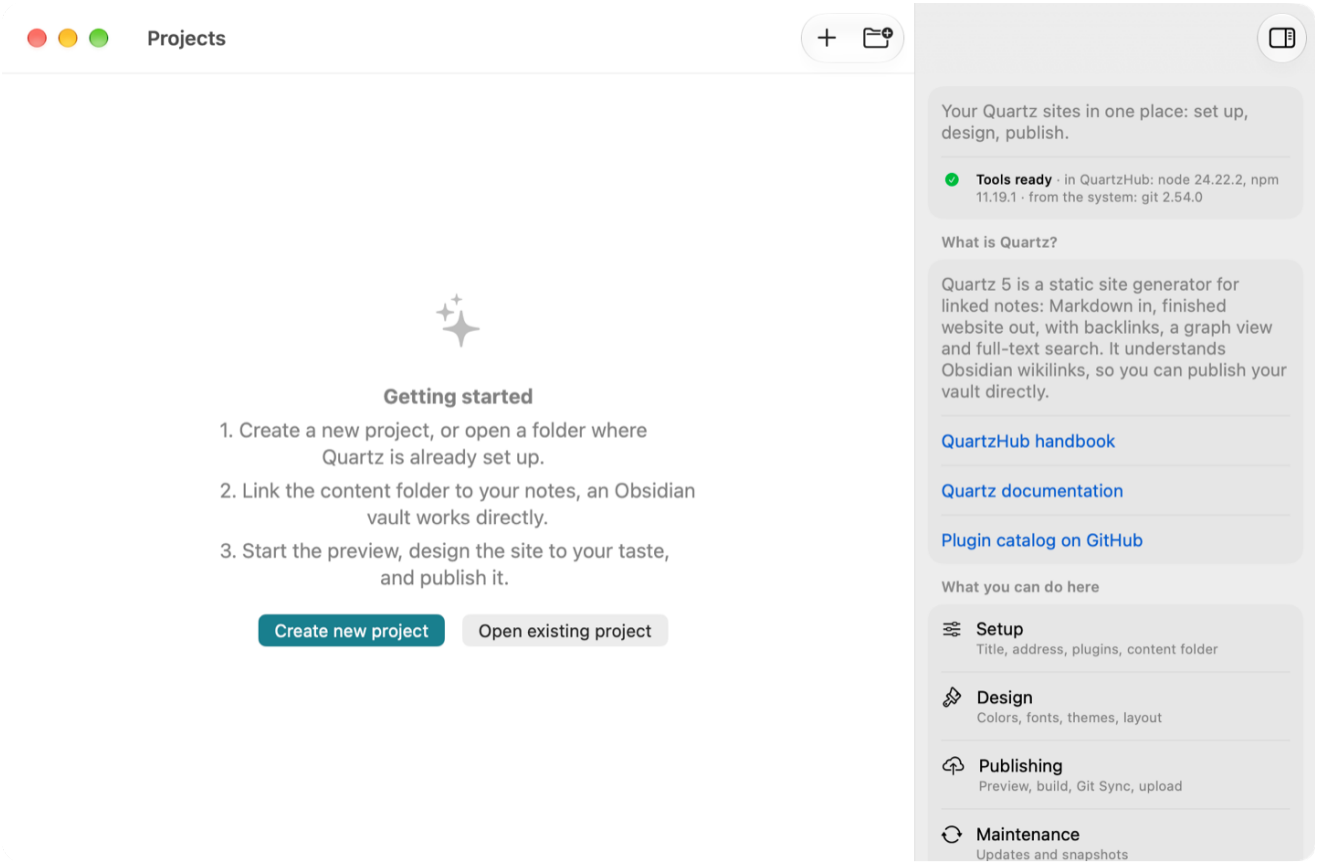
QuartzHub comes as a DMG on the **releases page**. Open the DMG and drag QuartzHub into the Applications folder.

> **macOS will not start QuartzHub the first time**
>
> QuartzHub is only ad hoc signed and not notarised by Apple; that would take a Developer ID certificate. Gatekeeper, macOS's launch check, therefore refuses the first start, saying Apple cannot check the app. Start it once and close the message, then go to **System Settings ▸ Privacy & Security**, click **Open Anyway** in the **Security** section and confirm with your login password. Since macOS 15 the context menu (right click ▸ Open) no longer works for this.
>
> If you like the Terminal, remove the quarantine flag macOS puts on downloaded files instead:
>
> ```
> xattr -dr com.apple.quarantine /Applications/QuartzHub.app
> ```

Updating means downloading the new DMG and replacing the app in Applications. Projects, settings and connections live outside the app and stay as they are (see 8.2).

The first run

At the first run the **Projects** window opens.



The Projects window at the first run: on the left the project list, still empty, with the first steps; on the right the inspector.

The inspector on the right says what QuartzHub knows about the Mac:

- **Tools ready** means: `node`, `npm` and `git` answered when asked for their version. Next to each it says where it comes from, “in QuartzHub” or “from the system”.
- If one does not answer, a note above the project list says what does not work without it: builds, new projects, plugins from a git source, snapshots. As Node and npm are part of QuartzHub, a note about them always means this installation is incomplete. Reinstalling QuartzHub is the quickest fix.

Below, the inspector briefly explains what Quartz is and links to the Quartz documentation and the plugin catalogue.

Questions from macOS

Two things macOS asks not at the first run, but when the time comes:

- **Local network.** The first time you publish to a server on your own network (SFTP, rsync, FTP), macOS asks whether QuartzHub may access the local network. Without that permission the connection fails; QuartzHub then says where to grant it later: **System Settings ▸ Privacy & Security ▸ Local Network**.

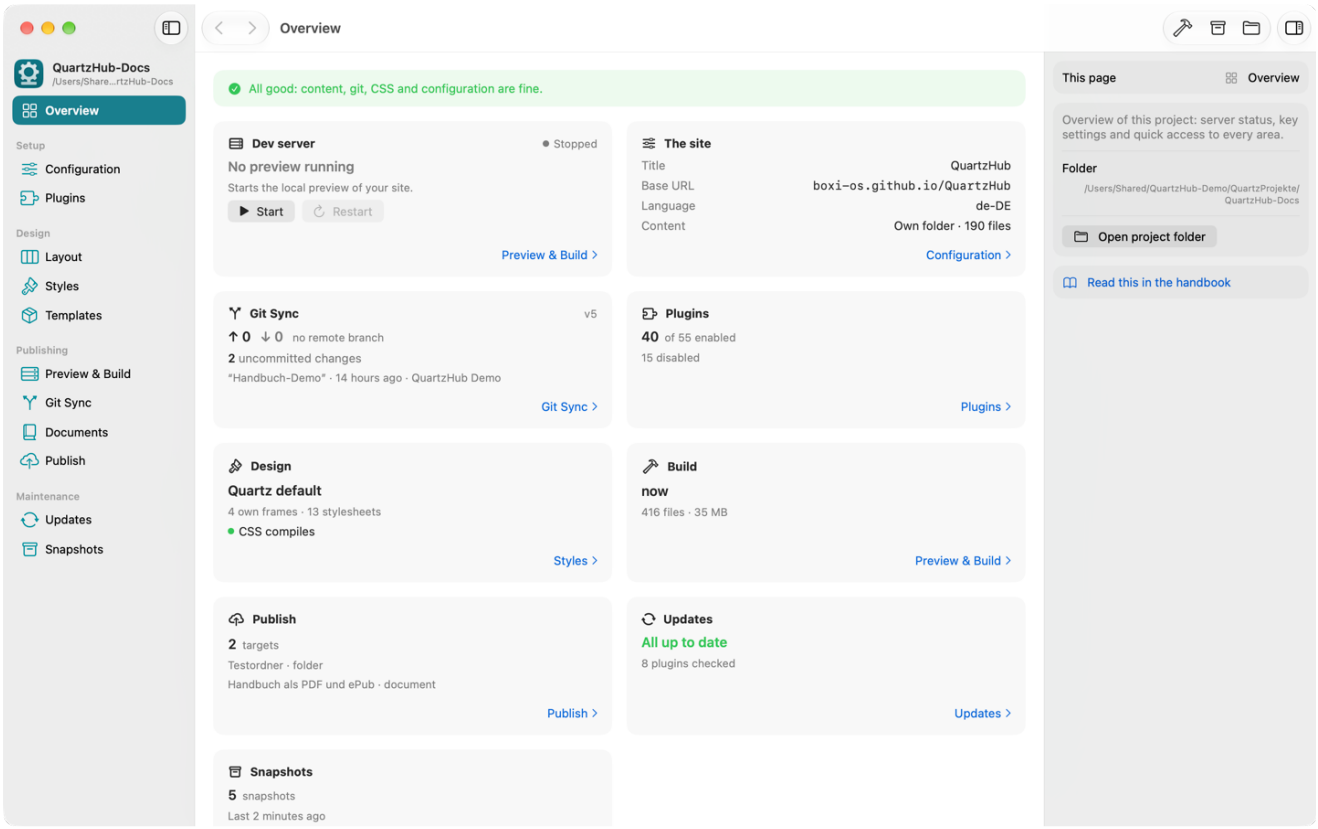
- **Keychain.** QuartzHub keeps passwords and `tokens` for publishing in the macOS keychain, never in a file. When a new version of QuartzHub reads an item for the first time, macOS asks once; “Always Allow” is enough.

“Cannot check” is not “all good”

This holds in many places in QuartzHub, for the first time here: if something cannot be determined, QuartzHub says so instead of giving the all-clear. A catalogue that could not be loaded is not empty, and a version that could not be determined is not the latest.

1.3 – Reading the window

Every page of a project is built the same way. Having read this page, you need not learn the controls again in parts 3 to 7.



The project window: the sidebar on the left, the page in the middle, the inspector on the right.

Two kinds of window

The **Projects** window lists all projects; that is where you create, open, duplicate and remove them (2.1). Every project opens in a **window of its own**. Two projects can be open side by side, and opening a project that is already open brings its window to the front. Back to the list via **Window ▸ Projects** (⇧⌘0).

The sidebar

At the top: the project's image, name and folder. Below, the pages, in the order in which you usually set up a project:

Group	Pages	Part
–	Overview	[[en/3-overview/index
Setup	Configuration, Plugins	[[en/4-setup/index

Design	Layout, Styles, Templates	[[en/5-design/index
Publishing	Preview & Build, Git Sync, Documents, Publish	[[en/6-publishing/index
Maintenance	Updates, Snapshots	[[en/7-maintenance/index

The button at the top left hides and shows the sidebar.

Title and history

The window's **title** names the place, not the project: the page on top, below it the tab if the page has tabs. The **Window** menu shows "project – page" instead, so two project windows can be told apart.

Back and **Forward** at the left of the toolbar go through the window's history, page by page and tab by tab, as in a browser. The shortcuts are ⌘[and ⌘]. A link to another page, such as "Plugins ›" on the overview, is a step like any other; Back returns to where you came from. The same commands and all pages are in the **Go** menu.

Tabs

Configuration, Plugins, Layout and Styles have several tabs. They sit as segments in a row of their own above the page. When the window is too narrow for all segments, they turn into a menu named after the chosen tab.

The inspector

On the right is the **inspector**, in the same place on every page. When something on the page is selected, a row in a list, an area in a grid, it shows its details, settings and actions. When nothing is selected, it explains the page, shows what applies to it and leads to the matching chapter with **Read this in the handbook**; with a selection, **Help ▸ This Page in the Handbook** does. Where the inspector saves at once instead of waiting for **Save**, it says so itself.

The button at the far right of the toolbar shows and hides it, as do **View ▸ Show Inspector** and ^⌘I. That applies to the whole window, not to one page.

The toolbar

On the right of the toolbar, always in the same order:

1. the **page's actions** as symbols, such as ➕ to create something. At most one action per page is highlighted in colour, the one the page is mostly about, such as **Start** on Preview & Build;
2. **Save**, on pages that edit a file;

3. the button for the **inspector**.

Whatever applies to the whole project is also in the **Project** menu: **Start Server** (⌘R), **Stop Server** (⌘.), **Run build** (⌘B) and **Show Project Folder in Finder**.

Saving

Pages that edit a file, such as Configuration and Styles, write only on **Save** (⌘S). The button is only enabled when there is something to save. Unsaved changes also show as the dot in the red button at the top left of the window.

Leaving a page with unsaved changes, through the sidebar, the history, a link, closing the window or quitting, asks: **Save**, **Discard changes** or **Cancel**. If saving fails, you stay on the page. Nothing gets lost silently.

Notes and explanations

Three kinds of text appear next to the controls:

- **A box** with a symbol says something about the state of a thing: in the accent colour a piece of information, orange a warning, red an error. Its text can be selected and copied, such as a path in an error message.
- **A red line right below a field** says what is wrong with what was just typed.
- **Grey text** explains what a field does.

Folding and dragging

Fold groups open and close with a click anywhere in their header row, not only on the arrow.

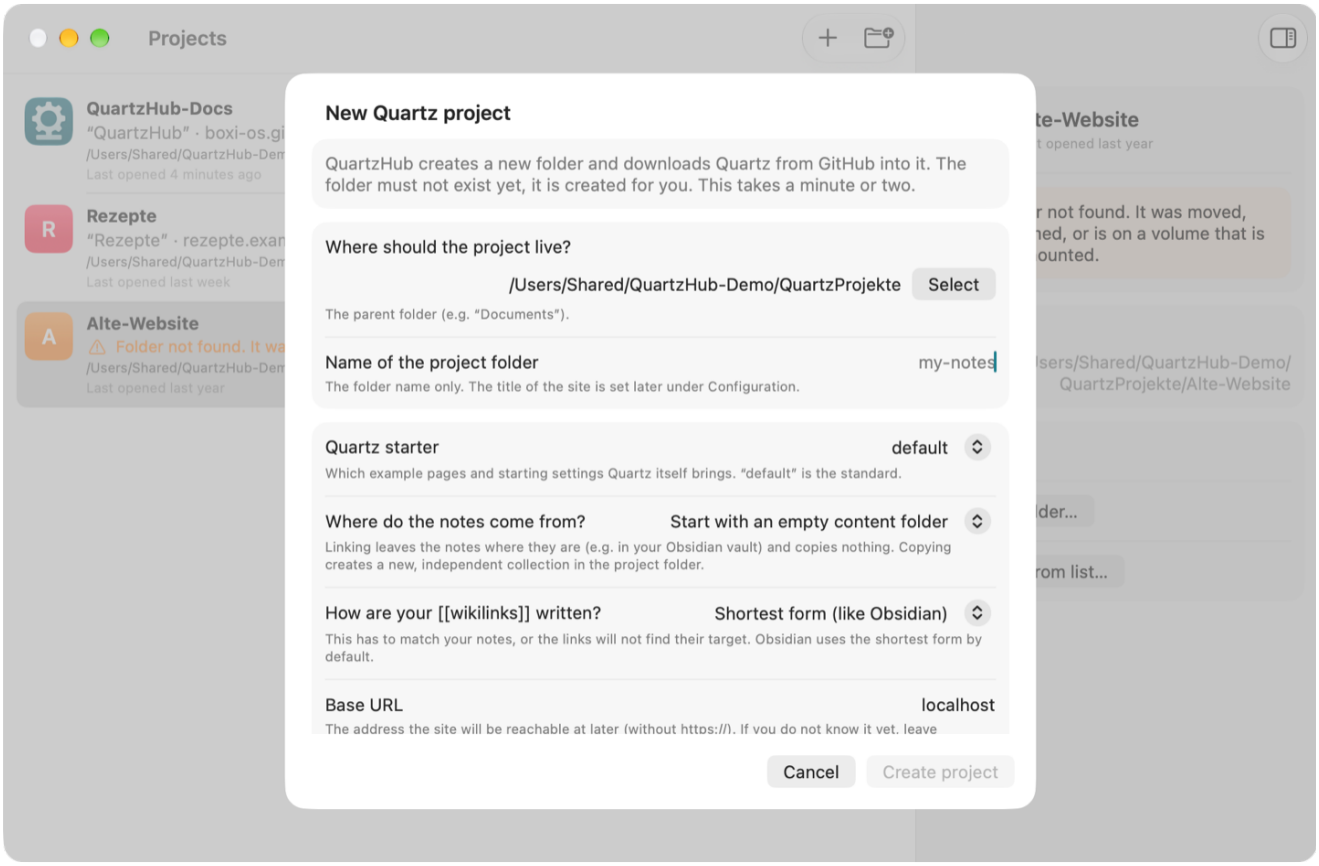
Where a page is split into **areas one above the other**, such as a list with the site's preview below, a **grip** sits between them. Dragging changes the split, a double click resets it, and the window remembers it.

1.4 – Your first site

This page walks once through everything a first site needs. Each step has a chapter of its own later; here it is only about getting through once.

1. Create a project

In the **Projects** window click **Create new project**, or choose **File ▸ New Project…** (⌘N) from any window.



The "New Quartz project" dialog.

- **Where should the project live?** The folder in which the project folder is created, such as "Documents".
- **Name of the project folder**, such as `my-notes`. This is only the folder's name; the site's title is set later.
- **Base URL**: the address the site is to be reachable at one day, without `https://`. If you do not know it yet, leave `localhost`.
- **Where do the notes come from?** For a first try, **Start with an empty content folder**. With an Obsidian vault, choose **Point at existing notes**: the notes stay where they are and QuartzHub reads them there (4.2).
- **Install the basic template** gives the site a finished design: colours for light

and dark, custom page grids, fonts. **With example pages** adds a few pages that show what it looks like before your own notes are there.

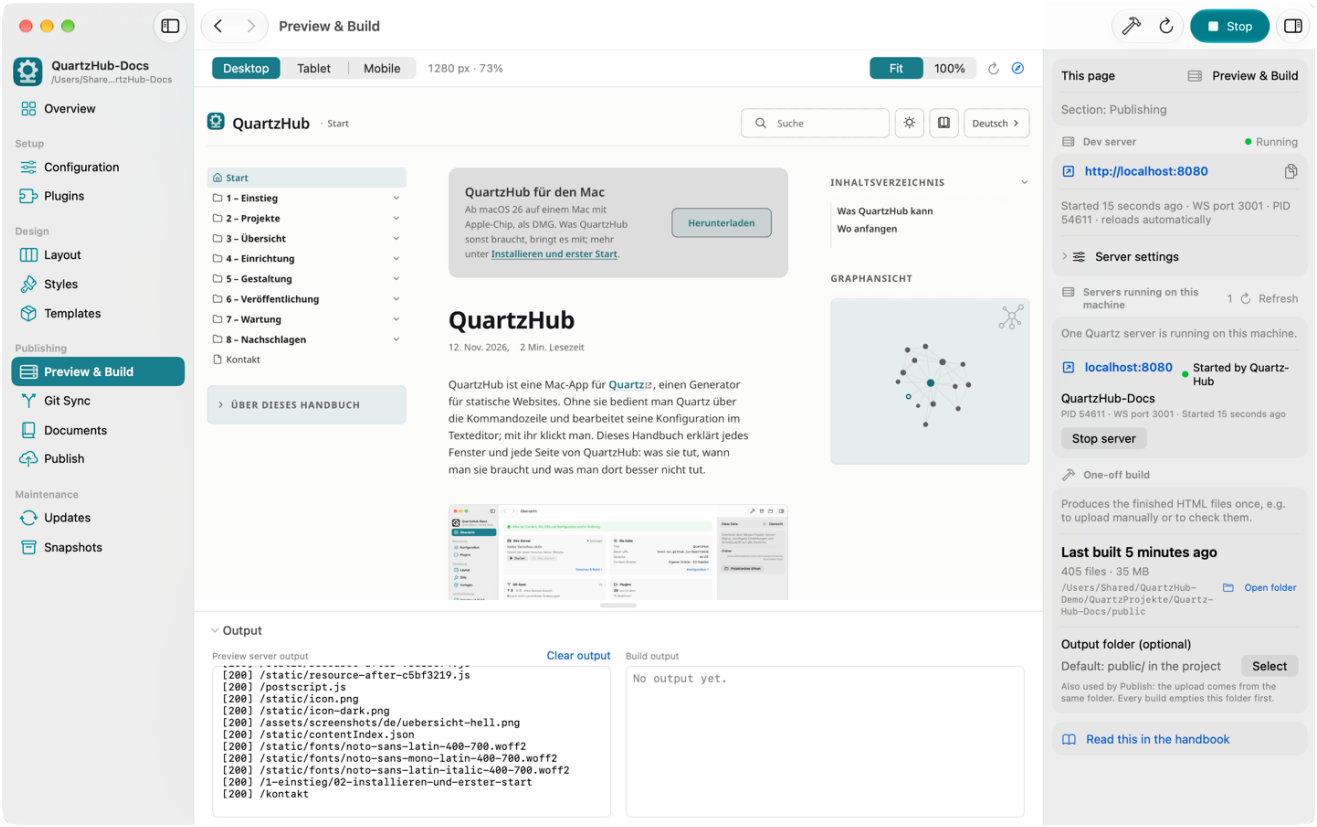
Create project loads Quartz from GitHub into the new folder and installs what it needs. That takes one or two minutes. Then the new project's window opens.

2. Read the overview

The project window opens on the **Overview**. Its cards say how the project is doing: whether the dev server is running, what the site is called, whether plugins are active, when it was last built. "All good" at the top means there is nothing to do (3.1).

3. Start the preview

Click **Preview & Build** in the sidebar, then **Start** in the toolbar (⌘R). Quartz builds the site and keeps it ready as a preview; after a few seconds it appears in the middle of the window.



The preview in the window. At the top the width (desktop, tablet, mobile), at the bottom Quartz's output.

The preview rebuilds on every change, to the notes as well as to the settings. So you can write in Obsidian and watch (6.1).

4. Title and look

- **Configuration** ▸ **Site** holds the **site title** and the **base URL**; **Save** (⌘S) writes them (4.1).
- **Styles** ▸ **Basics** sets colours and fonts (5.5), **Layout** what goes where on the page (5.2).

5. Publish

Once the preview looks right, the site goes online. **Publish** knows several ways: GitHub Pages, a web space over SFTP or FTP, a folder, a webhook. Which one fits is described in 6.7.

Before changes to configuration, plugins, frames and styles and before every update, QuartzHub takes a snapshot by itself, for the configuration at most one every 15 minutes. If something goes wrong while setting up, it leads back to the state before (7.2).

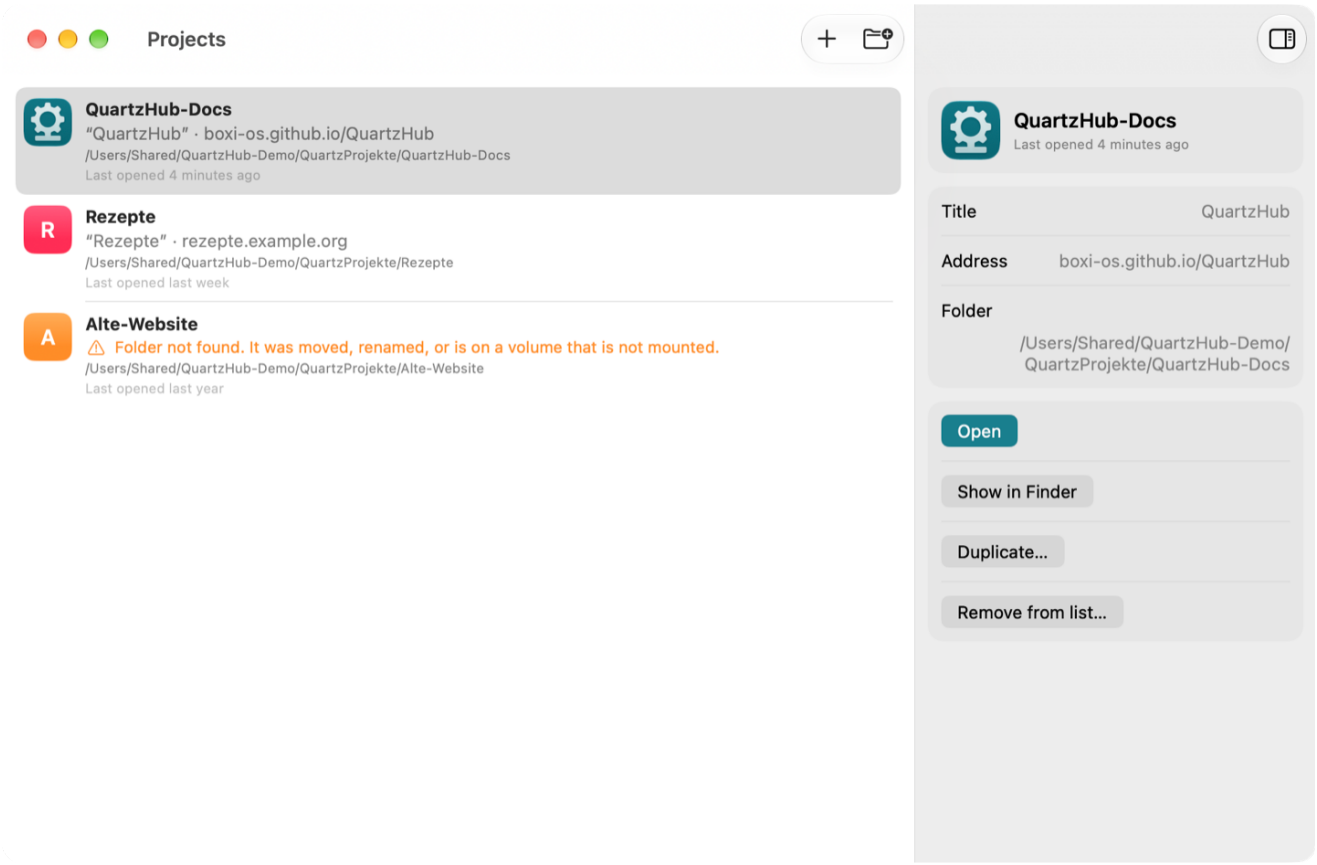
2 – Projects

A project is a folder holding a Quartz site: configuration, stylesheets, plugins and the content folder with the notes. QuartzHub keeps a list of these folders; the folders themselves live wherever you like.

1. **The Projects window**: the list, what its rows say and what to do when a folder is missing.
2. **Creating a new project**: every question of the dialog and what is in the folder afterwards.
3. **Opening an existing project**: a cloned repository or a Quartz set up by hand.
4. **Duplicating a project**: what comes along, what stays and why.
5. **The settings**: appearance, language, project folder, running servers at quit, runtime.

2.1 – The Projects window

The **Projects** window opens at start. From any other window, **Window ▸ Projects** (⇧⌘0) leads back to it.



Three projects, the top one selected. On the right the inspector with its details and actions.

The rows

Each row is a project:

- **Image and name.** The name is the name of the project folder. The image is the project image (4.3); without one, the first letter stands on a colour.
- **Title and address** of the site, as the configuration has them.
- **The folder** on disk. A path too long for the row is shortened in the middle; the whole one is in the tooltip and in the inspector.
- **When the project was last opened.**
- **"Dev server on port …"** while this project's preview is running.

What was opened last comes first; projects without that date follow by name. From six projects on, a search field appears in the toolbar; it searches name, folder and title.

QuartzHub reads the list again whenever it comes back to the front. Rename a project

folder in the Finder, and the list shows it when you switch back.

Opening

Double-clicking a row opens the project in a window of its own, as do **Open** in the inspector and the Return key. If the project is open already, its window comes to the front.

The inspector

With a row selected, the inspector shows title, address and the whole folder path, and below them the actions:

- **Open**
- **Show in Finder**
- **Duplicate…** (2.4)
- **Remove from list…**

The same commands are in a row's context menu.

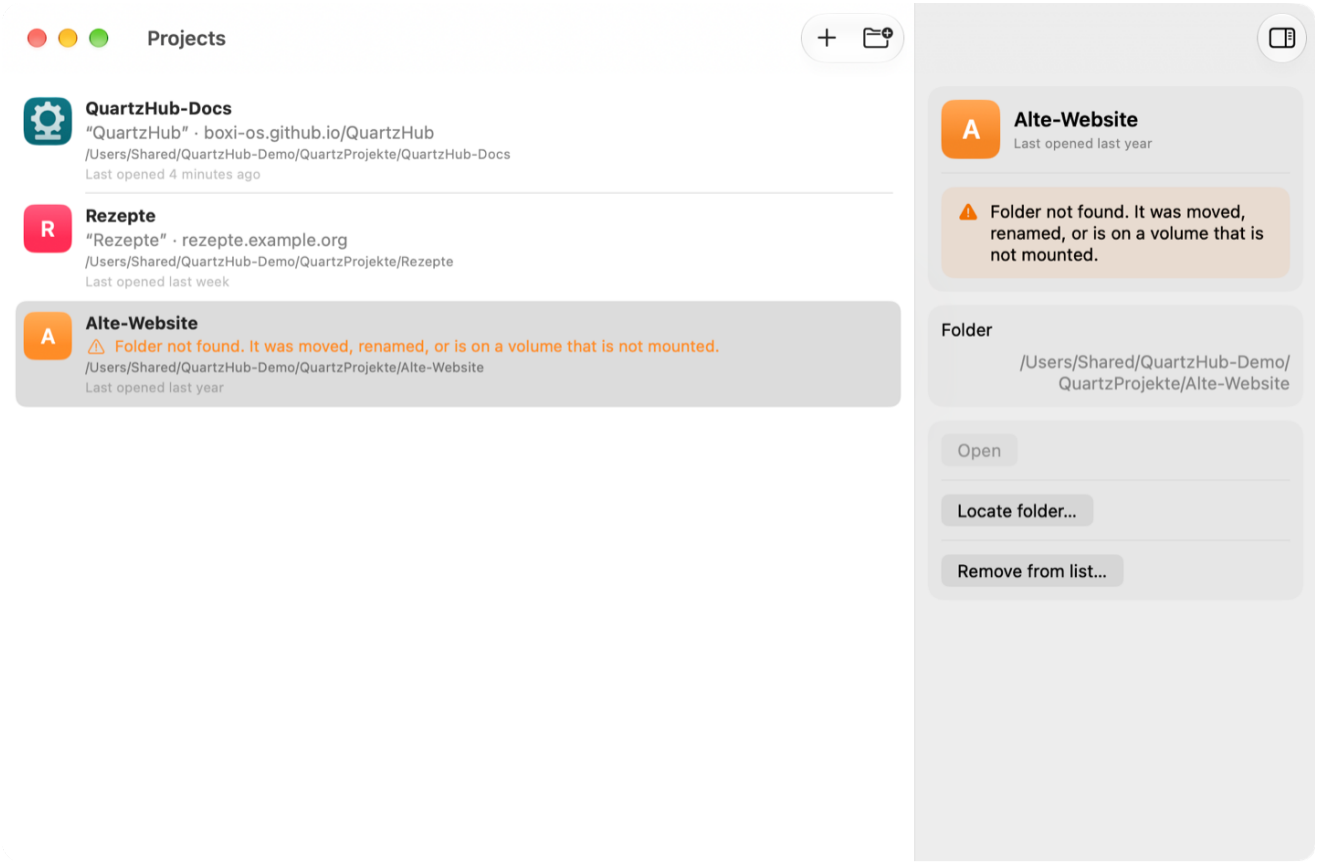
Without a selection, the inspector says whether the tools are ready (1.2), briefly explains what Quartz is, links to this handbook and shows what you can do in a project.

When GitHub has a newer version of QuartzHub, a band above the list names it, and **Download** opens its page. Nothing is installed by itself.

New and existing

The two buttons in the toolbar create a project (➕, 2.2) or add a folder in which Quartz is already set up (2.3). From any window, **File ▸ New Project…** (⌘N) and **File ▸ Open Existing Project…** (⌘O) do the same.

When a folder is missing



A project whose folder is gone.

A line in orange says that something is wrong with the folder:

- **Folder not found.** The folder was moved, renamed, or is on a volume that is not mounted right now. For a volume, connecting it is enough. Otherwise **Locate folder…** in the inspector shows QuartzHub the new place. The entry stays the same and is named after the new folder afterwards.
- **Not a Quartz project.** The folder has no `quartz.config.yaml`. Such an entry cannot be opened; it comes about when the wrong folder is chosen under "Open existing project".

Removing from the list

Remove from list… asks once and then takes the project off the list. **The folder on disk is left untouched**; opening it again brings the project back.

Two things are lost on the way, and the question names them when they apply:

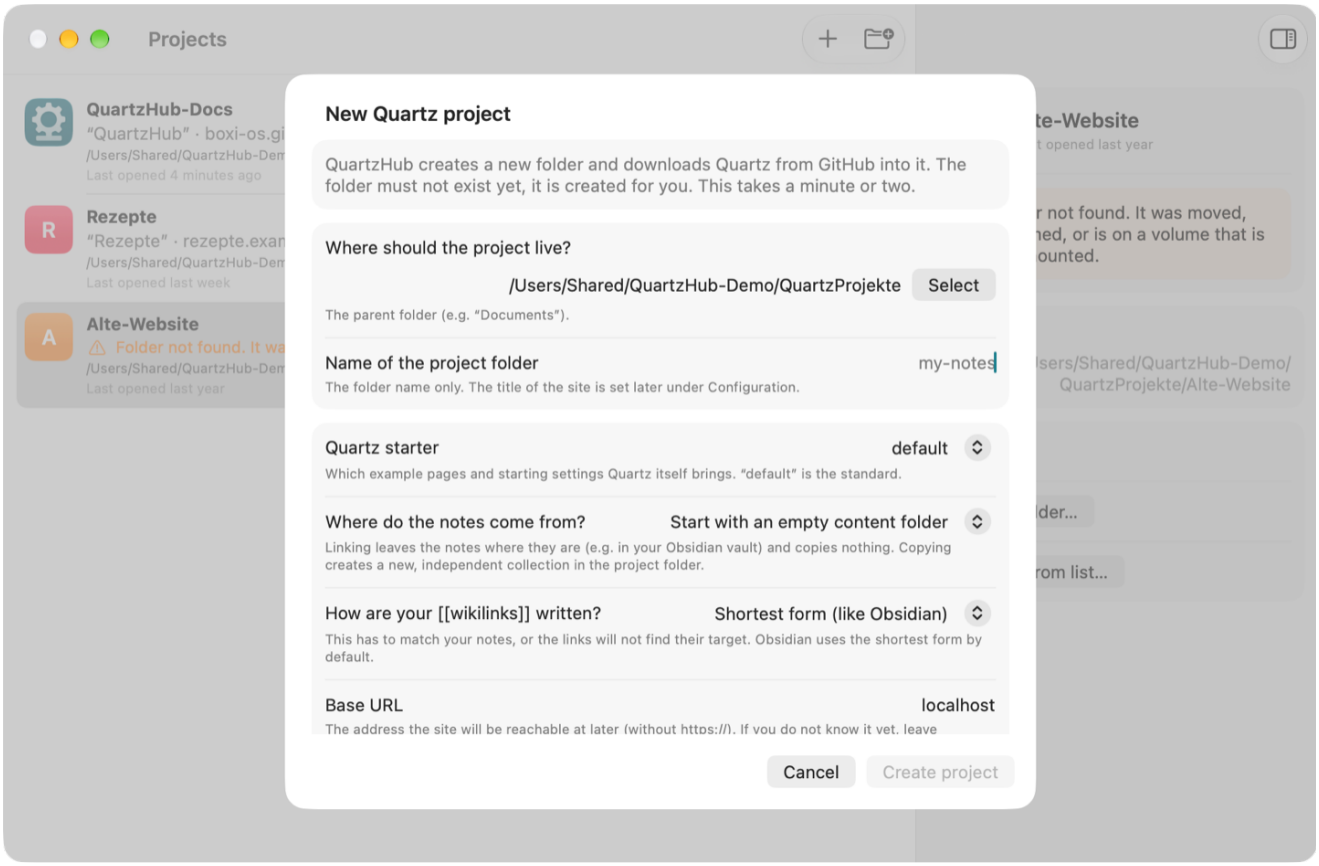
- **A running dev server** of this project is stopped.
- **The recipient addresses and captcha secrets** of the form relay (4.9) are deleted, because they belong to the list entry and do not live in the project folder. Adding the project again means entering them again.

When tools are missing

If `node`, `npm` or `git` do not answer, a note in orange appears above the list: which tool is missing or will not run, and what does not work without it. Node and npm are part of QuartzHub; if they are missing, this installation is incomplete, and reinstalling QuartzHub is the quickest fix. **Check again** asks the tools once more, say after installing git.

2.2 – Creating a new project

Create new project in the **Projects** window, ➕ in its toolbar or **File ▸ New Project…** (⌘N) open the “New Quartz project” dialog. 1.4 goes through it briefly for a first try; this page has every question.



The dialog with its defaults.

Place and name

- **Where should the project live?** The folder in which the project folder is created. It starts at the default project directory from the settings (2.5).
- **Name of the project folder.** The folder name only; it is also the name in the project list. It must not contain “/” and cannot be “.” or “..”. The site’s title is set later under **Configuration ▸ Site**.

Below, the dialog shows which folder will be created. **That folder must not exist yet**; QuartzHub creates it.

Quartz starter

Quartz brings a few starters: example pages and starting settings for a particular

purpose. **default** is the standard. The others (**obsidian**, **ttrpg**, **blog**, **docs**) set Quartz up for such a purpose; what exactly they do is in the **Quartz documentation**. The starter is something else than the basic template further down.

Where do the notes come from?

- **Start with an empty content folder.** The project folder gets a content folder of its own, empty.
- **Copy existing notes.** The notes from the **source folder** are copied into the project's content folder once. After that they are two separate collections: what changes in the source folder does not arrive any more.
- **Point at existing notes (symbolic link).** The content folder becomes a link to the source folder, an Obsidian vault for instance. The notes stay where they are, and every change there shows in the preview at once. QuartzHub writes nothing into the vault.

Switching between folder and link later is possible without losing a note (4.2).

How are your `[[wikilinks]]` written?

Quartz has to know how a link like `[[Page]]` is meant:

- **Shortest form (like Obsidian)**: just the name, as long as it is unique. This is Obsidian's default.
- **Absolute (from the root folder)**: the path from the content folder, such as `[[Folder/Page]]`.
- **Relative (from the current note)**.

The setting has to match the notes, or the links will not find their target. In Obsidian, this is the setting for the format of new links.

Base URL

The address the site is to be reachable at one day, without `https://`, such as `notes.example.org` or `name.github.io/project`. Quartz builds links, the RSS feed and preview images from it. If you do not know it yet, leave `localhost` and fill it in later under **Configuration ▸ Site**.

Basic template

Install the basic template gives the site a finished design: colours for light and dark, custom page grids (**frames**), typefaces the site serves itself, and every component styled individually. It adds the quartz-forms plugin with a contact form that sends nothing yet (4.9).

With example pages adds a few pages that show what the site looks like before your own notes are there; the dialog says how many. That only works with an empty content

folder: when copying, the example pages would overwrite notes of the same name, and QuartzHub never writes into a linked vault.

QuartzHub loads the basic template from GitHub, keeps it and asks again after a day. If it could not be loaded, the dialog says so and offers **Try again**; without the template the project starts with Quartz's own defaults. The template can still be applied later (5.9).

What happens on creating

Create project takes a minute or two. Meanwhile the bottom of the dialog says "Creating…". In that time QuartzHub

1. downloads Quartz from GitHub into the new folder, as a git repository with its whole history. The connection to Quartz's repository is cut afterwards, so a later push never lands there; updates from Quartz still come (7.1);
2. installs the npm packages Quartz needs;
3. sets Quartz up with the answers from the dialog;
4. builds the site once, so that plugins which install something on their first build are ready the next time;
5. applies the basic template, if chosen. The site's language then becomes QuartzHub's.

If one of the first three steps fails, the dialog stays open and says what did not work; the project list then has nothing new. Otherwise the project is there: it is in the list, and its window opens. If something went wrong with the template or there is something to report about it, the dialog says so first and leads on with **Go to the project**. The project is usable in that case too.

What is in the folder

`quartz.config.yaml`	the configuration: title, address, plugins
`content/`	the notes, a folder or a link
`quartz/`	Quartz itself, with the stylesheets under `quartz/styles/`
`node_modules/`	the npm packages; large, but reinstallable at any time
`.quartz-gui/`	what QuartzHub keeps about the project: frames, snapshots, targets, books
`public/`	the built site, new after every build

Everything except `.quartz-gui/` is an ordinary Quartz project. What lives there is described in 8.2.

2.3 – Opening an existing project

Open existing project adds a folder in which Quartz is already set up to the project list. The command is in the **Projects** window (the folder symbol in the toolbar), among the first steps of the empty window, and in the menu as **File ▸ Open Existing Project…** (⌘O). The folder dialog starts in the default project directory (2.5).

Which folder

The folder that holds `quartz.config.yaml`, not one above or below it. It is added and opened at once; in the list the project is named after the folder.

If there is no `quartz.config.yaml`, QuartzHub still adds the folder to the list but does not open it, and the row says **Not a Quartz project**. **Remove from list…** gets rid of such an entry; it changes nothing about the folder.

If the folder is in the list already, no second entry appears; the existing project opens.

Where a project can come from

From a git repository. A project created on another Mac and pushed with **Git Sync** (6.5) is fetched with `git clone` and then opened here. A fresh clone lacks the npm packages, because `node_modules/` does not travel; the **Overview** says so and fetches them with **Install** (3.1). The downloaded plugins are missing as well; **Plugins ▸ Installed** restores them from `quartz.lock.json` (4.5).

Set up by hand. A Quartz cloned and set up following Quartz's own instructions is a project like any other to QuartzHub. Opening it changes nothing. Only when you set something does QuartzHub write, and then into exactly the file you would have changed by hand.

A content folder that is a link

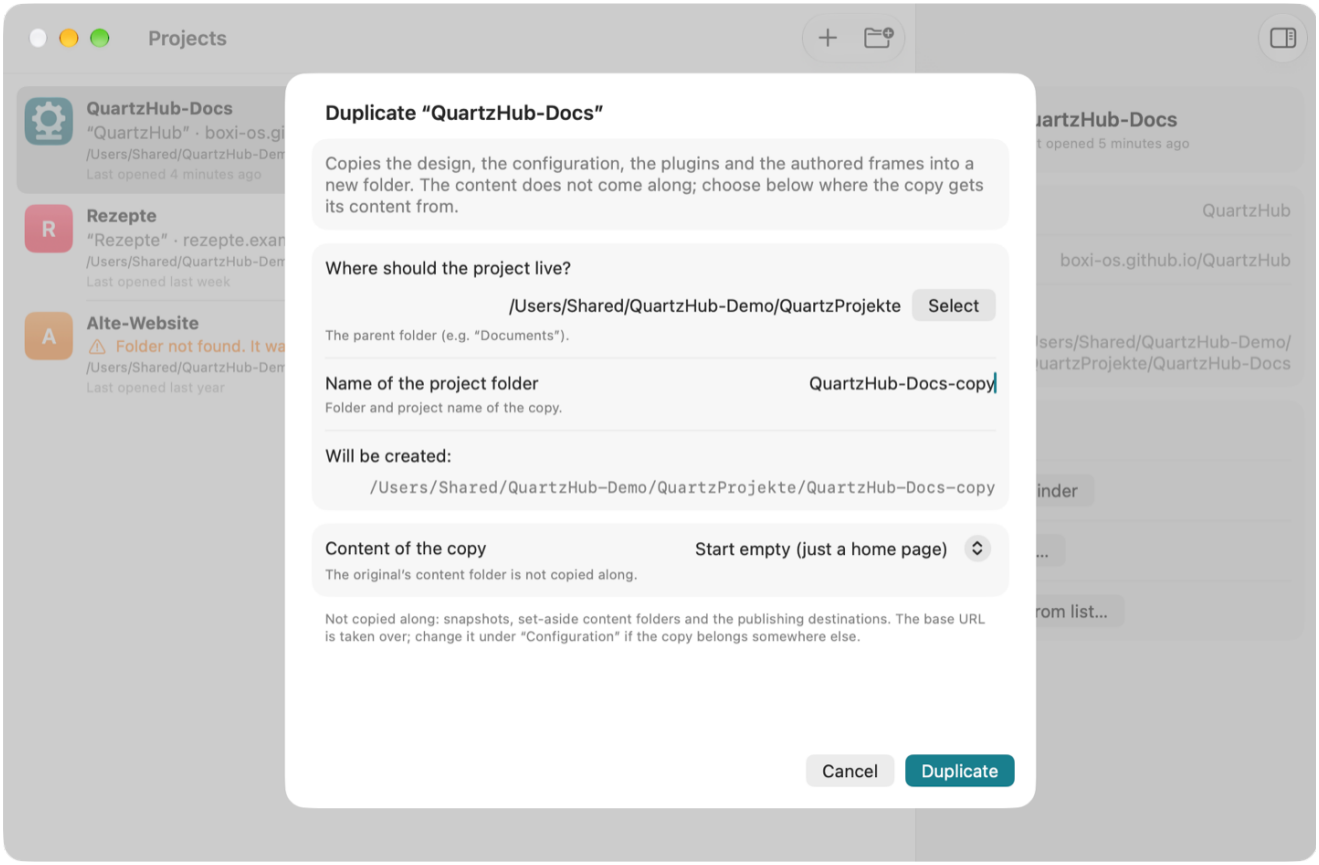
If `content/` is a link to a vault, QuartzHub follows it. If the vault is on a volume that is not mounted, or was moved, the Overview says so; the content folder can be linked again under **Configuration ▸ Site** (4.2).

2.4 – Duplicating a project

A copy is a new project with the design of an existing one. What it is good for:

- **a second site with the same look**, for another topic or another group;
- **a try without risk**: an update, a new plugin, another layout, without touching the original.

Duplicate… is in the inspector of the **Projects** window when a project is selected, and in its row's context menu.



The Duplicate dialog. At the bottom it says what is not copied along.

Place, name and content

- **Where should the project live?** and **Name of the project folder** as when creating (2.2). The target folder must not exist yet, must not lie inside the original, and the original must not lie inside it.
- **Content of the copy.** The original's content folder never comes along. The choices are:
  - **Start empty (just a home page)**: a content folder with an `index.md` that carries the project's name and says where to link notes; without it the site would show a 404 page;

- **Link to a folder (an Obsidian vault, for instance)**: the content folder becomes a link, as when creating;
- **Copy a folder in**: the notes from the folder are taken over once.

What comes along and what stays

Comes along	Stays with the original
Quartz, the configuration including the base URL, the stylesheets	the content folder
the installed plugins	the snapshots
the custom frames, theme presets and breakpoints	set-aside content folders
the git repository with its history	the publishing targets and what QuartzHub knows about earlier uploads

The targets stay behind for a good reason. A copy that inherited the original's target would overwrite its site at the first **Publish now**. The copy gets targets of its own (6.7). The connections, meaning passwords and tokens, apply to all projects anyway.

The base URL comes along. If the copy is to live at another address, change it under **Configuration ▸ Site** (4.1).

The git repository comes along without its connection to the outside. The history stays, so updates from Quartz can still be merged; but a `git push` of the copy does not land in the original's repository. For Git Sync, connect it to one of its own (6.5).

What happens on duplicating

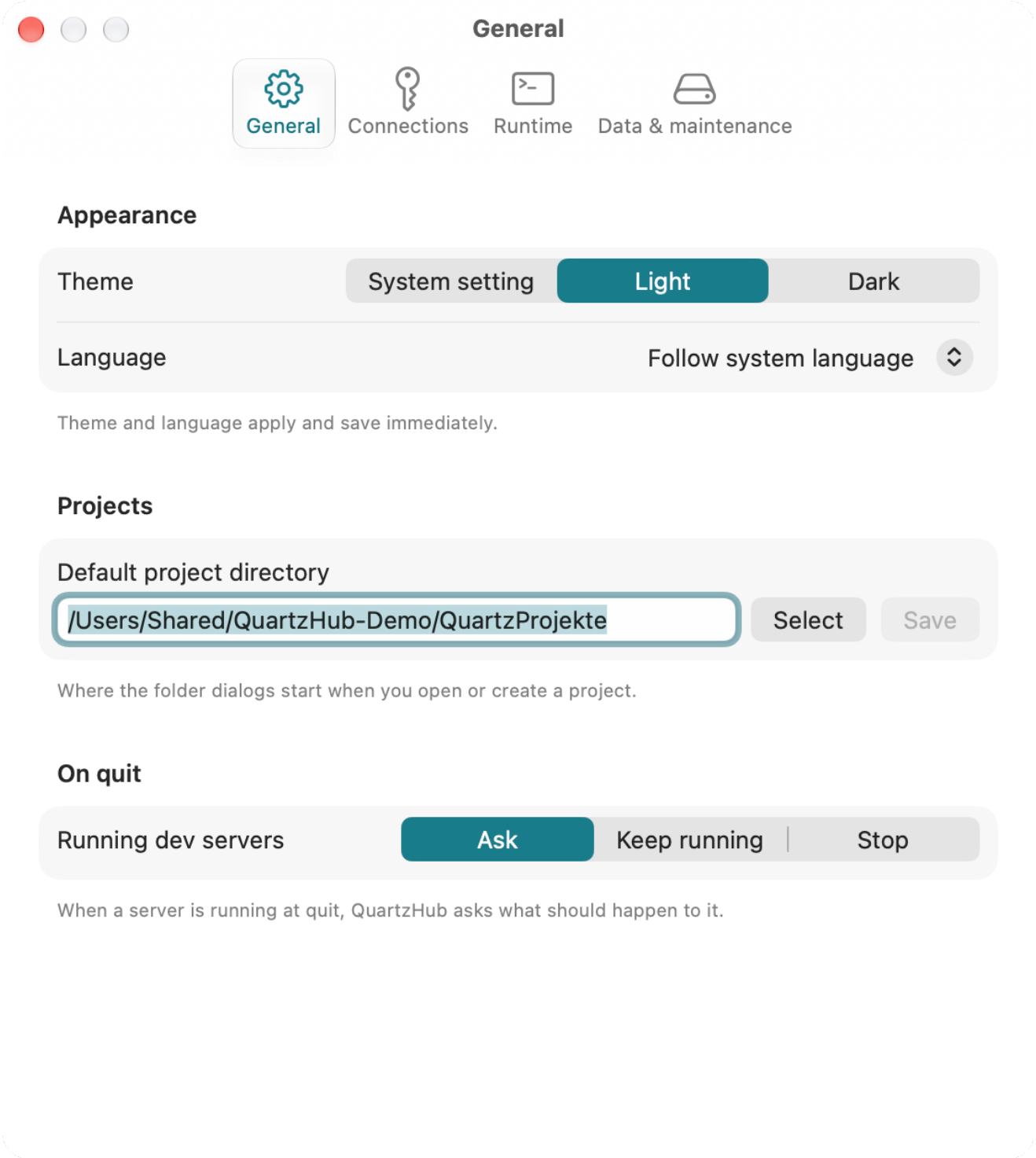
The bottom of the dialog says "Copying…". QuartzHub copies what comes along, sets up the content and installs the npm packages afresh instead of copying `node_modules/`; that takes a while. Then the copy is in the list, and its window opens. It builds the site at the first preview.

If something fails, the dialog stays open and gives the reason.

2.5 – The settings

The settings apply to QuartzHub as a whole, not to one project. **QuartzHub ▸ Settings…** (⌘,) opens them in a window of their own with four tabs. What you change there applies at once; only the default project directory and the connections have a button to save.

General



Appearance

- **Theme**: **System setting**, **Light** or **Dark**. With “System setting” QuartzHub follows the Mac.
- **Language**: **Follow system language**, **Deutsch** or **English**. The language is QuartzHub’s own, not the sites’; theirs is in their configuration (4.1).

Projects

- **Default project directory**: where the folder dialogs start when you create, open or duplicate a project. **Select** picks the folder and saves it at once; a typed path is saved with **Save** or the Return key.

On quit

Running dev servers says what happens to previews still running when QuartzHub quits:

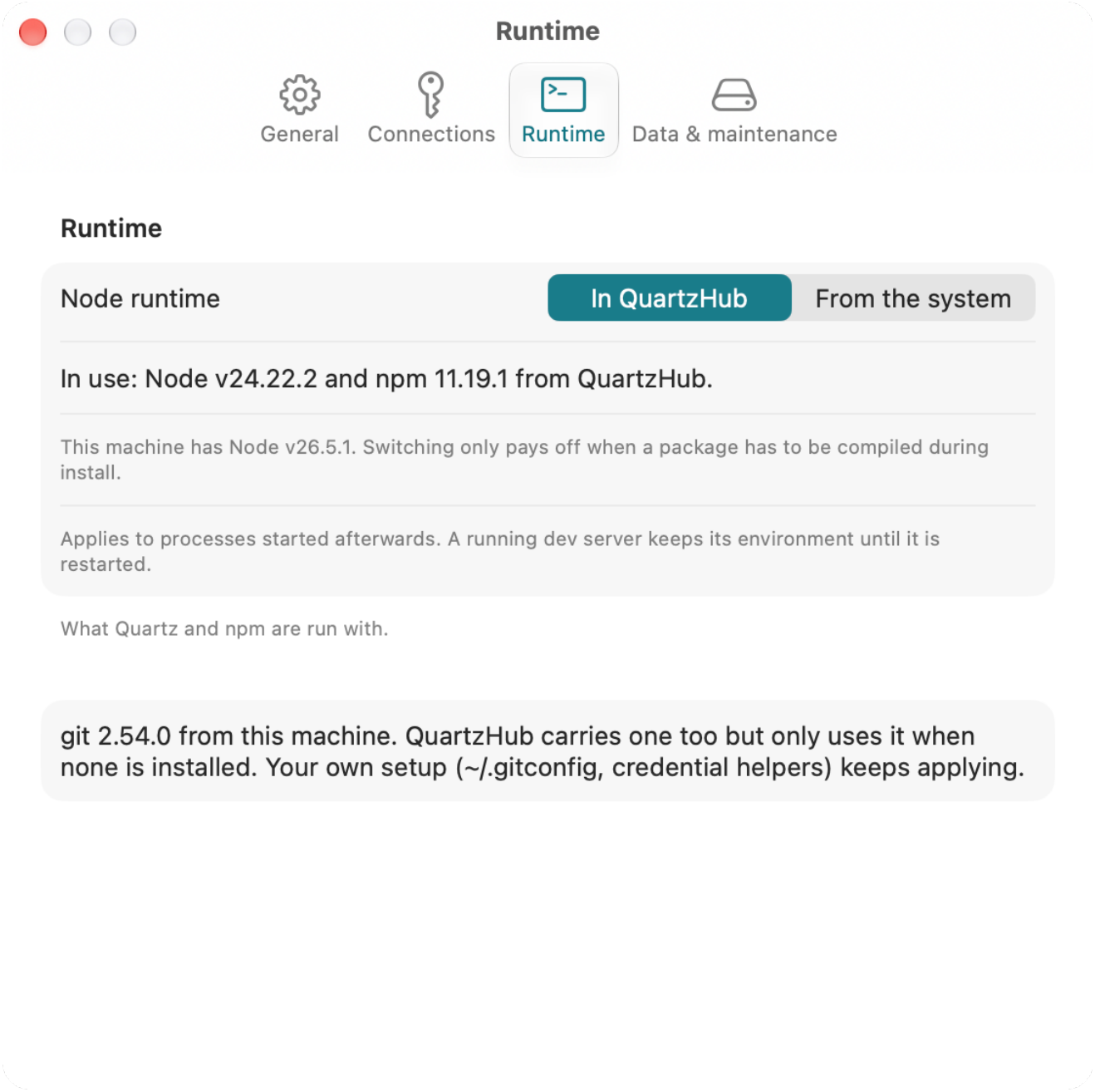
- **Ask**: QuartzHub asks every time. The question offers **Cancel**, **Keep running** and **Stop servers**; with **Don’t ask again** the chosen answer becomes the setting.
- **Keep running**: the servers keep running and stay reachable in the browser. At the next start QuartzHub finds them again and asks whether they should keep running or stop.
- **Stop**: the servers are stopped without asking.

Regardless of this, QuartzHub always asks at quit while an update, the restoring of a snapshot or a publish is running: stopping halfway can leave the project half changed.

Connections

This holds the personal access token for GitHub and the connections to servers (SFTP, FTP) and webhooks. They apply to all projects; which project publishes where is set in the project under **Publish**. 6.7 describes both.

Runtime



Node runtime says what Quartz and npm run with:

- **In QuartzHub** (the default): the Node QuartzHub brings. It is the same on every Mac and fits Quartz.
- **From the system**: the Node installed on the Mac. That only pays off when a package has to be compiled during install and needs a particular Node for it. Quartz requires Node 22 or newer. Tools such as nvm, fnm, asdf or mise pick the Node per folder, so the runtime can then differ from project to project. If QuartzHub finds no Node on the Mac, it says so beforehand.

A change applies to everything started afterwards. A running preview keeps its Node until it is restarted.

Below, it says which **git** QuartzHub uses. If the Mac has one installed, QuartzHub takes that one, so your own setup (`~/.gitconfig`, credential helpers) keeps applying; otherwise the one it brings. There is no switch for it.

Data & maintenance

- **Theme documentation**: labels and descriptions of the options of community themes (**5.6**), fetched from GitHub once and kept. **Clear cache** makes QuartzHub load them afresh next time, say after a theme changed its options.
- **Storage location**: the folder in which QuartzHub keeps the project list, the settings and the connections, with **Show in Finder**. Normally that is `~/Library/Application Support/QuartzHub`. The connections' passwords and tokens are not there but in the macOS keychain (**8.2**).

If QuartzHub could not read one of its own files at startup, say because it was killed while saving, this tab says so at the top. The file was moved aside instead of overwritten; whatever it held is missing from QuartzHub and has to be set up again.

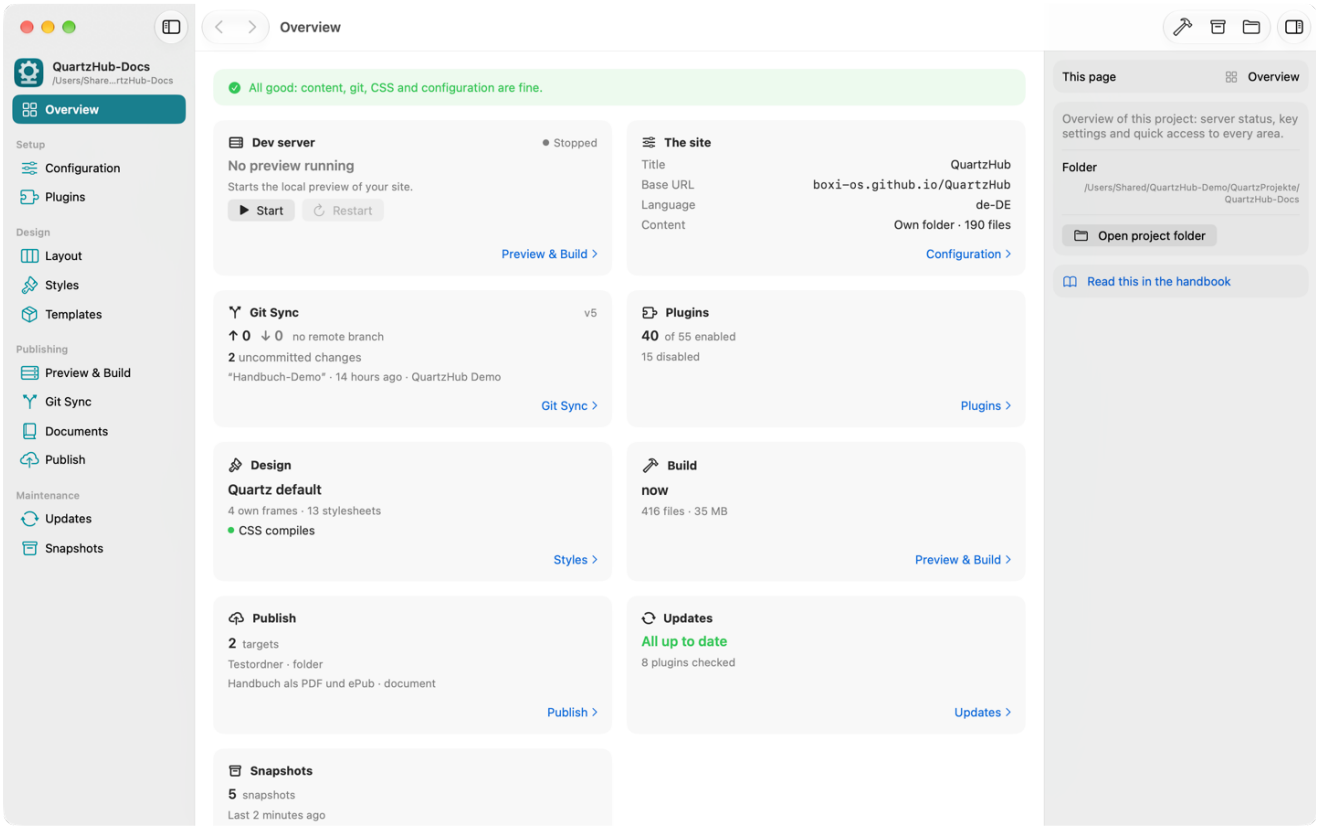
At the very bottom are the versions of QuartzHub and macOS, for a bug report.

3 – Overview

Every project window opens on the **Overview**. It answers two questions: do I need to do something? And how does the project stand right now? For anything further it leads to the page in charge of it.

1. **The overview**: the “Needs attention” band, the nine cards and the three actions in the toolbar.

3.1 – The overview



The overview of a project with nothing to do, freshly built.

At the top: needs attention, or “All good”

Above the cards stands one of the two:

- **Needs attention** in orange, with the number of points. Each point says what is wrong, mostly also why, and leads with a link to the page where it can be fixed.
- **“All good: content, git, CSS and configuration are fine.”** in green.

If neither stands there, a check has not finished or could not run, say because the CSS could not be checked. No “All good” then does not mean something is broken, only that QuartzHub does not know (1.2).

These are the points the band can name:

Point	What is behind it
There is no content folder	The project has no `content/`; without notes there is no site.
The content	`content/` is a link, and its target is gone: moved, renamed or on

folder points nowhere	a volume that is not mounted ([[en/4-setup/02-the-content-folder
The site has no start page	The content folder has no `index.md`; the site's own address shows a 404 page.
No base URL set	Quartz builds links, the RSS feed and preview images from it. With the "Fonts" plugin serving the fonts, the build fails without it ([[en/4-setup/01-website
npm packages are missing	Quartz does not build without them, as with a freshly cloned project. **Install** is right in the row: it runs `npm install` and fetches the plugins from `quartz.lock.json`.
Whether all npm packages are installed could not be checked	The project's `package.json` could not be read.
Own frames are missing	Own frames live in `.quartz-gui/` and do not travel with git. Until they are back, copied over from the other computer or restored from a snapshot, Quartz builds with the default frame ([[en/5-design/04-layout-frames
The dev server exited with an error	The preview was running and crashed; what Quartz said about it is on **Preview & Build** ([[en/6-publishing/03-reading-the-output
Your CSS does not compile	A stylesheet has an error, with file and line. Quartz would fail on it ([[en/5-design/08-custom-css
Files in conflict	A git merge left conflicts; until they are resolved, nothing can be pushed ([[en/6-publishing/05-git-sync
A merge (or rebase, cherry-pick, revert) was started and never finished	git is in the middle of an operation that has to be finished or aborted.
The git status could not be read	git answered the question about the status with an error.
A Quartz core update did not run to the end	An update was interrupted, or `npm install` failed during it. Running **Run update** once resolves it ([[en/7-maintenance/01-updates

The cards

Below the band are nine cards, in one to three columns depending on the window's

width. Each starts with the one number or name it is about and ends with a link to the page that owns the topic.

Dev server. Whether the preview is running. If it is, its address stands there; a click opens it in the browser. Below, since when it has been running, the WebSocket port it reloads through, and the process number (PID); then the buttons **Start** or **Stop** and **Restart**. If the server exited with an error, it says why here. → **Preview & Build** (6.1)

The site. Title, base URL and language of the site, and the content folder: "Own folder" with the number of files, or the target of a link. "not set" and "target missing" stand out in colour. → **Configuration ▸ Site** (4.1)

Git Sync. At the top right the branch. Below, how many commits are still to be pushed (↑) and pulled (↓), how many changes are not committed yet, and the last commit with time and author. "no remote branch" means: the project is not connected to any repository on the net. The card only reads what git on the Mac knows; it does not go to the network for it. → **Git Sync** (6.5)

Plugins. How many plugins are enabled and how many there are in all. → **Plugins** (4.5)

Design. The active community theme or "Quartz default", the number of own frames and stylesheets, and whether the CSS compiles: **CSS compiles**, **CSS error** or **CSS unchecked**. → **Styles** (5.5)

Build. When the site was last built, with the number and size of the files. The card looks at the output folder for this; "Never built" means there is none. → **Preview & Build** (6.2)

Publish. How many targets are set up, the first three with name and kind. → **Publish** (6.7)

Updates. Whether there is something new for Quartz or the plugins: **Core update available**, **Plugin updates available**, **All up to date**, **Update unfinished** or **Cannot check**. Below, how many plugins were checked and how many checks failed. It is the only card that asks the network, at GitHub; without a connection it says "Cannot check", not "All up to date". → **Updates** (7.1)

Snapshots. How many there are, when the last one was taken and why, such as "Before config change". → **Snapshots** (7.2)

The toolbar

At the top right are three actions often needed from here:

- **Run build** (hammer): builds the site once, like **Project ▸ Run build** (⌘B). When it is done, the Build card says "Build finished."; if it fails, it says the output is on **Preview & Build**.
- **Take snapshot**: records the state of the project before you try something (7.2).
- **Open project folder**: shows the project folder in the Finder. The button is also

in the inspector, below the full path.

4 – Setup

The **Setup** group in the sidebar has two pages. **Configuration** sets what the site is: title, address, language, where the notes come from. **Plugins** sets what Quartz does with the notes and what appears on the pages.

Configuration

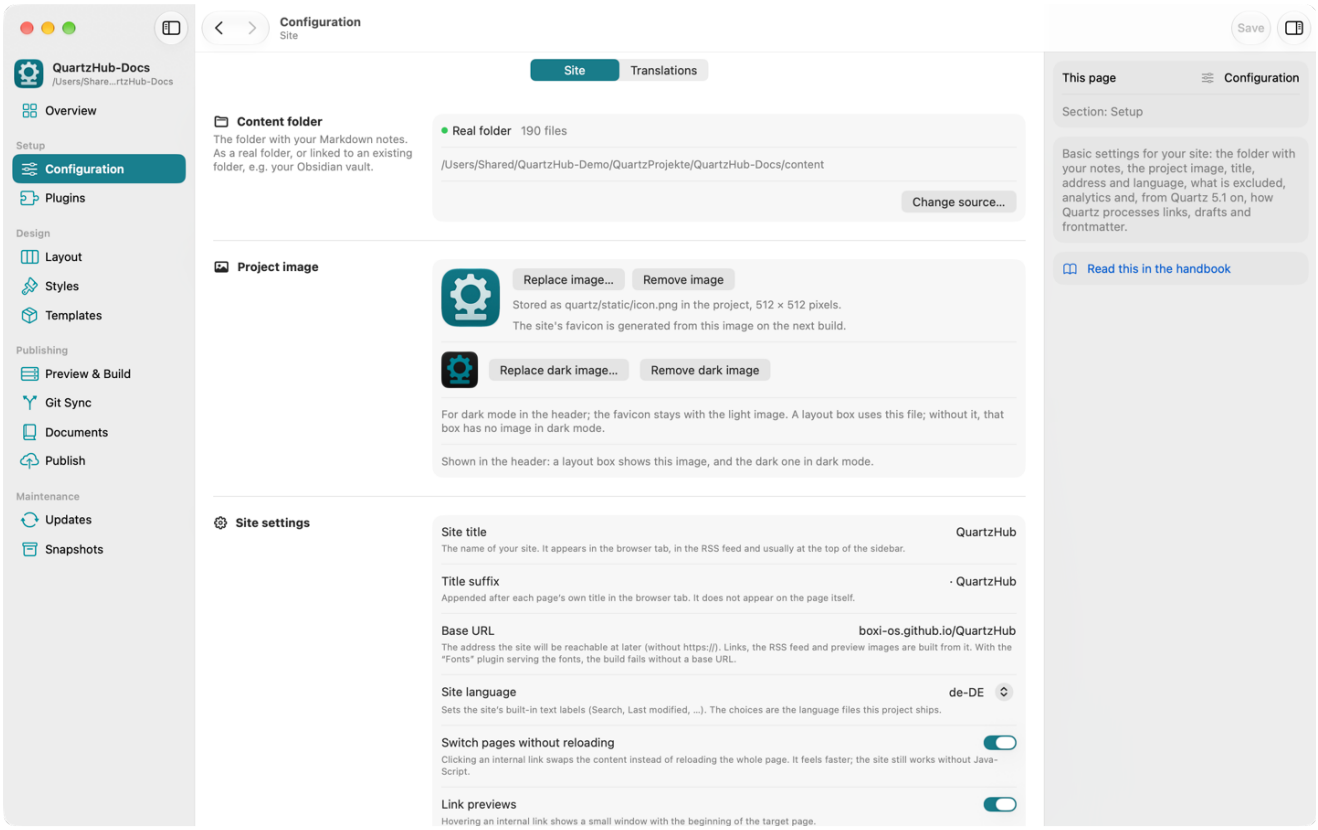
1. **Site**: title, address, language, exclusions, analytics and the processing of Quartz 5.1.
2. **The content folder**: a real folder or a link to a vault, and how to switch without losing anything.
3. **The project image**: favicon, picture in the header, light and dark version.
4. **Translations**: the site's built-in texts, such as "Search".

Plugins

5. **Plugins**: the installed plugins, their order and their options.
6. **Navigations**: menus, accordions, previous and next.
7. **Additional content**: boxes of your own with text, HTML or pictures.
8. **Languages**: one site in several languages.
9. **Forms**: contact forms and how their messages arrive.
10. **Marketplace**: finding and installing new plugins.

4.1 – Site

Configuration ▸ Site edits the project's `quartz.config.yaml`. At the top are the content folder and the project image; they have chapters of their own (4.2, 4.3), because they save differently from the rest.



Configuration ▸ Site: content folder and project image at the top, the site settings below.

Saving

Everything below the project image is written to the file only by **Save** (⌘S). Before that, QuartzHub takes a snapshot ("Before config change"), at most one every 15 minutes, so a series of small changes does not fill the list. The file is changed rather than rewritten: comments and the order of the keys stay.

Leaving the page with unsaved changes asks first (1.3).

If the file cannot be read, say because it was broken by a hand edit, the page says so with the reason and shows only the content folder. Fix the file in an editor, or get it back from a snapshot (7.3).

Site settings

- **Site title**: the name of the site. It appears in the browser tab, in the RSS feed and usually at the top of the sidebar.
- **Title suffix**: appended to every page's title in the browser tab, such as " · My notes". It does not appear on the page itself.
- **Base URL**: the address of the finished site, without `https://`. Quartz builds links, the RSS feed and preview images from it; with the "Fonts" plugin serving the fonts, the build fails without it. For GitHub Pages it is usually `name.github.io/repository` (6.8).
- **Site language**: sets the site's built-in texts such as "Search" or "Last modified". The choices are the language files the project ships; the texts themselves are changed under **Translations** (4.4). For a site in several languages see 4.8.
- **Switch pages without reloading**: a click on an internal link only swaps the content. That feels faster; the site still works without JavaScript.
- **Link previews**: hovering over an internal link shows a small window with the start of the target page.
- **Prefetch pages** (Quartz 5.1 and later): loads target pages before you click, so they appear at once: **Links on screen**, **When pointing at a link** or **Off**.

Exclude notes and folders

- **Patterns (one per line)**: files and folders in the content folder that never get into the site, such as `private/**` for a whole folder or `*.excalidraw.md` for a kind of file.
- **Publish files** (Quartz 5.1 and later): **All files in the content folder** or **Only those in use**. With "Only those in use", pictures and other files only reach the site when a published page uses them; the attachments of unpublished notes stay private.

> ⚠ **What is in the content folder is public**
>
> Quartz publishes every note in the content folder that is neither excluded nor marked as a draft, including notes nothing links to. Private notes belong in an excluded folder, or not in the content folder at all.

Analytics

Embeds a provider's counting script into every page. With **No analytics**, nothing is loaded and nothing is measured.

Provider offers Plausible, Google, Umami, GoatCounter, PostHog, Tinylytics, Cabin, Clarity, Matomo, Vercel and Rybbit. Depending on the provider, its fields appear below, named as Quartz names them, such as `websiteId` or `host`; an asterisk means "required". If a required field is missing, a note says which; saving works anyway. Switching provider drops the old provider's fields.

In many countries a counting script means the site has to tell its visitors about it. What applies there is not this handbook's subject.

Processing (Quartz 5.1 and later)

Since Quartz 5.1, Quartz does some things itself that used to need plugins: links, drafts, the content index and frontmatter. Their settings are then in the configuration. These sections only appear in a project with Quartz 5.1 or newer.

If the old plugin entries are still in the configuration, the **Processing** section says so. Quartz keeps reading them. **Move settings** has Quartz move the settings into the configuration and remove the old entries; QuartzHub takes a snapshot first. Unsaved changes have to be saved before.

Links

- **Resolve Markdown links**: how a link of the form `[text](path)` finds its note: **Shortest path (as in Obsidian)**, **Relative to the note** or **From the content folder**.
- **Remove folders from link text**: "folder/note" becomes "note".
- **Open external links in a new tab**, **Icon after external links**, **Load images and videos only when scrolled to**.
- **Mark links to missing notes**: such links get the CSS class `broken`.

Drafts

- **Publish drafts**: pages with `draft: true` in their frontmatter are left off the site otherwise.

Content index, sitemap and RSS

- **Create the content index**: without it, search, graph, sitemap and RSS are missing, and QuartzHub cannot export documents (6.6).
- **Sitemap**, **RSS feed**, the **Items in the RSS feed**, **Whole page in the RSS feed** (otherwise only the description), **Feed name** ("index" gives `index.xml`) and **Include pages without text**.

Frontmatter

- **Delimiters**: the characters above and below the frontmatter, usually `---`.
- **Language**: YAML or TOML.

If part of the processing is turned off in the configuration, its section is grey and says why.

At the very bottom the page points to **Styles**: colours and fonts are there (5.5).

4.2 – The content folder

The content folder is `content/` in the project folder: the Markdown notes Quartz builds the site from. It is at the very top of **Configuration ▸ Site**. Unlike the rest of the page it is not part of `quartz.config.yaml`; everything here applies at once, without **Save**.

Folder or link

The content folder is one of the two:

- **Real folder**: the notes live in the project. Good when the site has a collection of its own.
- **Symbolic link**: `content/` points to a folder elsewhere, usually an Obsidian vault or a folder in it. The notes stay where you write them, and every change shows in the preview at once.

The section shows which of the two applies, how many files are in it (hidden ones such as `.obsidian` do not count) and the path, for a link also its target. If the target is gone, it says so in red, and the overview reports “The content folder points nowhere”.

> **Publishing only part of a vault**
>
> The link need not point at the whole vault. A subfolder such as `Vault/Public` keeps everything else safely out. Single notes and folders are excluded under **Exclude notes and folders** (4.1).

Changing the source

Change source… opens the “Change content source” dialog:

- **New source folder**: the folder with the notes, typed or picked with **Select**.
- **Strategy**: **Link** makes `content/` a link to the folder, **Copy** copies its contents once into a real folder.

Apply then does three things:

1. It takes a snapshot (“Before content switch”). If that fails, nothing changes.
2. It **sets the previous content folder aside** instead of deleting it: a real folder is moved to `.quartz-gui/content-backups/`; for a link, only where it pointed is recorded. A linked vault is never touched.
3. It creates the new content folder, as a link or as a copy.

The source folder must not lie inside the content folder or contain it; the dialog says so if it does.

Getting set-aside content folders back

Set-aside content folders are listed under **Snapshots**, in a list of their own (7.3). **Restore** puts one of them back and sets the current one aside as well. They can be deleted there too.

> ⚠ **Set aside is not backed up**
>
> A set-aside real folder is the only copy of those notes; snapshots do not contain it. Deleting it cannot be undone. Of a link only the target is recorded; the notes stay in the vault.

When the start page is missing

If there is no `index.md` at the top of the content folder, the site's address shows a 404 page. The section says so and offers **Create start page…**: the page gets the title you enter and, below it, a list of the folders and notes at the top of the content folder right now. The list is not updated later; the page is a beginning to write on yourself.

If the content folder is a link, the file is created in the linked folder, among your own notes; the dialog says so beforehand. An existing `index.md` is never overwritten.

Content folder and git

To git, a link is only a reference. Taking the project to another Mac with **Git Sync** therefore does not take the notes along; there the content folder has to be linked again. Whether snapshots include the content folder is set under **Snapshots**; they never include a linked vault (7.2).

4.3 – The project image

The project image is under **Configuration ▸ Site**, right below the content folder. It serves three times:

- **in QuartzHub**: in the project list and at the top of the project window's sidebar;
- **as the site's favicon**, if the "favicon" plugin is enabled; the section says whether it is;
- **in the site header**, if you switch that on.

Pictures are written at once, without **Save** and without a snapshot.

Choosing an image

Choose image… takes a PNG or JPEG file, ideally square. QuartzHub stores it as a PNG at `quartz/static/icon.png`, scaled down to at most 512 pixels on the longer edge. The picture Quartz brought there is kept the first time.

Remove image puts the original picture back; if there was none, there is none afterwards.

If it says "The site shows an image that was not chosen in QuartzHub", the picture comes from a template or was copied in by hand. One of your own replaces it.

Dark version

Image for dark mode… stores a second picture at `quartz/static/icon-dark.png`. It only applies to the picture in the header while the site is shown dark; the favicon stays with the light one. **Remove dark image** deletes the file at once.

Showing it in the header

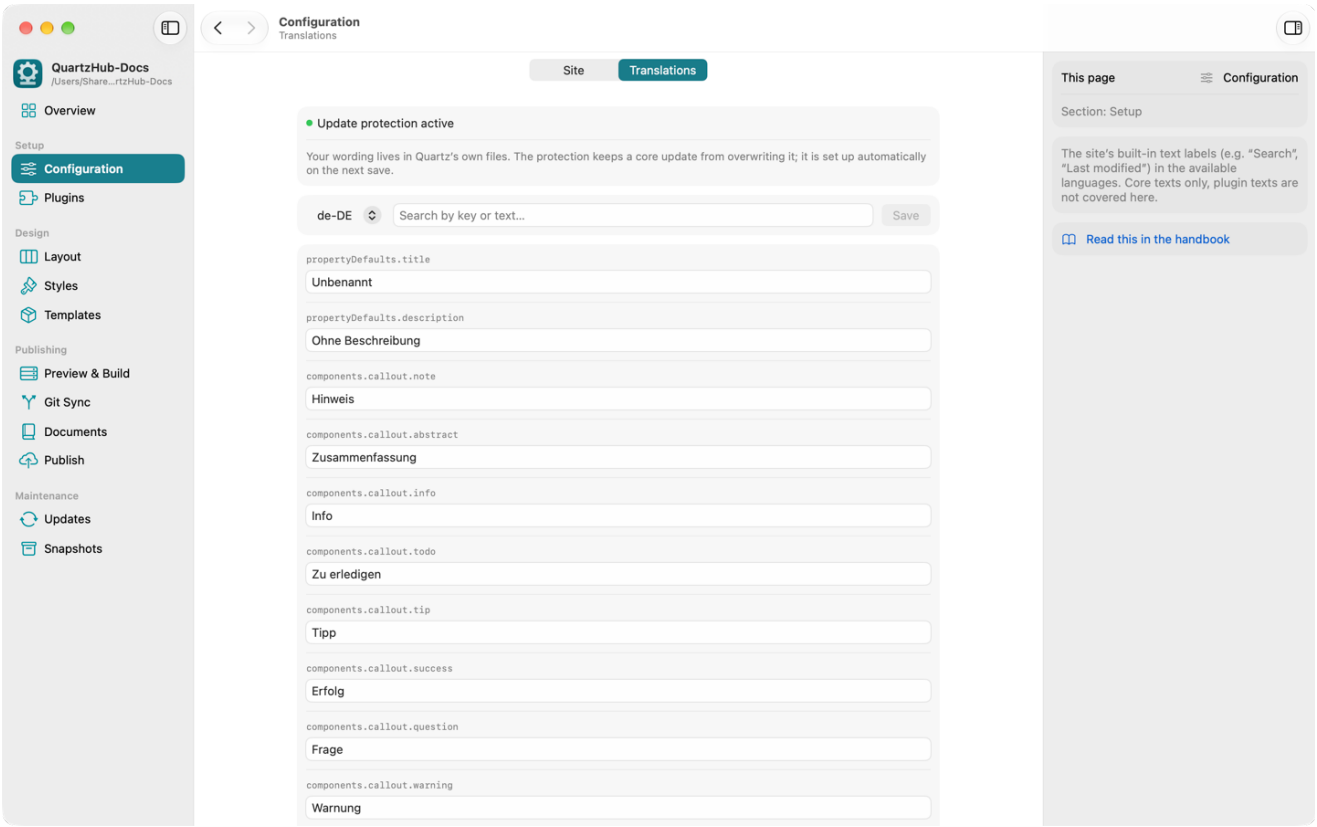
Show in the site header puts the project image on top of every page as a link to the home page, with the dark picture in dark mode. That works through the quartz-layout-box plugin. If it is missing, QuartzHub asks when you switch this on whether to install it; for that it first saves the page's unsaved changes and needs a network connection. Afterwards the switch is on, but only **Save** writes it to the configuration.

If a template's layout box already shows the picture, such as the basic template's, a note about that stands in place of the switch.

Where exactly the picture sits in the header is moved on the stage under **Layout ▸ Components** (5.3).

4.4 – Translations

For every language Quartz brings a file with the site's built-in texts: "Search", "Last modified", "Table of contents", the titles of the callouts and so on. **Configuration ▸ Translations** edits these files. Texts that plugins bring are not in them; those are set with the plugins, where they allow it.



The texts of the German language file. Update protection at the top.

Changing texts

- **Language**: the language file being edited. This only picks the file; which language the site speaks is set under **Site** (4.1).
- **Search field**: finds entries by key (such as `components.search.title`) or text.
- Each entry shows its key and the text below. Entries marked **JS** are not plain text but a piece of program code, for instance where a number is put into the text. They are saved verbatim as code; changing them means writing valid TypeScript.

Save on this tab writes all changed entries of the chosen language; next to it, it says how many are still unsaved; ⌘S does the same. An entry that could not be written keeps its change and says why below.

Unsaved changes stay when you go to another page and come back. Choosing another

language, closing the window or quitting QuartzHub asks first: save, discard or cancel.

Saving does not take a snapshot. QuartzHub does keep the state before the first change of a language, in `.quartz-gui/locale-baseline/`.

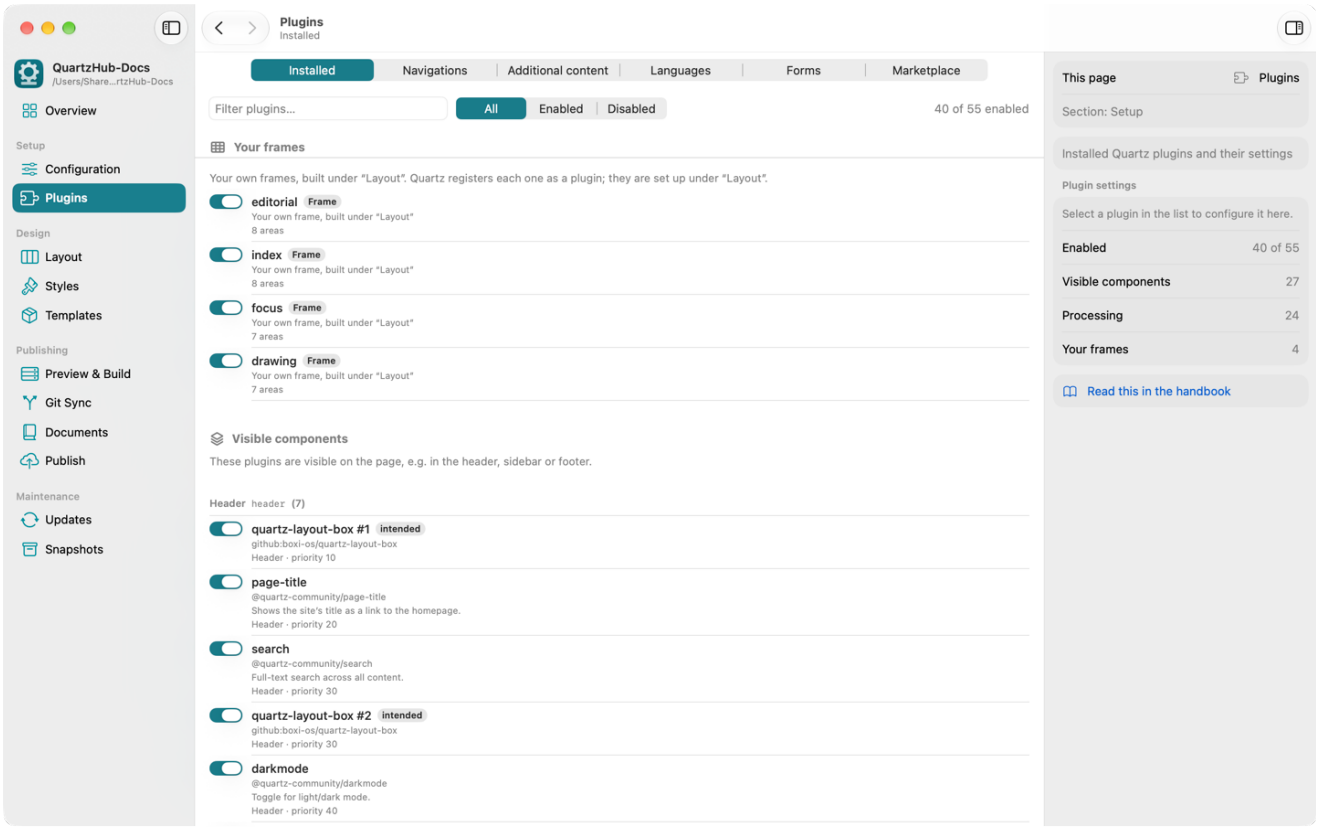
Update protection

The language files belong to Quartz. A core update (7.1) would therefore overwrite your own wording with Quartz's new texts. **Update protection** tells git to keep your version of these files when in doubt. It is set up by itself at the first save; **Enable now** sets it up right away.

The protection has a flip side: if an update changes a language file you changed yourself, your version stays whole, and new texts from the update do not get in. If a text is missing afterwards, add it here.

4.5 – Plugins

Almost everything Quartz does, a plugin does: reading Markdown, resolving links, search, the sidebar, the table of contents. **Plugins ▸ Installed** shows all plugins of the project. The other tabs of the page are editors for single plugins (4.6 to 4.9) and the marketplace (4.10).



Plugins ▸ Installed: your frames at the top, the visible components by position below.

The list

At the top, a search field and **All**, **Active**, **Inactive** filter the list; next to them it says how many are active. The plugins stand in groups:

- **Your frames**: the page grids from the layout editor. Quartz runs each one as a plugin; they are set up under **Layout ▸ Frames** (5.4).
- **Visible components**: what appears on the page, by position: **Header**, **Left sidebar**, **Right sidebar**, **Before the content**, **After the content**, **Footer** and **In the content**. Within a position by their priority.
- **Processing**: what changes the notes behind the scenes, such as formatting, links, pictures, and is not visible itself. First the **Page types**, then **Transformers, filters & emitters**, each in the order Quartz applies them.
- **Maintenance**: see below.

Each row shows the switch, the name, where the plugin comes from, a short description for known plugins, and its place (`position` and `priority`, or `order`). Disabled plugins are paler. Marks may be added:

- **Update available**: there is a newer version (7.1).
- **Built into Quartz**: since Quartz 5.1, Quartz does this itself; it is set up under **Configuration ▸ Site** (4.1).
- **Error according to Quartz**, **Note from Quartz** or **intended**: the result of Quartz's own check, see below.

Switching on and off, reordering, removing

A row's **switch** writes to the configuration at once. The order is changed by dragging, with **Arrange ▸ Move Up** and **Move Down** (⌥⌘↑, ⌥⌘↓) or in the context menu, each within a group; QuartzHub renumbers the group (10, 20, 30 …). It does not work while a filter is active.

Remove is in the context menu. It asks, takes a snapshot and then has Quartz remove the plugin. Your own frame is deleted from disk as well. The context menu also has **Open repository on GitHub**.

The options

With a plugin selected, the inspector shows its settings. **It saves without a button: switches right away, text fields when you leave them** or press Return. Every change takes a snapshot, at most one every 15 minutes.

- At the top name, source, description and **Plugin enabled**.
- **Placement** (visible components only): `position`, `priority`, `display` (all devices, mobile only, desktop only), `condition` (such as `not-index`, `has-tags`, `has-backlinks`, `has-toc`) and `group` with its options. The mouse does the same on the stage: **Show on the stage** leads to **Layout ▸ Components** (5.3).
- **Options**: a form made from what the plugin says about its options. Yes/no becomes a switch, a choice a menu, lists get buttons to add and reorder. From eight options on there is a search field; rarely used ones are under **Rarely needed**.

About the fields:

- **changed** means: the value is in the configuration. The button next to it removes it again, and the plugin takes its default.
- **no effect** means: the option only applies together with another one that is set differently right now.
- What QuartzHub cannot show as a form is edited as **JSON**. Invalid JSON is not saved.
- Values that are in the configuration but that the plugin does not describe are under **Unknown keys**.
- If a plugin does not describe its options at all, the fields come from its defaults,

and **Add another option** takes a key with a value.

QuartzHub reads the description of the options from the plugin's package, without running it.

Quartz's own check (Quartz 5.1 and later)

From Quartz 5.1 on, QuartzHub runs `quartz plugin doctor`, Quartz's own check of the plugins, after every change. What it reports appears as a mark on the row and in the inspector under **Quartz reports**, in Quartz's words. "intended" means: Quartz objects to something QuartzHub does on purpose, such as several entries of the same plugin, one per box or navigation.

If GitHub Markdown and Obsidian Markdown run in the wrong order, a note above the list says so; **Swap the order** fixes it.

Maintenance

At the very bottom are two buttons that work on the downloaded plugins in `.quartz/plugins/`:

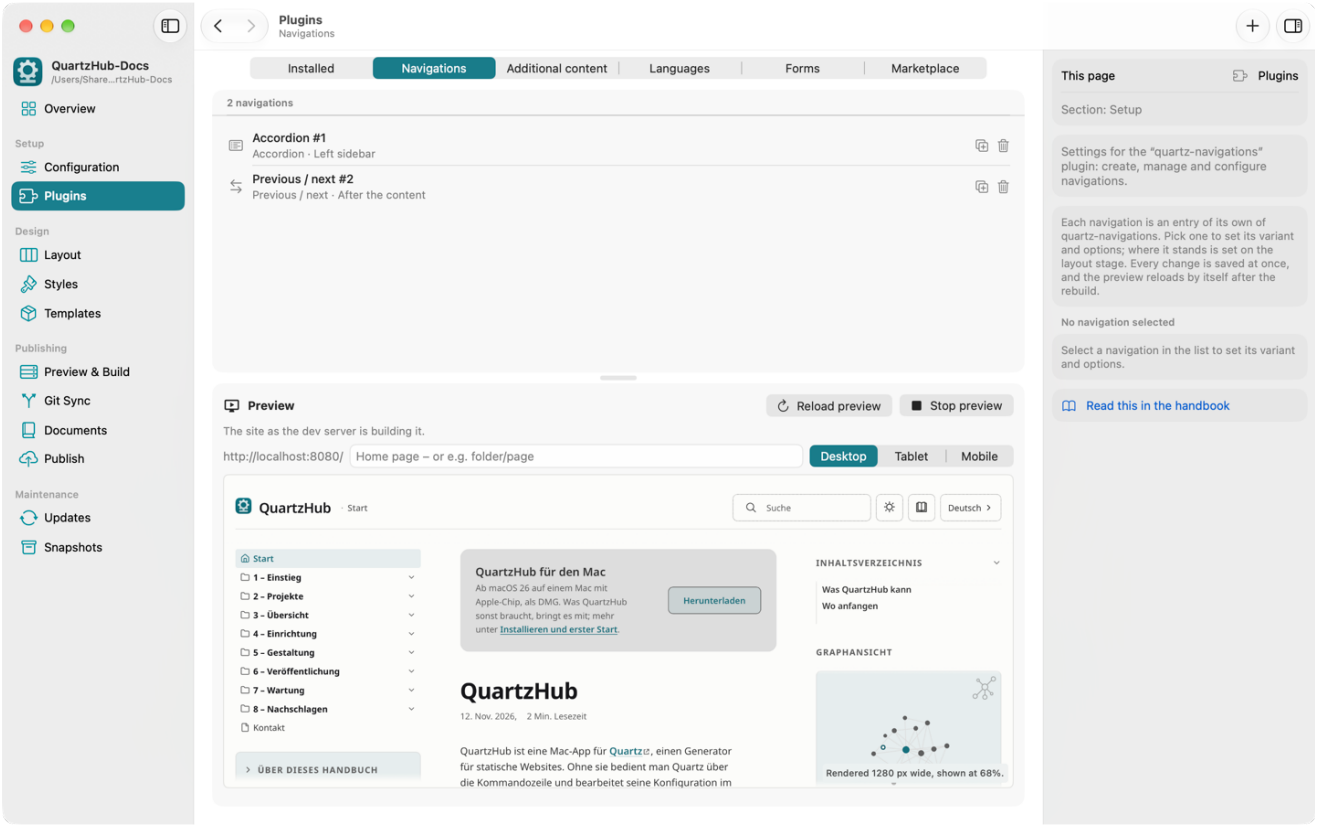
- **Restore from quartz.lock.json** downloads every plugin in the version `quartz.lock.json` records. That is needed after cloning on another Mac (2.3); missing npm packages are installed first.
- **Remove orphaned plugin folders** deletes downloaded plugins the configuration no longer points to.

Neither changes the configuration.

4.6 – Navigations

Plugins ▸ Navigations sets up the quartz-navigations plugin. It builds navigations from the folders and notes of the content folder: a tree list in the sidebar, a menu in the header, a sitemap in the footer, previous and next below the text. Each navigation is an entry of its own of the plugin; a site can have several.

If the plugin is not installed, the tab says so; it is installed in the marketplace via `boxi-os/quartz-navigations` (4.10).



The project's navigations at the top, the preview below, the inspector on the right.

How the editor tabs are built

Navigations, Additional content, Languages and Forms are built the same way:

- **The list** of entries **at the top**, **the preview** of the site **below**. The grip between them can be dragged (1.3).
- **The inspector on the right** with the settings of the selected entry.
- **Saving is immediate**: switches when flipped, text fields when you leave them. Every change takes a snapshot, at most one every 15 minutes. ⌘S does not apply here.
- **The preview** shows the site through the dev server; if it is not running, **Start preview** starts it. At the top you choose the page and the width (**Desktop**, **Tablet**,

Mobile). The selected entry is marked in the preview, and below it says whether it occurs on this page. **Zoom onto element** enlarges it. After a change the preview reloads as soon as Quartz has rebuilt.

- **Show on the stage** leads to **Layout ▸ Components**, where the place on the page is moved (5.3). **All options** leads to the entry under **Installed**, with the general form (4.5).
- If the editor does not know the installed version of the plugin, a note says that details may differ.

Creating a navigation

➕ **New navigation** in the toolbar asks for the variant. The navigation goes where that variant usually stands; it can be moved on the stage afterwards.

Variant	What it is	Placed first
Tree	Nested list, all open	Left sidebar
Accordion	Folders fold open, the path to the page is open	Left sidebar
Side panels	Folders open a panel beside them	Left sidebar
Select list	A jump menu, folders as groups	Left sidebar
Bar	First level in one line	Header
Dropdown	A line, folders open a panel below	Header
Mega menu	Panel with the second level as columns	Header
Tabs	First level as tabs, children in a second line	Header
Columns	Headings with children below, a sitemap	Footer
Previous / next	Previous and next page	After the content

The list

Each row shows name, variant and position, disabled ones with “off”. A row’s buttons **duplicate** the navigation or **remove** it; the last one cannot be removed. Removing deletes the entry with all its options from the configuration; the plugin stays installed.

Setting up

For the selected navigation the inspector shows:

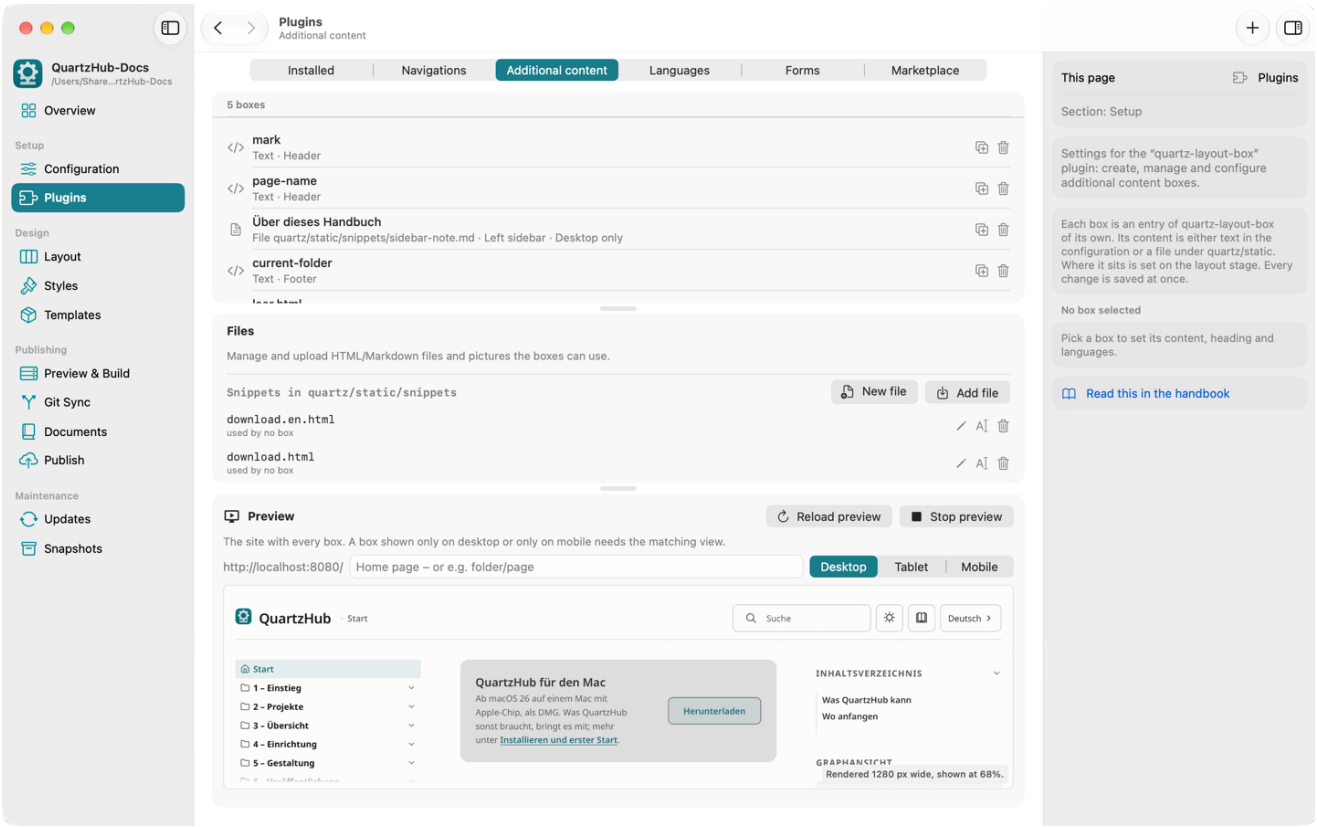
- **Navigation active**: switches it on or off without deleting it.
- **Variant**: the same ten tiles as when creating; a click changes the variant, the other settings stay.
- **Options**: the settings of the chosen variant first, then the general ones. Options that have no effect with this variant are grey and say why, such as “Only applies

with the variant …”. Some variants behave differently on a phone; what applies there is set by an option of its own.

What each option does, the plugin describes next to the field.

4.7 – Additional content

Plugins ▸ Additional content sets up the quartz-layout-box plugin. A box is a piece of content of your own that stands in the same place on every page: a note in the sidebar, an imprint link in the footer, a logo in the header. Each box is an entry of its own of the plugin. The tab works like the other editors (4.6).



The boxes at the top, the files in the middle, the preview below.

Creating and managing boxes

➕ **New box** creates a box with a sample text in the left sidebar at once. Each row of the list shows name, kind of content and position, plus "desktop only" or "mobile only" where set. A row's buttons duplicate or remove the box. Removing only deletes the entry; a file the box shows stays.

The content of a box

In the inspector, **Content** chooses where the box takes its content from:

- **Text in the configuration**: HTML, written right in the inspector. Good for short things.
- **File**: a Markdown or HTML file under `quartz/static/`. Good for longer things, and the file can be edited with any editor.

Switching from text to file asks whether to remove the text from the configuration; afterwards it is gone. A box can also have content of its own per language and page, which then takes precedence.

Below are the plugin's other options, such as heading, appearance and languages.

Files

The middle pane manages the files that boxes use:

- **Snippets** in `quartz/static/snippets/`: **New file** creates a Markdown or HTML file, **Add file** copies an existing one in. Each file says which boxes use it. Renaming a file updates the boxes that name it. Deleting moves it to the trash.
- **Pictures** in `quartz/static/images/`: **Add picture** takes PNG, JPEG, GIF, WebP, SVG and AVIF. If a picture a box uses is renamed, QuartzHub does not change the references in the text; that is up to you.
- **Project image**: `icon.png` and `icon-dark.png`, for looking only. In a snippet you include it with the path shown there (4.3).

Editing snippets

The pencil on a snippet opens it in the code editor, with line numbers and colours for HTML and Markdown. The snippet is a file of its own: **Save snippet** writes it, not the box's options and not ⌘S.

Available placeholders insert values at build time, in the text as in the box's title:

Placeholder	Becomes
`{{title}}`	title of the page, from its frontmatter
`{{slug}}`	path of the page, such as "folder/note"
`{{root}}`	way to the home page, for links and pictures: `{{root}}/static/images/logo.png`
`{{siteTitle}}`	title of the site
`{{baseUrl}}`	address of the site
`{{locale}}`, `{{lang}}`	language of the page, such as "en-US" or just "en"
`{{frontmatter.author}}`	any entry in the page's frontmatter

A click inserts the placeholder at the cursor. An unknown one stays as written.

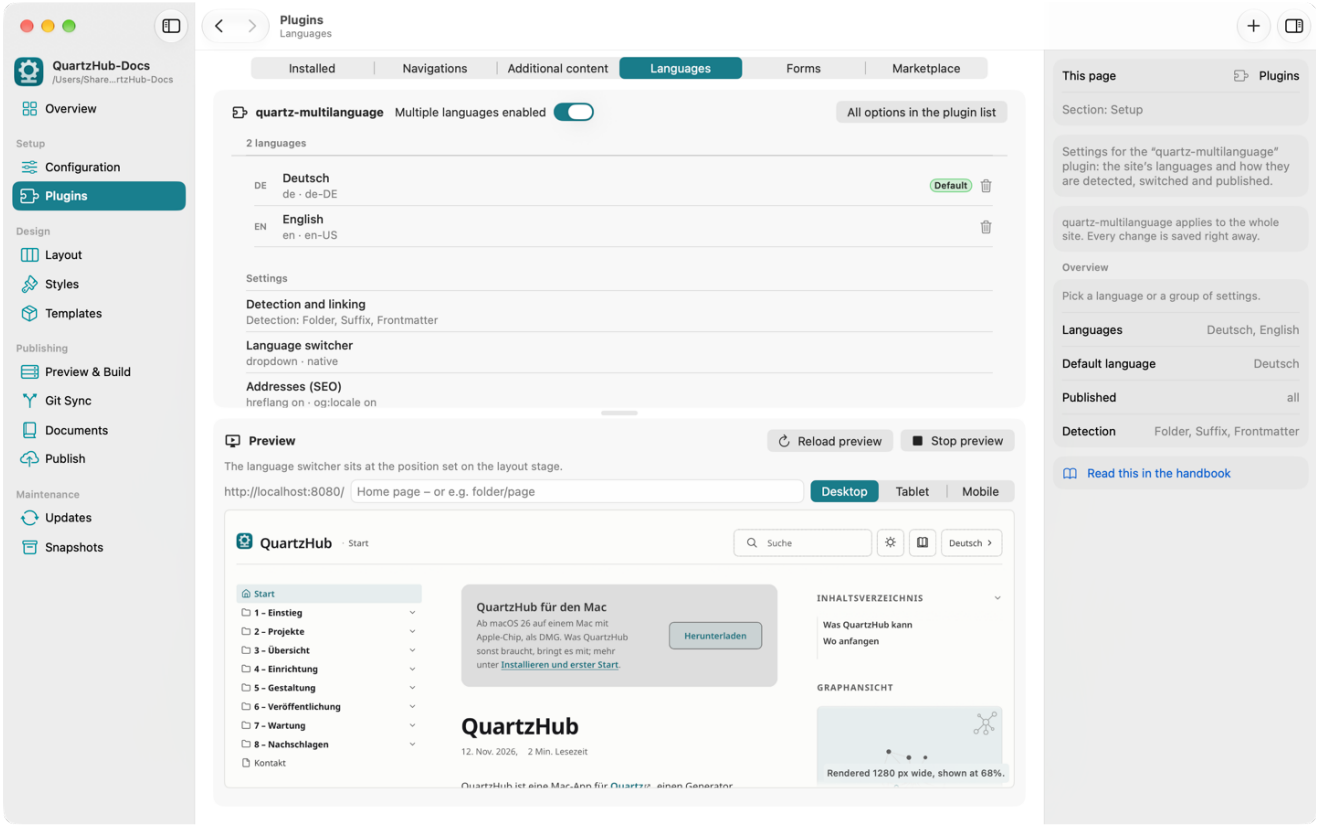
Insert image… copies a picture to `quartz/static/images/` and inserts the reference.

An SVG file can also be inserted as code; QuartzHub then removes whatever in it could run program code, and says what it removed.

4.8 – Languages

Plugins ▸ Languages sets up the quartz-multilanguage plugin. It turns one content folder into a site in several languages: it detects which language a note is written in, connects translations of the same page, puts a language switcher on the page and tells search engines which version is meant for which language. The plugin applies to the whole site, not per entry. The tab works like the other editors (4.6).

This handbook is built that way: the German pages at the top of the content folder, the English ones in the folder `en/`, connected through a shared key in the frontmatter.



Two languages, German as the default. The settings below them, the preview at the bottom.

Switching on and off

The **On** switch at the top builds the site with or without language detection, switcher and redirects. The settings are kept.

Languages

➕ **New language** asks for the code with which folders, file names and frontmatter name the language, such as `fr`. The list shows each language with code, name and locale; **Default** marks the default language, "not published" one whose pages do not reach

the site. The languages stand in the switcher in this order; reorder by dragging, with **Arrange ▸ Move Up/Move Down** (⌥⌘↑, ⌥⌘↓) or in the inspector with **Earlier** and **Later**.

For a selected language the inspector shows:

- **code**, **label**, **native** and **locale**. A new code does not change folder or file names in the content folder; default language and publishing follow along.
- **Default language**: applies to pages that detection assigns to no language. It can only be handed over by making another language the default.
- **Publish**: switched off, the site is built without this language's pages, for a translation that is not finished yet, say. The last published language stays on.

Removing a language takes it out of the configuration. Its pages stay and count as pages of the default language afterwards.

Settings

Below the languages are the groups of settings, each with a summary such as "Detection: folder, suffix, frontmatter". A click shows the group in the inspector. Which groups there are is up to the plugin; usually they are:

- **Detection**: how the plugin tells a note's language: by folder (`en/…`), by a suffix in the file name (`note.en.md`) or by the frontmatter.
- **Language switcher**: how it looks and what it shows. Where it stands is set on the stage (**Show on the stage**).
- **Addresses (SEO)**: what search engines are told about which version of a page is meant for which language.

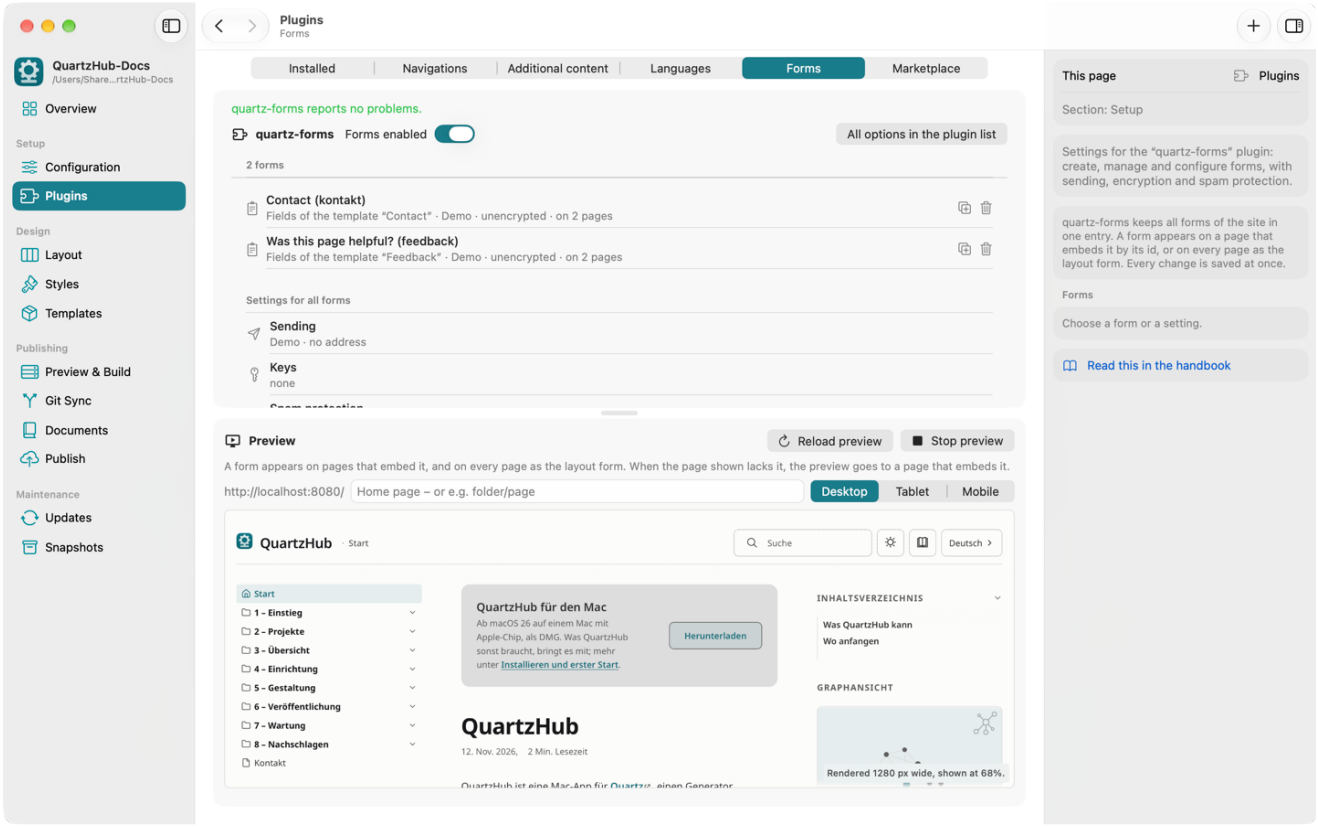
What each option does, the plugin describes next to the field.

> **Built-in texts per language**
>
> The site's built-in texts such as "Search" come from Quartz's language files for every language; they can be changed under **Translations** (4.4).

4.9 – Forms

A site made by Quartz is a collection of fixed files; it cannot take a message itself. The quartz-forms plugin therefore builds forms that send their message to a service which passes it on: an endpoint of your own, a provider such as Formspree or Web3Forms, or the **relay**, a small PHP script on your own web space. **Plugins ▸ Forms** sets all of this up.

If the plugin is not installed, the tab says so; it is installed in the marketplace via `boxi-os/quartz-forms` (4.10). The basic template brings it along, with a contact form that sends nothing yet. The tab works like the other editors (4.6).



At the top the forms and the settings for all of them, below the preview. "Demo" means the form sends nothing.

Creating a form

➕ **New form** asks what to **Start with**:

Template	Fields
Contact	name, e-mail, message, consent
Feedback	rating and comment on the page
Sign-up	name, e-mail, date, people

Survey	anonymous: choice, scale, comment
Empty	one message field, the rest is yours

Then the **Id**, such as `contact`: lower-case letters, digits and hyphens. A page embeds the form with it.

Embedding a form

A form appears in one of two ways:

- **On a page** that names it by its id, in one of three ways:
  - in the frontmatter: `form: contact`, for several `form: [contact, feedback]`;
  - as a placeholder in the text: `{{form:contact}}`;
  - as a code block of the language `quartz-form` with the id as its content.

  The frontmatter key and the code block's language can be changed under **Settings for all forms ▸ Technical**.
- **As the layout form** in the same place on every page, below the text for instance. A page hides it with `form: false` in its frontmatter.

The list says for each form on how many pages it stands. If a page names a form that does not exist, it says so above the list, because the build stops on it. The preview jumps to a page that embeds the selected form.

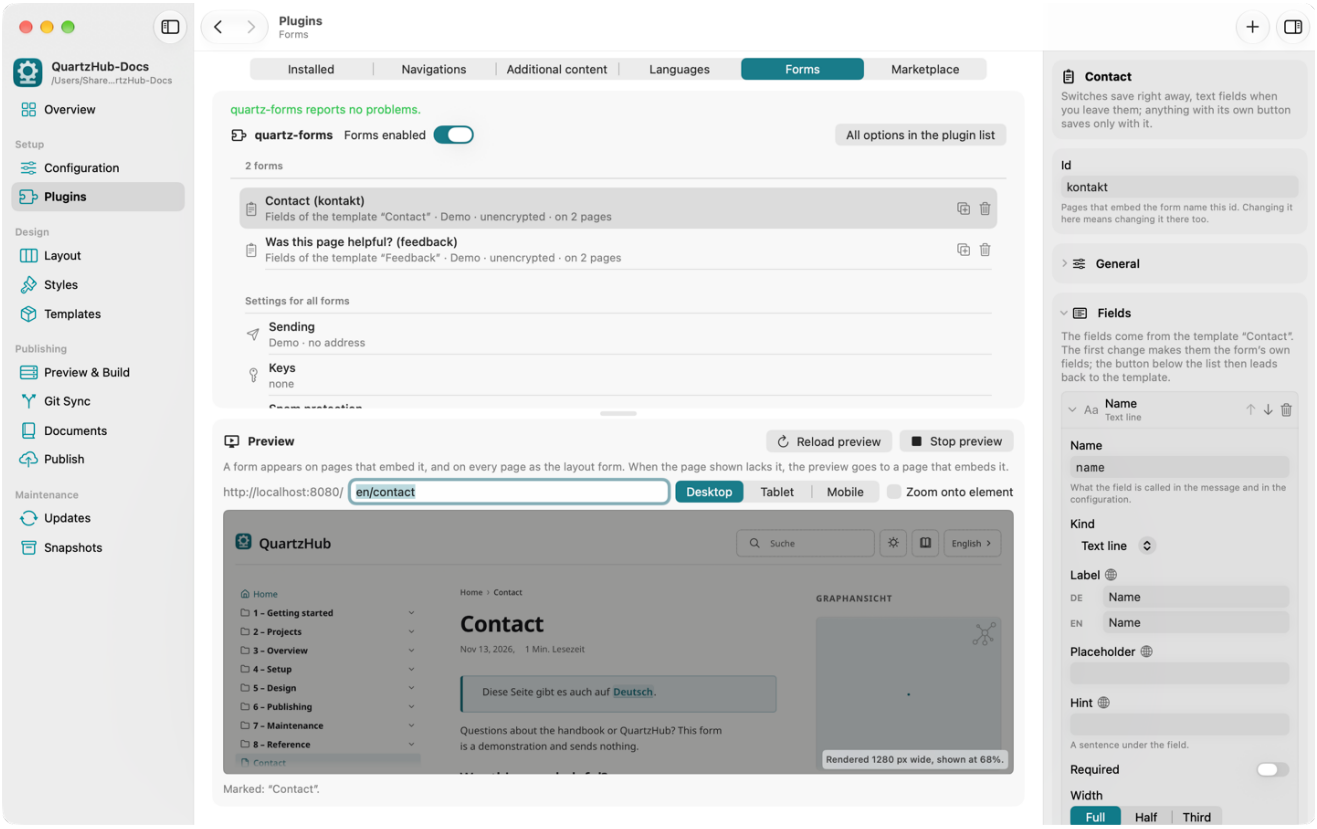
The list and the check

Each form shows title and id, fields, way of sending, encryption and the number of pages, plus marks such as **Layout** or **anonymous**. Copying creates a copy with the id `…-copy`; removing first says which pages embed the form, because they no longer build afterwards.

After every change QuartzHub has the plugin check its forms. Above the list it says that quartz-forms reports no problems, or the number of problems; a problem stops the build. The messages also appear with the form and the group they concern.

Setting up a form

In the inspector, the parts of a form fold open, **General** and **Fields** first. Changing the id asks first when pages use it; they then have to be changed too.



The "Contact" form chosen: its fields on the right, the first one open; the preview shows a page that embeds it.

Settings for all forms

Below the forms are the shared groups. Each form can change single values for itself.

- **Sending**: where the forms send to: the **Relay**, an **Endpoint** of your own, **Formspree**, **Web3Forms**, the visitor's **Mail program** or **Demo** (sends nothing).
- **Keys**: public PGP keys that encrypting forms send to. A key comes from a file, which is copied to `quartz-forms/keys/` in the project, or is pasted. QuartzHub does not accept private keys.
- **Spam protection**: traps for bots, a time trap, a puzzle, a captcha provider and a limit per hour. A captcha's secret key is entered with the relay.
- **Layout form**: which form stands on every page, and where.
- **Texts**: the plugin's wording, such as error messages, replaceable per language. The plugin's wording stands in grey; **Plugin wording** resets your own text.
- **Technical**: how the plugin recognises an embedding.
- **Relay**: only when a form sends through the relay; see below.

The relay

The relay is a PHP script (`send.php` with its configuration) that passes the forms'

messages on as mail. It lives on your own web space, so it needs a web space with PHP. QuartzHub has the plugin generate it and uploads it through one of the project's targets.

What the relay needs:

1. **Target**: a publishing target of the project of the type SFTP, FTP or folder (6.7). The relay goes into a **Folder** below its path, such as `forms`, and never deletes anything there. If the site target transfers with rsync and deletes what the build lacks, QuartzHub offers to exclude the folder there.
2. **Address of send.php**: where the forms send to. **Use the suggestion** builds it from base URL and folder.
3. **Sender**: an address of the web space's domain, or mail servers reject the messages.
4. **Recipients**: the forms name recipients by name, such as "contact"; here each name gets its address.
5. **Captcha secret**, if a form uses a captcha.

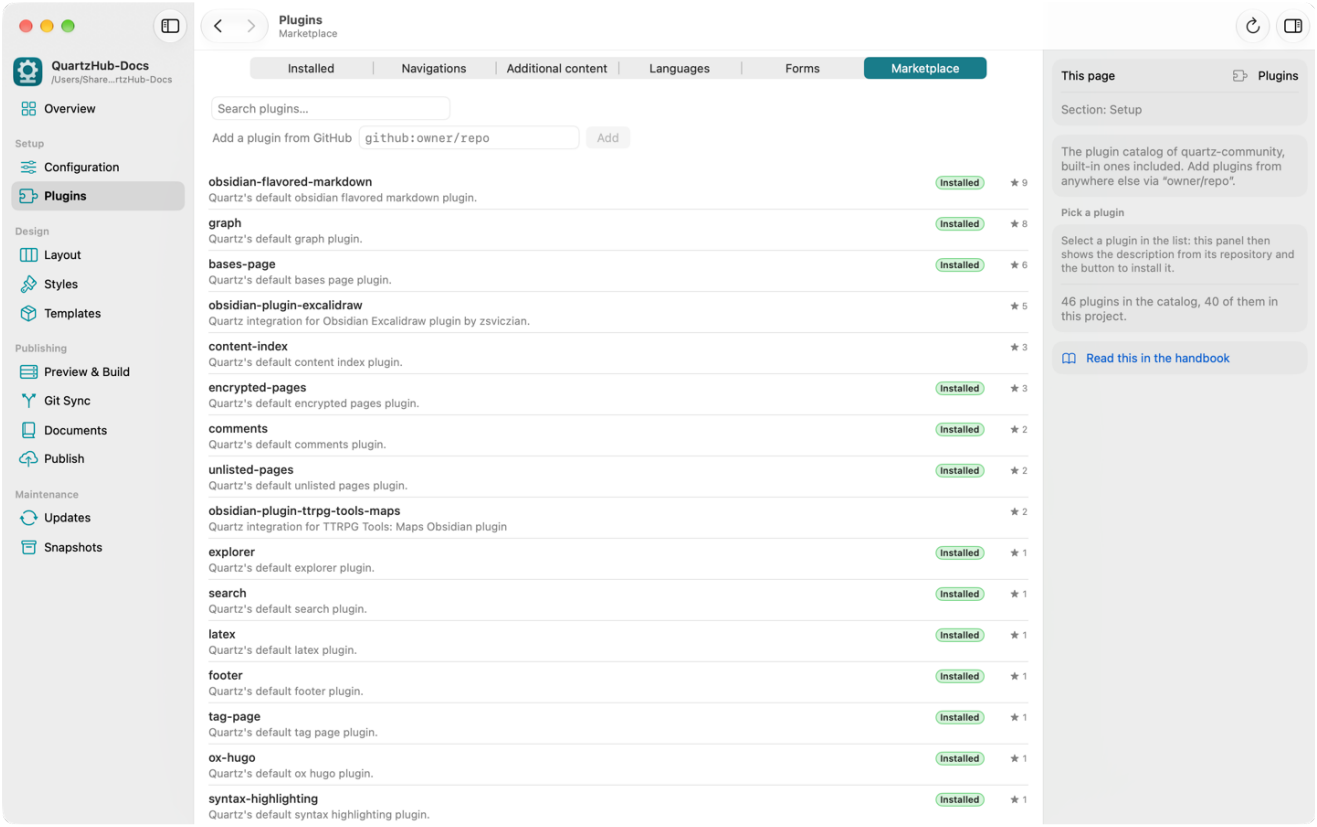
Optional: **Allowed origins** (the relay only accepts requests from these addresses), **Sending the mails** through PHP `mail()` or sendmail, and under **Rarely needed** a subject prefix, a folder for counters and a new secret.

Upload relay uploads it and then asks by itself whether it answers; this probe sends no mail. The answer appears below, such as "The relay answers at …", or what is wrong: no PHP, a redirect, a refused origin, an unreadable configuration. At the top of the group it says whether the relay was changed since the upload. **Save as folder …** writes it to a folder, for a web space you fill some other way; **Remove from server …** takes it down again.

> ⚠ **Where the addresses live**
>
> QuartzHub keeps the relay's recipient addresses and secrets in the keychain, never in the project folder and never on the site. On another Mac they are therefore missing; the group says so, and you enter them again. Removing the project from the list deletes them (2.1). In the uploaded `config.php` and in a saved folder they stand in plain text.

4.10 – Marketplace

Plugins ▸ **Marketplace** shows the catalogue of the **quartz-community** on GitHub: Quartz's own plugins and other people's.



The marketplace, sorted by stars. Installed plugins are marked.

The catalogue

QuartzHub loads the catalogue from GitHub the first time the tab is opened and keeps it for a quarter of an hour. **Reload catalog** in the toolbar fetches it again at once.

Each row shows name, description, stars and, where it applies, **Installed** or **Archived**. At the top are the repositories that declare themselves a Quartz plugin, sorted by stars. Everything else of the quartz-community, such as Quartz itself, libraries and templates, is folded below; most of it is not a plugin. The search field searches name and description.

Without signing in, GitHub only answers a limited number of requests per hour. If that is used up or GitHub does not answer, the tab says so, and below is only a placeholder. A personal access token for GitHub raises the limit; QuartzHub uses the one from the settings (2.5).

Looking at a plugin

With a plugin selected, the inspector shows its description from the repository (the README), a link to GitHub and **Install**. Pictures in the README only appear as text there; GitHub shows them in full.

Installing

Install takes a snapshot and then has Quartz add the plugin: Quartz downloads it, enters it in `quartz.config.yaml` and records the version in `quartz.lock.json`. Afterwards it is under **Installed**, with its defaults. This needs a network connection.

Add a plugin from GitHub installs a plugin that is not in the catalogue. Enter `github:owner/repository` and press **Add**. QuartzHub passes the text to Quartz unchanged; what else Quartz accepts is described in the **Quartz documentation**.

> ⚠ **Plugins are programs**
>
> A plugin runs at every build with the rights of your account. Only install what comes from a source you trust.

5 – Design

The **Design** group has three pages. **Layout** sets what stands where on the page. **Styles** sets how it looks: colours, fonts, CSS. **Templates** packs a finished design into a file you take over into another project.

1. **The layers**: in which order basics, theme, variables and custom CSS override each other. Once you have read it, you stop searching why a colour has no effect.

Layout

2. **Layout overview**: which component appears on which page type.
3. **Components**: the stage on which components are moved.
4. **Frames**: building page grids of your own.

Styles

5. **Basics**: base colours, fonts and how they are delivered.
6. **Community themes**: ready-made colour schemes.
7. **Variables**: changing single CSS variables precisely.
8. **Custom CSS**: writing stylesheets yourself.

Templates

9. **Templates**: saving a design as a file and applying it.

5.1 – The layers

The look of a Quartz site comes from several layers lying on top of each other. What lies higher wins. The four tabs under **Styles** are these layers, in this order:

Layer	Where	What it sets
1. Basics	**Styles ▸ Basics**	the base colours for light and dark and the three fonts, from the configuration
2. Community theme	**Styles ▸ Community themes**	a ready-made colour scheme; overrides the basics
3. Variables	**Styles ▸ Variables**	single CSS variables; beat the theme
4. Custom CSS	**Styles ▸ Custom CSS**	rules of your own, for everything the other layers cannot do

The inspector on the right shows the same order, plus whether a theme is active and how many variables are overridden.

What follows from it

With a community theme active, many basic colours have no effect. Styles ▸ Basics shows this: at the top it says how many colours and fonts the theme overrides, and those fields are paler. They apply again once the theme is off.

Against a theme, a colour is changed under Variables. The values there win, even where the theme sets the same variable. The exception are colours a theme sets for one kind of element only, such as callouts; those are changed in custom CSS.

Custom CSS is the last resort, not the first. A colour that exists as a variable is better changed under Variables: it then applies everywhere the variable is used, separately for light and dark.

Within custom CSS

Quartz only includes one file, `quartz/styles/custom.scss`. That file loads everything else, in the order listed under **Custom CSS** on the right under **Load order**:

1. your own stylesheets in `quartz/styles/custom/` and `imported/`, in their order;
2. `_quartzcontrol.scss`: the fonts and variables QuartzHub writes;
3. `custom.scss` itself, always last.

What loads later wins among equally specific rules. A rule in `custom.scss` therefore beats a value under **Variables**; the Variables tab warns when `custom.scss` sets the same

variable. A file under `custom/`, however, loads before the variables and loses to them.

> **Why the variables beat the theme**
>
> Quartz and the themes put their rules into CSS layers (`@layer`). What a project writes itself is in no layer, and a rule outside all layers beats every rule inside one, however specific it is.

And the layout?

The layout is none of these layers. It sets which components stand where and what the page's grid looks like (5.2); the styles set how it looks.

5.2 – Layout overview

Three terms

Components are the visible parts of a page: search, table of contents, graph, a navigation, a box. Each is a plugin entry (4.5) with a **position**: header, left or right sidebar, before or after the content, footer. Components are placed once for all pages.

Page types are the kinds of page Quartz builds: content pages (the notes), folder pages, tag pages, the 404 page and, depending on the plugins, more such as Bases or Canvas pages. Each page type can differ from the rest: hide components, clear areas, use a frame of its own.

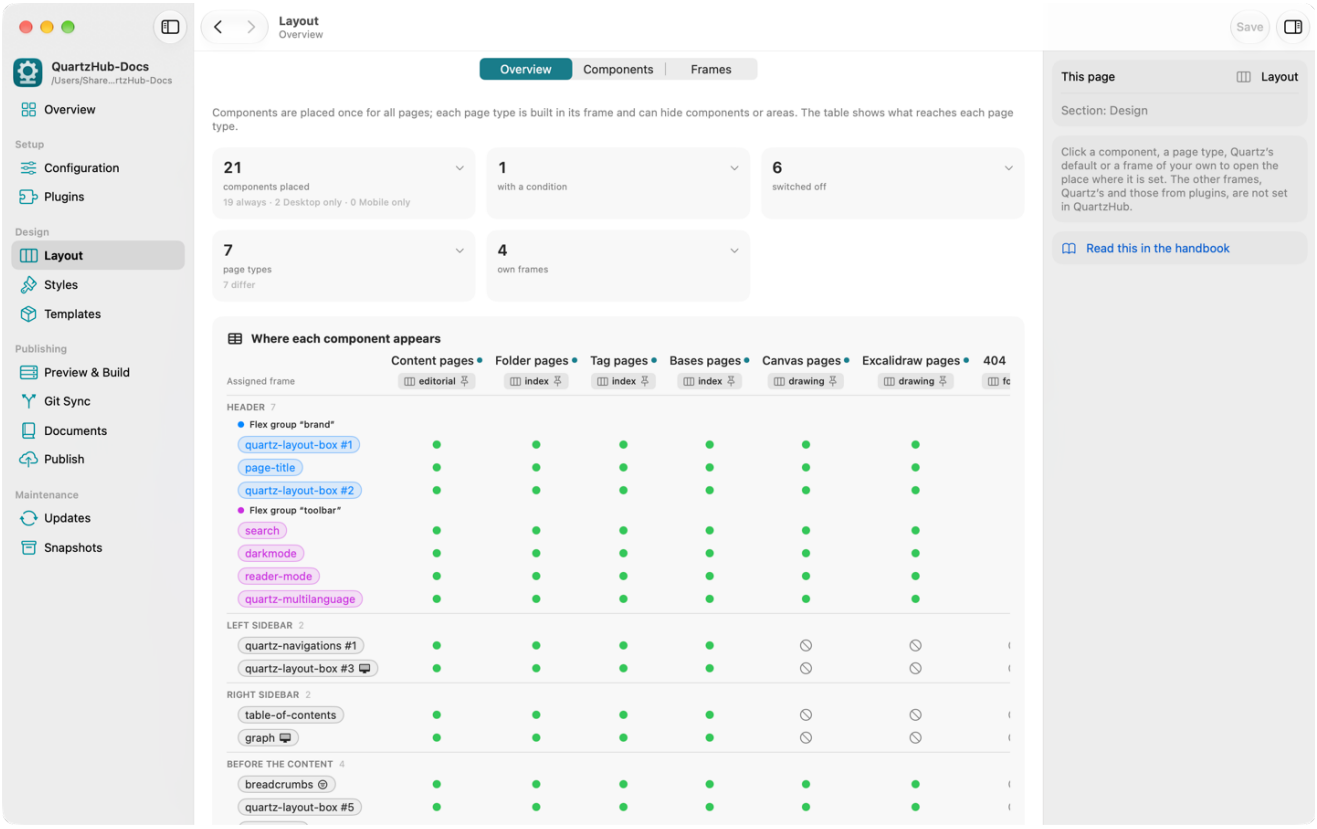
A **frame** is the grid of a page: which areas there are, where they lie and how wide they are. Quartz brings Standard, Full width and Minimal; your own are built under **Frames** (5.4).

Layout has three tabs for this: **Overview** shows everything, **Components** is the stage for moving things, **Frames** the editor for grids of your own.

Saving

Overview and Components change `quartz.config.yaml`, Frames changes a frame. Both are written only by **Save** (⌘S), each for the tab in front. Before the configuration is saved, QuartzHub takes a snapshot, at most one every 15 minutes. Leaving the page with unsaved changes asks first.

The overview



The overview: the figures at the top, the table below.

The overview changes nothing; it shows and leads on. A click on a component, a page type or one of your own frames opens the place where it is set.

The figures at the top each open a list below:

- **components placed**, split into always, desktop only, mobile only;
- **with a condition**: components that only appear on some pages, say only with a table of contents;
- **switched off**: components in the tray that are not built;
- **page types**, and how many of them differ, with what is hidden or cleared in each;
- **own frames**, and which are assigned to no page type.

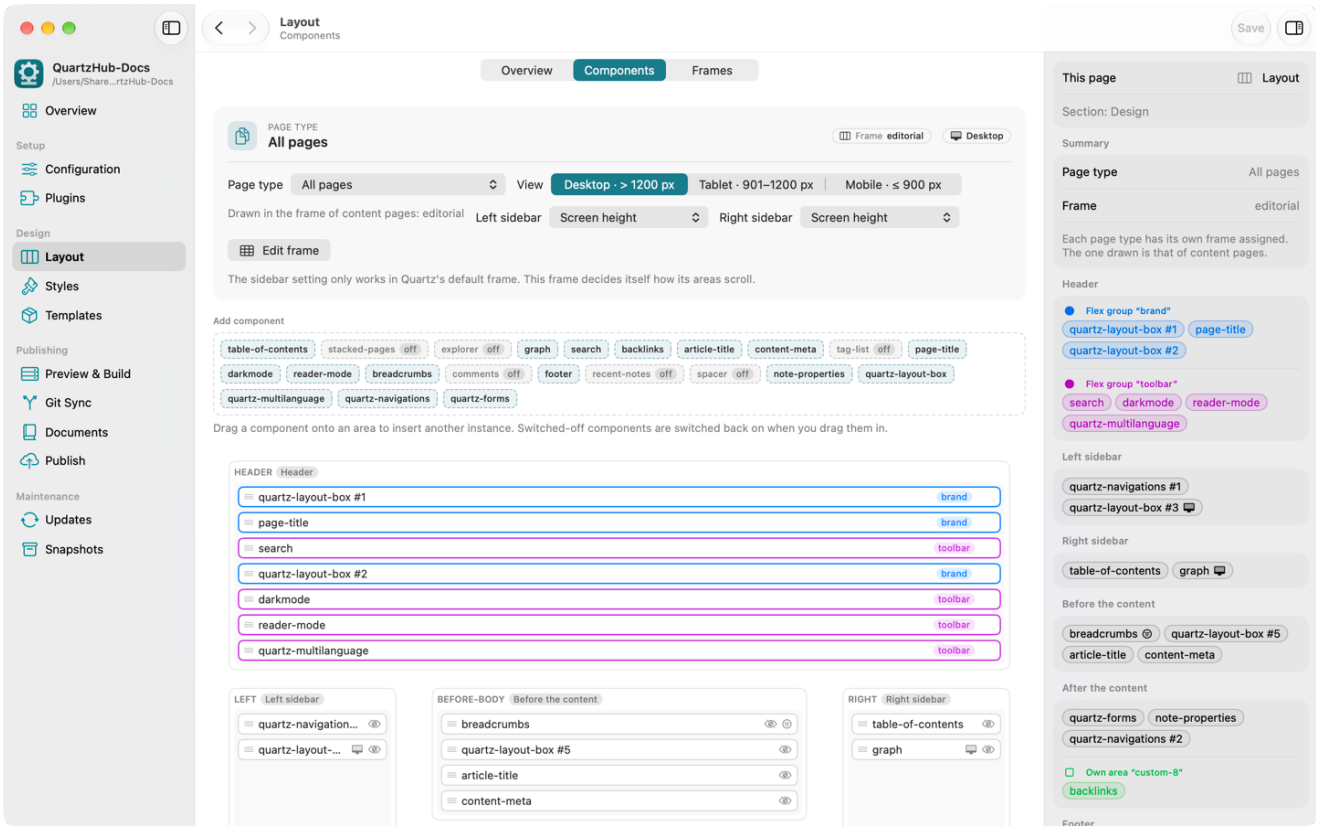
The table "Where each component appears" has a column per page type and a row per component, grouped by position. The first row shows each page type's frame; a dot at a page type means it differs from "All pages". Each cell says whether the component gets there:

- **appears**, or **only on** certain widths;
- **hidden** on this page type, or its **area cleared**;
- its area is **not assigned in this frame** or shown at no width;
- **unknown**, when the frame comes from a plugin and QuartzHub does not know its structure.

A cell’s tooltip gives the exact reason. The legend is below the table.

5.3 – Components

Layout ▸ **Components** shows the page in miniature, drawn in the frame of the chosen page type: the **tray** of components at the top, the **stage** with its areas below, the **flex groups** at the bottom. Everything here goes into a draft; only **Save** (⌘S) writes the configuration (5.2).



The stage for all pages in the desktop view. The summary per position on the right.

The head

- **Page type**: **All pages** or a single page type. A customized one reads “(customized)”.
- **View**: **Desktop**, **Tablet** or **Mobile**. With a frame of your own, the widths are shown.
- **Frame** (only for a page type): Standard, Full width, Minimal, one of your own frames, or the one the page type’s plugin brings.
- **Left/Right sidebar** (Quartz 5.1 and later): what the sidebar does when its content is taller than the screen: **Screen height** (the single components scroll), **Scrolls itself** or **Scrolls with the page**. This only works in Quartz’s standard frame.
- **Edit frame** opens one of your own frames in the editor. **Remove customization** resets a page type to “All pages”.

> **What applies to all and what to one page type**
>
> With a page type chosen, only **hiding** and **clearing** apply to it. What you move or set in the inspector applies to all pages.

Moving

- **Dragging** onto another component puts the dragged one before it; onto the free rest of an area, at the end.
- **Dragging from the tray** inserts another instance, a second navigation for instance. A switched-off component is switched back on on the way.
- **Dragging into the tray** switches a component off; a duplicate is removed.
- With the **keyboard**, the inspector does the same: **Position**, **Earlier** and **Later**.

QuartzHub renumbers the components of a position as it goes (10, 20, 30 …), so their order in the configuration matches the stage. A click into empty space or Esc clears the selection.

Clearing (the eraser on an area, only for a page type) leaves an area empty on that page type; a second click fills it again.

Dashed components are not on the page shown; the tooltip says why.

The inspector

For a selected component:

- **Hide on {page type}** (only for a page type). If there are several instances of the same component, this hides all of them; Quartz cannot exclude single duplicates separately.
- **Position**, **Order** with **Earlier** and **Later**.
- **Visibility**: **Always**, **Desktop only** or **Mobile only**. “Desktop only” applies on the tablet too; Quartz only switches at the mobile boundary.
- **Condition**: **Always**, **Not on the home page**, **Only on pages with tags**, **… with backlinks**, **… with a table of contents**, or a **custom** one a plugin brings.
- **Group** and, inside a group, how the component takes up room in it.
- **Duplicate**, **Put in the tray** or **Remove duplicate**.
- **Options**: the same form as under **Plugins ▸ Installed**.

Without a selection, the inspector shows a **Summary**: the frame and, per position, its components.

Flex groups

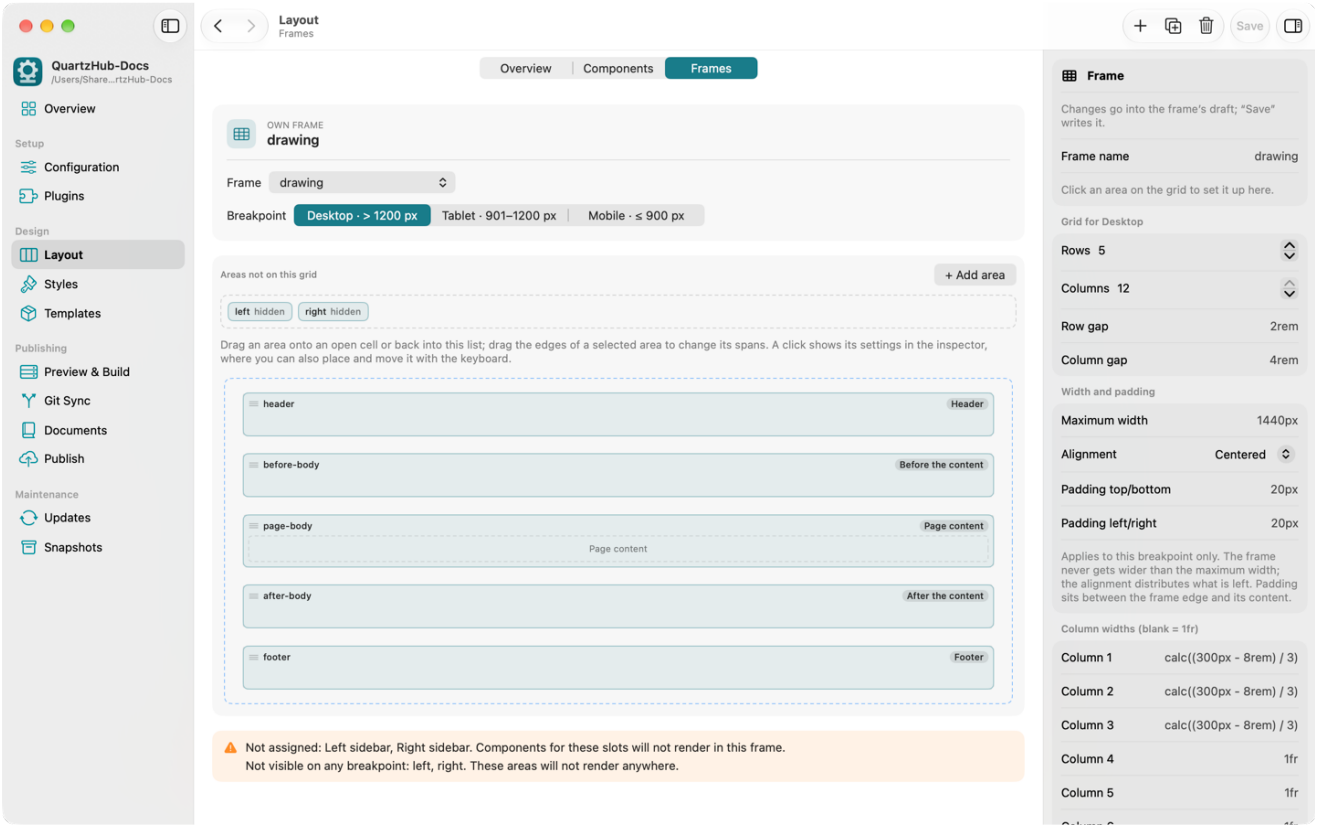
Components of the same group stand side by side or one below the other instead of each on its own line, such as search, the light/dark switch and the language switcher in one line in the header. Per group you set **Arrangement** (side by side, stacked, each also

reversed), **Wrapping**, **Gap** and **Priority**. A new group is created at the bottom with a name and **Add**; a component joins it via **Group** in the inspector or by dragging it into the group.

5.4 – Frames

A frame is the grid of a page: which areas there are (header, sidebars, content, footer), where they lie and how wide they are. What appears in the areas is set under **Components**; that is also where the frame per page type is chosen (5.3).

Layout ▸ Frames builds frames of your own without writing CSS. Each frame is set up separately for desktop, tablet and mobile.



One of your own frames in the desktop view. The grid's settings on the right.

Choosing and creating frames

At the top, **Frame** chooses between **Quartz standard** (view only) and your own frames, **Breakpoint** between desktop, tablet and mobile. The toolbar has:

- ➕ **New frame**: an empty frame with all areas, none placed yet;
- **Duplicate**: a copy of the chosen frame, "…-kopie";
- **Delete**: page types that use it fall back to the standard frame;
- **Save** (⌘S): writes the frame.

On **Quartz standard** the tab shows how Quartz divides a page without a frame of your own. **Customize as your own frame** creates a copy of it, **Create an empty frame** an empty one.

The grid

Above the grid are the **areas not on this grid**. **+ Add area** creates a new one.

- **Dragging** an area onto a free cell places it; back into the list takes it off the grid, on this breakpoint only. A shadow shows where it would land; red means it would cover another area.
- At the **edges** of a selected area you drag its spans across several rows or columns.
- A **click** shows its settings in the inspector. A click into empty space or Esc clears the selection.

Below the grid are warnings, for instance when areas overlap, reach past the grid, a position is assigned to no area (its components then do not appear in this frame), or an area is visible at no breakpoint.

The inspector

Without a selection it sets up the frame:

- **Frame name**.
- **Grid for {breakpoint}**: **Rows** and **Columns** (1 to 12), **Row gap** and **Column gap**.
- **Width and padding**: **Maximum width**, **Alignment** and the paddings. Without a maximum width the frame uses the whole window width.
- **Column widths** (empty = equal shares) and **Row heights** (empty = as tall as the content), as CSS values such as `16rem` or `1fr`.
- **Named grid lines**, if you want to address the lines in custom CSS later.
- **Copy to {breakpoint}**: takes over this breakpoint's whole grid onto another, without asking.

With an area it sets up that area:

- **Area name**: becomes a CSS-safe name of lower-case letters, digits and hyphens; when that differs from what was typed, the name it is saved as is shown below.
- **Slot**: which position the area shows, or none.
- **Area of its own**: the area then only shows the components dragged here under **Components**, not everything belonging to its position. That way a position gets two areas, a sidebar at the top and at the bottom for instance.
- **Row span**, **Column span** and **Visible on {breakpoint}**.
- **Move to** row and column: the same as dragging, with the keyboard.
- **Remove from grid** (this breakpoint only) and **Remove area**.

Breakpoints

Breakpoints sets from which window width your own frames switch to tablet and mobile. The default is 1200 and 800 pixels, as in Quartz. This section's own **Save** writes

at once and applies to all your own frames. Quartz's built-in frames and community themes bring their own values.

What happens on saving

To Quartz, a frame of your own is a plugin: a folder under `.quartz-gui/authored-frames/` with the description of the grid and the generated CSS. A new frame is registered with Quartz at its first save, after QuartzHub takes a snapshot. Before every build and every start of the preview, QuartzHub rewrites all frames.

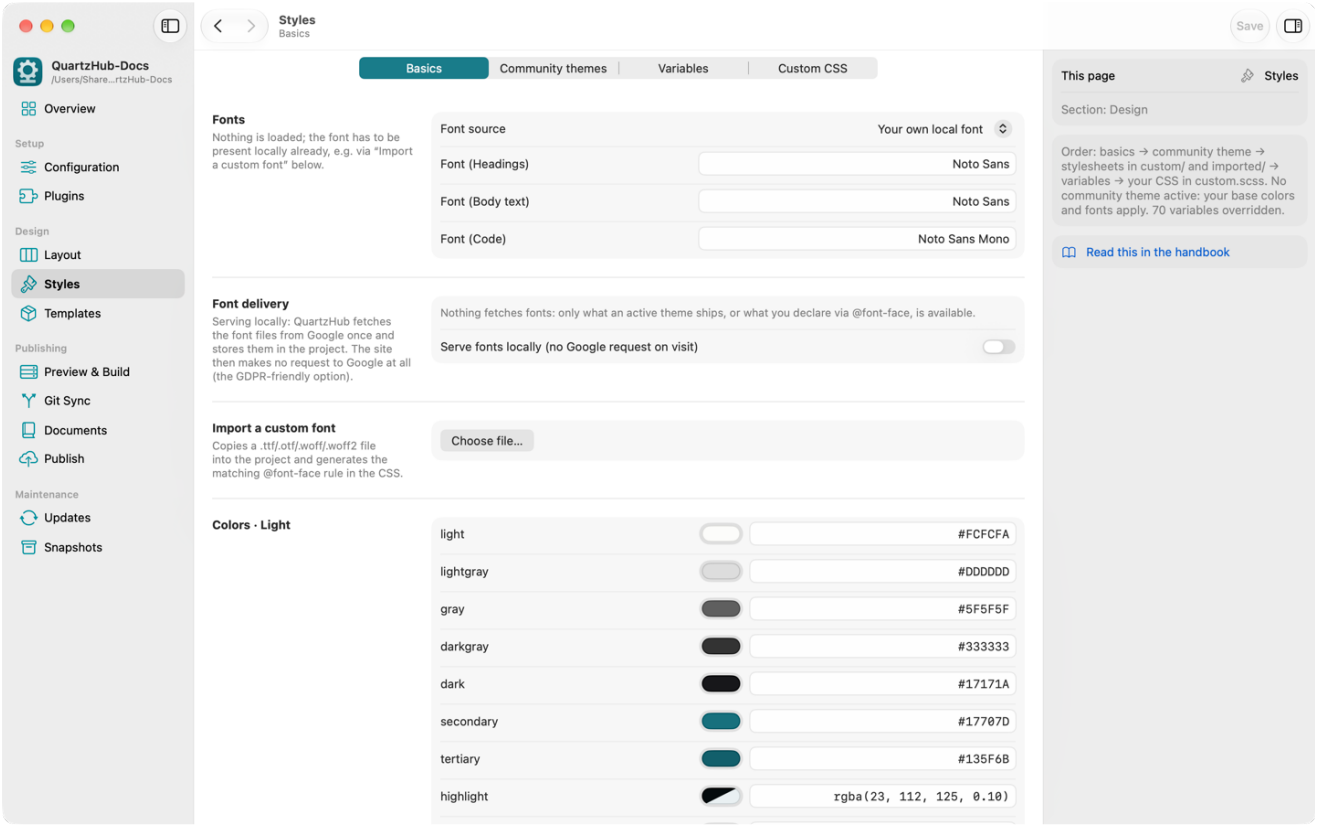
With the preview running, a change to an existing frame only shows after a restart; the tab offers **Restart dev server**.

Page types name their frame by its name. When a frame is renamed, saving it updates the page types that use it.

Your own frames do not travel with git (3.1).

5.5 – Styles ▸ Basics

Styles ▸ Basics is the bottom layer (5.1): the colours and fonts written in `quartz.config.yaml`.



Fonts, font delivery and the colours for light mode.

Saving

The four tabs under **Styles** share one **Save** (⌘S): what you change on one tab survives switching to another and is saved together with it. Before the configuration is written, QuartzHub takes a snapshot, at most one every 15 minutes. Some things apply at once, without saving; it says so where they are.

With a theme active

If a community theme is active, a note at the top says how many colours and fonts it overrides. Those fields are paler; they can still be changed and apply once the theme is off. Against the theme, values are changed under **Variables** (5.7).

Fonts

- **Font source**: **Google Fonts** or **Your own local font**. With Google the fields offer

every Google font; typing filters. Locally nothing is loaded; the font has to be in the project, via **Import a custom font** for instance.
- **Font (Headings)**, **(Body text)** and **(Code)**: the font for headings, text and code.

Below a field it says when something is wrong: Google does not know the name or spells it differently, the theme overrides the font, or a variable in the project does; then the field changes nothing on the site.

Font delivery

Where the fonts come from for visitors is a question of privacy: if the site loads them from Google, Google learns about every visit.

- **Serve fonts locally (no Google request on visit)**: on saving, QuartzHub fetches the font files from Google once and stores them in `quartz/static/fonts/`. The site then makes no request to Google. Switched off, saving removes the local copies again.
- Above, it says who loads fonts in the project right now: Quartz itself, QuartzHub, the “Fonts” plugin or the theme. What links Google directly is shown in orange.
- **Fetch into the project now** saves the page at once and fetches the fonts.
- **Load the theme’s own fonts** (only with a theme): a theme loads its fonts from unpkg.com; QuartzHub cannot serve them locally. Switched off, your own fonts apply.

If the “Fonts” plugin serves the fonts itself, it needs a base URL (**4.1**).

Known conflicts

With the “Fonts” plugin and a theme together, the headings do not follow the theme. **Add fix** at once creates a small file `custom/fix-heading-fonts.scss` that fixes it; afterwards it is listed under **Custom CSS** like any other.

Import a custom font

Choose file… takes a `.ttf`, `.otf`, `.woff` or `.woff2` file, copies it to `quartz/static/fonts/` at once and writes the matching `@font-face` rule. **Use directly for** sets it as the font for headings, text or code right away; that applies after saving.

Unused fonts lists imported fonts nothing uses. Removing deletes rules and files at once, without a snapshot.

Colours

Per mode, light and dark, the base colours with their names from Quartz:

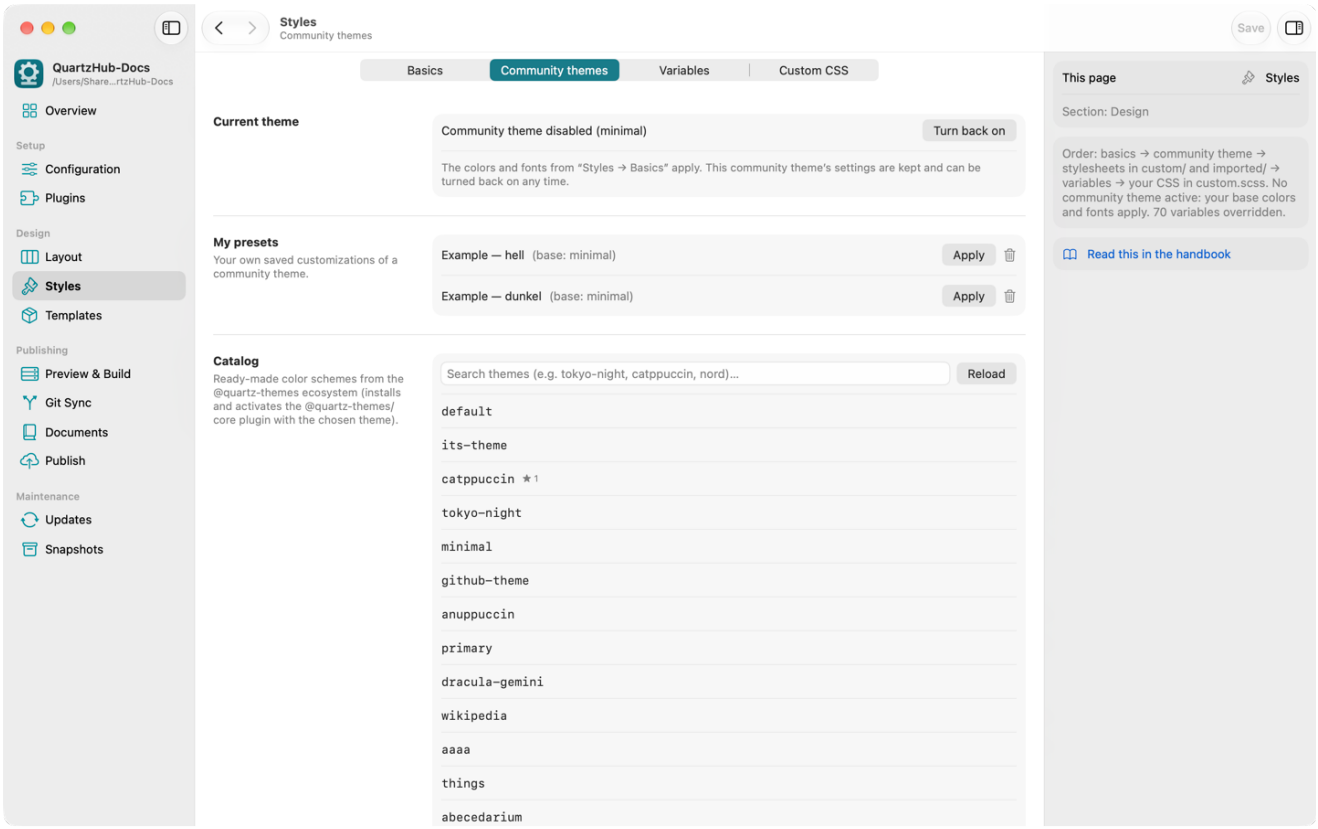
Name	What Quartz uses it for

`light`	background
`lightgray`	lines and borders
`gray`	heavier borders, links in the graph
`darkgray`	body text
`dark`	headings and icons
`secondary`	links, the current page in the graph
`tertiary`	hover, visited pages in the graph
`highlight`	background of internal links
`textHighlight`	highlighted text (`==like this==`)

Each has a colour well and a text field for any CSS colour value. What a theme overrides carries the mark **THEME**.

5.6 – Community themes

Community themes are ready-made colour schemes, many of them after themes for Obsidian. They come through the `@quartz-themes/core` plugin and lie on top of the basics (5.1).



A switched-off theme, two presets and the catalogue.

Choosing a theme

Catalog lists the themes by their stars on GitHub; the search field filters, for "tokyo-night", "catppuccin" or "nord" for instance. A click shows a theme in the inspector: description, modes, variants, whether it lets you adjust its colours. **Install & activate** installs it at once and sets it in the draft; only **Save** writes it to the configuration.

The list comes from npm and is kept for a quarter of an hour; without a network only a few known themes stand there as placeholders.

The current theme

At the top it says which theme is active. **Disable community themes** switches it off and keeps its settings; **Turn back on** switches it on. Without a theme, the colours and fonts from **Basics** apply.

The theme’s settings

Many themes bring settings of their own, such as an accent colour or the style of headings. QuartzHub shows them with labels and descriptions from the original theme, in groups and with a search field; **Reset** on an option takes it back. QuartzHub fetches the descriptions from GitHub once and keeps them; **Reload docs** fetches them again, **Settings ▸ Data & maintenance** clears all of them (2.5). Without descriptions, only the raw switches are shown.

What a theme does not offer as a setting is changed under **Variables**.

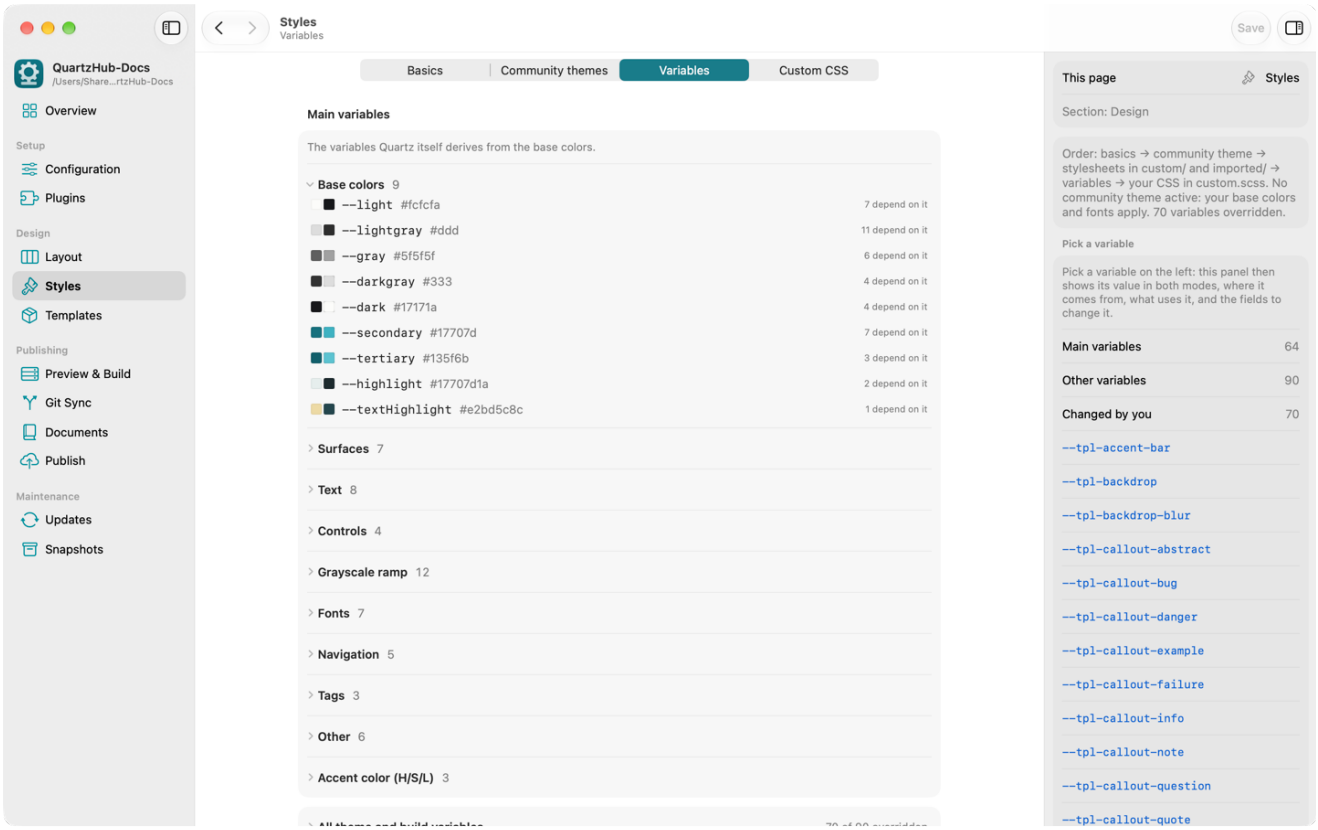
Presets

Save as preset saves the current theme with its settings under a name, at once, in `.quartz-gui/theme-presets.json`. **My presets** lists them; **Apply** installs the preset’s theme and takes its settings into the draft. A deleted preset is gone.

> **Themes load fonts from outside**
>
> A theme loads its fonts from unpkg.com. If you do not want that, switch off the theme’s own fonts under **Basics ▸ Font delivery** (5.5).

5.7 – Variables

Almost every colour and font and many spacings of a Quartz site come from CSS variables such as `--secondary` or `--bodyFont`. **Styles ▸ Variables** shows them all, says where each gets its value and who uses it, and changes them precisely. Values here beat the theme (5.1).



The main variables, the base colours unfolded. On the right, which ones are changed.

The list

- **Main variables**: the variables Quartz derives from the base colours, in groups such as base colours, surfaces, text, navigation, fonts. A group shows how many in it are changed.
- **All theme and build variables**: everything the theme, plugins or the last build bring. QuartzHub only knows the total after a build. A search field (such as “callout”, “h1”, “background”), **Changed only**, a filter for colours and other values and **Reload** help to find them.

Each row shows the colour for light and dark, the name, the value, how many other variables depend on it, and where it comes from: **changed**, **own variable** or **community theme**.

Looking at a variable

A click shows the variable in the inspector:

- **Current value**, for light and dark, with the colour value.
- **Source**: Quartz, the theme, a stylesheet or the last build, with the chain the value comes through, in the form `--a → --b → value`.
- **Uses these variables** and **Used by these variables**: a click jumps there. Below, it says how many further variables a change affects.

Changing a variable

Under **Your own value** you enter the new value, for colours also through a colour well. **Different in dark mode** separates light and dark. **Reset to original value** takes the change back. A variable that only exists in your own project can be removed; every place using it then has no value.

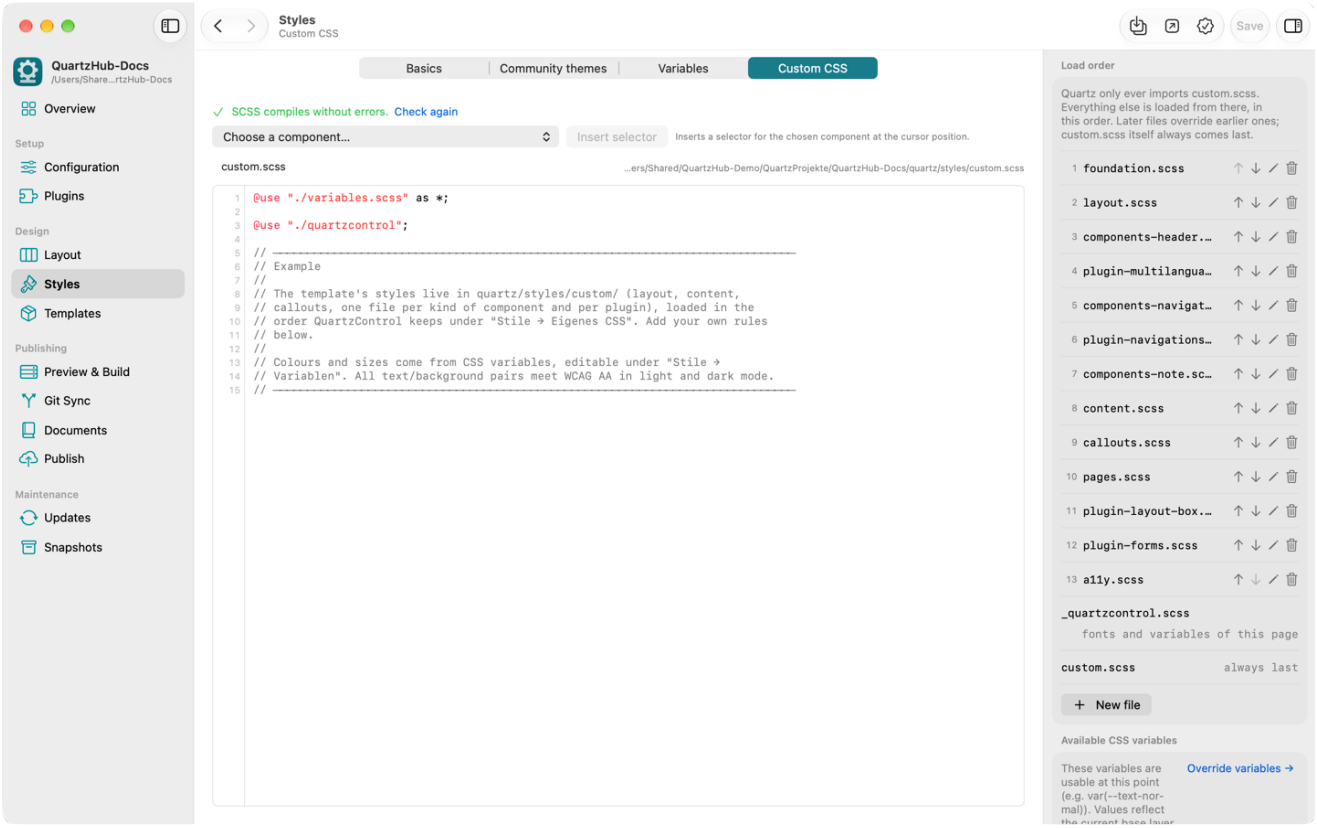
Saving is **Save** (⌘S). QuartzHub writes the values into a section of its own in `quartz/styles/_quartzcontrol.scss`. A value must not contain a semicolon, a curly brace, a backslash or a comment; otherwise QuartzHub saves nothing at all and says which value it is.

> **Better the source than what derives from it**
>
> Many variables derive from a few. Changing `--secondary` changes all links, highlights and whatever depends on them. The line “Your own value here affects … other variables” shows how far a change reaches.

If `custom.scss` sets the same variable itself, it wins there; a note at the top says so.

5.8 – Custom CSS

Styles ▸ Custom CSS is the top layer (5.1): rules of your own in SCSS or CSS, for everything basics, theme and variables cannot do.



custom.scss in the editor, the load order of the stylesheets on the right.

The files

Quartz only includes `quartz/styles/custom.scss`; that file loads all the others. The inspector on the right shows the order under **Load order**: first your own stylesheets in `quartz/styles/custom/` and `imported/`, then `_quartzcontrol.scss`, last `custom.scss` itself. Files loaded later override earlier ones.

- **New file** creates a stylesheet under `custom/` and appends it to the order.
- **Import file…** (toolbar) copies a `.scss` or `.css` file into `imported/`. If a file of the same name is there, QuartzHub asks whether to replace it.
- Per file: **Move Up**, **Move Down**, **Rename**, **Delete**. These file actions apply at once, without a snapshot.
- **Present, but not loaded** lists files that lie in the folders but are not loaded; **Load it** takes them in.

A click on a name opens the file in the editor, in a tab. Changes to the text are written

by **Save** (⌘S).

> ⚠ **_quartzcontrol.scss belongs to QuartzHub**
>
> What stands between the markers in `_quartzcontrol.scss`, QuartzHub rewrites at every change to the order, fonts or variables. Text outside the markers stays, but rules of your own belong in a file of their own.

The editor

A code editor with line numbers, colours for SCSS and CSS and the macOS find bar. Around it:

- **Choose a component…** and **Insert selector**: inserts a component's CSS selector at the cursor, for search or the table of contents for instance.
- In the inspector under **Available CSS variables**: a click inserts `var(--name)`, a click on the colour its value; with ⌥ it is copied instead of inserted.
- **Callout colors**: a click on a callout type inserts a scaffold for its colours.
- **Colors and fonts in effect**: what applies right now, light and dark, with a font sample.

Checking before the build

An error in the SCSS makes the whole build fail. QuartzHub therefore checks with the project's Sass:

- On opening the tab and after saving, it checks all stylesheets together. At the top it then says "SCSS compiles without errors." or the error with a link to file and line. **Check again** checks once more.
- **Check code** in the toolbar checks only the open file, unsaved too.

If no Sass is installed in the project, QuartzHub says it cannot check instead of giving the all-clear. A CSS error also appears on the overview (3.1).

Projects from older versions

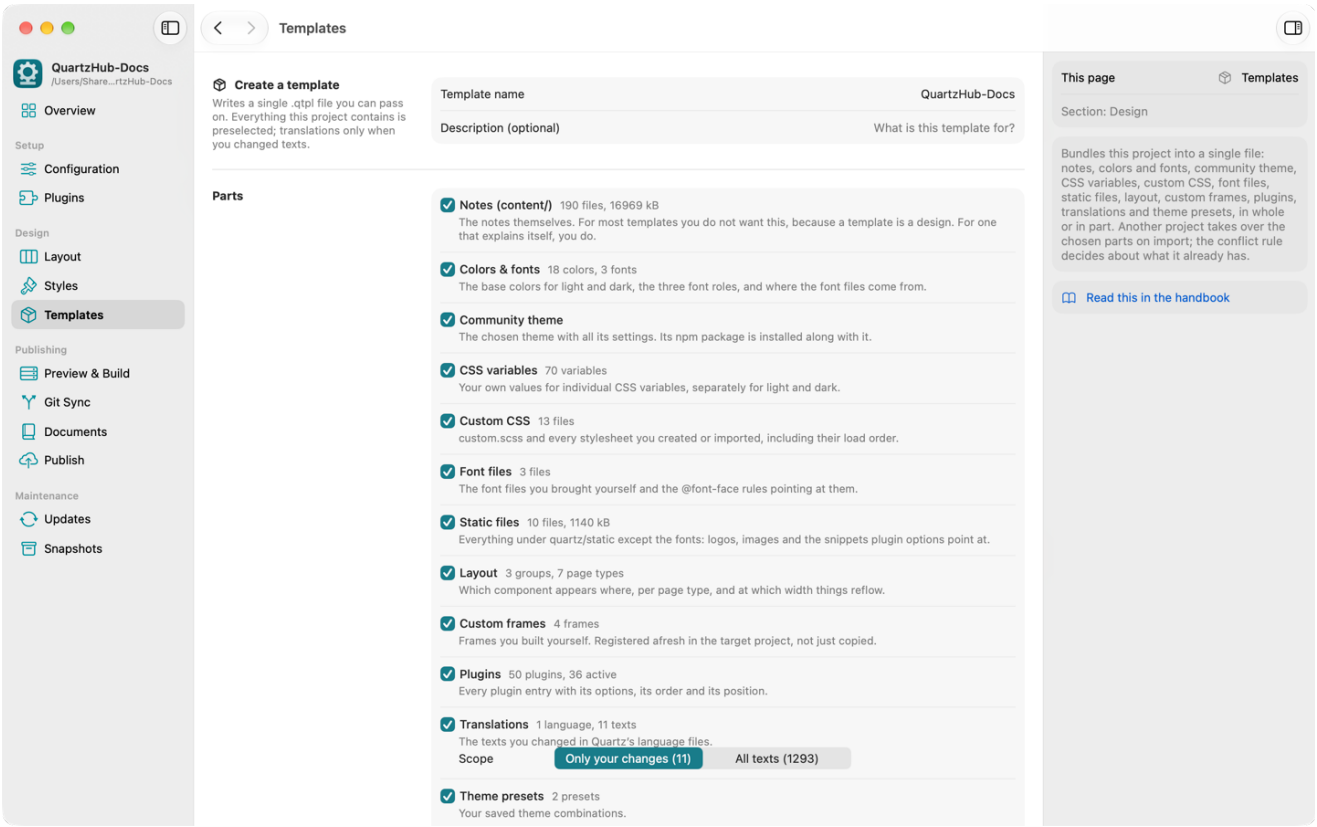
Two notes appear only in projects set up with an early version:

- **Move now** moves load order, fonts and variables out of `custom.scss` into the file `_quartzcontrol.scss` of their own.
- **Merge** combines the old templates' stylesheets into the twelve files of the new split and keeps every rule.

Both take a snapshot first and only work without unsaved changes.

5.9 – Templates

A template is a single file ending in `.qtpl` that carries a project's design: colours, theme, CSS, fonts, layout, frames, plugins, translations, content too if you like. After applying it, another project is set up like the one it comes from. **Templates** creates and applies them. The page saves nothing by itself; creating and applying are done with the buttons.



"Create a template" with the parts at the top, "Apply template" below.

The parts

A template consists of up to twelve parts; when creating as when applying you choose which:

Part	What it carries
Notes (content/)	the files in the content folder; usually not wanted for a design, but for a template that explains itself
Colours & fonts	the base colours for light and dark, the three fonts and where they come from
Community theme	the chosen theme with all its settings; the package is installed when applying
CSS	

variables	your own values under **Variables**
Custom CSS	`custom.scss` and all your own stylesheets with their load order
Font files	the fonts you brought yourself and their rules
Static files	everything under `quartz/static` except the fonts: logos, pictures, snippets
Layout	which component appears where, per page type, and the breakpoints
Custom frames	the frames you built; registered afresh in the target project
Plugins	all plugin entries with options, order and position
Translations	the changed texts from Quartz's language files
Theme presets	the saved presets

Creating a template

Under **Create a template**: **Template name** (prefilled with the project name) and **Description (optional)**, then the parts with their figures, such as "58 plugins, 41 active". What the project has is ticked.

For **Translations**, **Scope** chooses between **Only your changes** and **All texts**. QuartzHub only knows what was changed since you edited a text under **Configuration ▸ Translations** (4.4); for languages before that, take "All texts".

Save template… asks for the place and writes the file. Static files may be at most 25 MB together, the notes at most 100 MB: a template is a design, not a backup. A part above that is not in the list; a note below it says why.

> ⚠ **What a template gives away**
>
> A template names the folder of the project it comes from and carries whatever the chosen parts contain. Before passing one on, check what is in it, especially **Notes (content/)**, **Static files** and **Plugins**, whose options may contain addresses, for instance.

Applying a template

Under **Apply template**:

- **Basic template** loads the template also offered when creating a project (2.2) from GitHub. Only the design parts are ticked, not content, layout, plugins,

translations and presets.
- **Choose a template…** opens a `.qtpl` file. Everything it contains is ticked. Templates in the old format, a folder instead of a file, carry the mark **Old format** and can be applied too.

Below are name, description, date and origin of the template and, after a check, what each part would do in this project: how much is new, replaced, kept as is or refused. The check writes nothing.

On conflict:

- **Template wins**: what the project already has is replaced by the template.
- **Project wins**: what the project already has stays; only what is missing is added. Colours & fonts and the layout are then not taken over at all, not even in part.

If something has to be installed, a theme or a plugin, it says so below; that needs a network connection.

Apply template asks once, takes a snapshot (“Before template import”) and applies the parts one after another. If one fails, the others carry on. At the end it says “Done: everything applied, nothing left open.” or lists what did not work or stayed unchanged.

A few things a template never does:

- It does not write into a linked vault; if the content folder is a link, **Notes (content/)** is skipped.
- It does not replace a project image you chose yourself (4.3).
- It writes nothing outside the project folder.
- With quartz-forms, the project keeps its sending, keys, spam protection and forms; only missing forms are added.

If you do not like the result, the snapshot leads back (7.3).

6 – Publishing

The **Publishing** group has four pages: **Preview & Build** shows the site and builds it, **Git sync** keeps the project in step with a repository, **Documents** makes books from it, and **Publish** takes the result where it should go.

Preview & Build

1. **The preview**: the site in the window while you write.
2. **The build**: the finished files, and where they go.
3. **Reading the output**: what Quartz says, and what of it matters.
4. **Other servers**: previews QuartzHub did not start itself.

Git Sync and documents

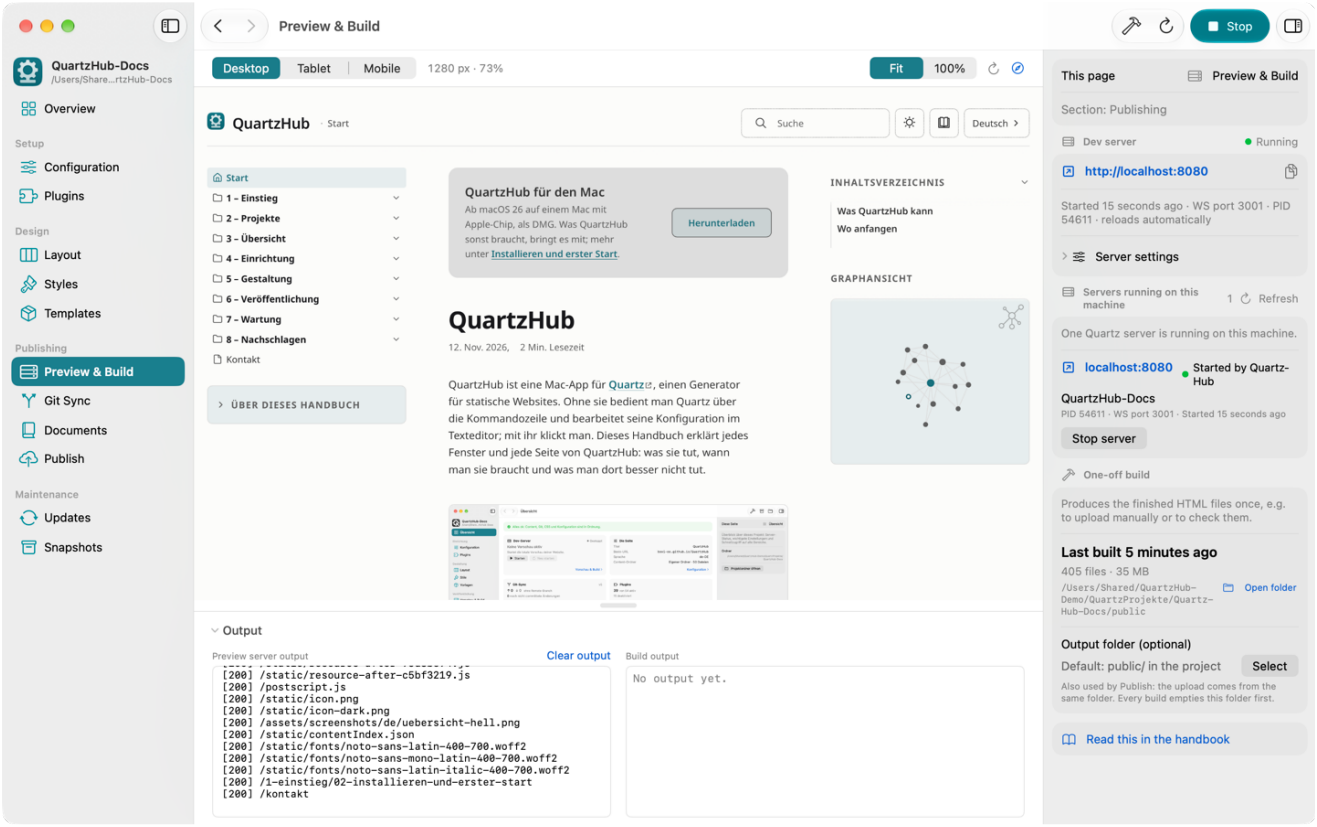
5. **Git Sync**: keeping and sharing the project in a repository.
6. **Documents**: books as PDF, ePub or HTML.

Publish

7. **Connections and targets**: the difference, and which target fits.
8. **GitHub Pages**
9. **To a web space**: SFTP, rsync, FTP.
10. **To a folder**
11. **Webhook**: Netlify, Cloudflare Pages, Vercel.
12. **What gets uploaded**: the preview of the changes and the run.

6.1 – The preview

The preview is Quartz's own dev server: Quartz builds the site, serves it on the Mac and rebuilds on every change. **Preview & Build** shows it in the middle of the window.



The running preview. Width and scale at the top, the output below, the server on the right.

Starting and stopping

Start in the toolbar, or **Project ▸ Start Server** (⌘R) from any window of the project. Then the button reads **Stop** (⌘.), with **Restart** next to it. The state is in the inspector: **Stopped**, **Starting…**, **Running**, **Stopping…** or **Error**.

The server counts as started when it answers. The first time that takes as long as a build, half a minute or more for a large site. After that, Quartz rebuilds after every change, to the notes as to the configuration, and the page reloads by itself.

Before every start QuartzHub puts the project in order: it rewrites your own frames, puts back a missing line in `custom.scss` and fetches missing fonts. What it does is in the output.

The start is refused, with the reason in red at the top, when

- the project's npm packages are missing, after a fresh clone for instance; **Install** on the overview fetches them;

- a server QuartzHub did not start is already running for the same project (6.4).

The preview in the window

- **Desktop**, **Tablet**, **Mobile**: the site is built at a fitting width, desktop at least 1280 pixels, and fitted into the window. Next to it are width and scale, such as “1280 px · 73%”.
- **Fit** or **100%**: scale down, or show at real size and scroll sideways.
- **Reload preview** and **Open in browser**.

Links within the site open in the preview, links to the outside in the browser.

Server settings

In the inspector under **Server settings**:

- **Port** (default 8080) and **WS port** (3001, for the automatic reload). If a port is in use, the server takes the next free one at start and says so in the output.
- **Remote dev host (optional)**: only for a preview behind a tunnel, from another computer for instance. The server then does not move to other ports.

The settings apply until QuartzHub quits. They can only be changed with the server stopped.

What happens to running servers when QuartzHub quits is set in the settings (2.5).

6.2 – The build

A build produces the finished site once: HTML, CSS, pictures, everything that goes onto a server. The preview builds too, but only for itself; publishing needs a build.

Building

- **Build now** (hammer) in the toolbar of **Preview & Build**,
- **Project ▸ Run build** (⌘B) from any window of the project,
- **Run build** on the overview,
- **Build now** under **Publish**.

During the build, the inspector says since when it has been running and what Quartz is doing: preparing, reading the notes, writing the pages. Then “Successful in 12.3s” or “Failed in …”, and what Quartz said about it is in the output (6.3).

One-off build shows when the site was last built, with the number and size of the files, and **Open folder**. QuartzHub reads this from the output folder itself.

The output folder

The default is `public/` in the project. **Output folder (optional)** sets another one, in the project or elsewhere. It also applies to **Publish**: uploads come from the same folder.

> ⚠ **Every build empties its output folder**
>
> Quartz deletes everything in the output folder before each build. QuartzHub therefore refuses folders that cannot take that: the project folder itself, a folder containing the project, the home folder, and folders of the project such as `content/` or `quartz/`. If a folder holds files that do not look like a build, QuartzHub asks first.

The preview always builds into `public/`, whatever this setting says. While it runs, a build cannot write into `public/` at the same time; QuartzHub says so and does not build.

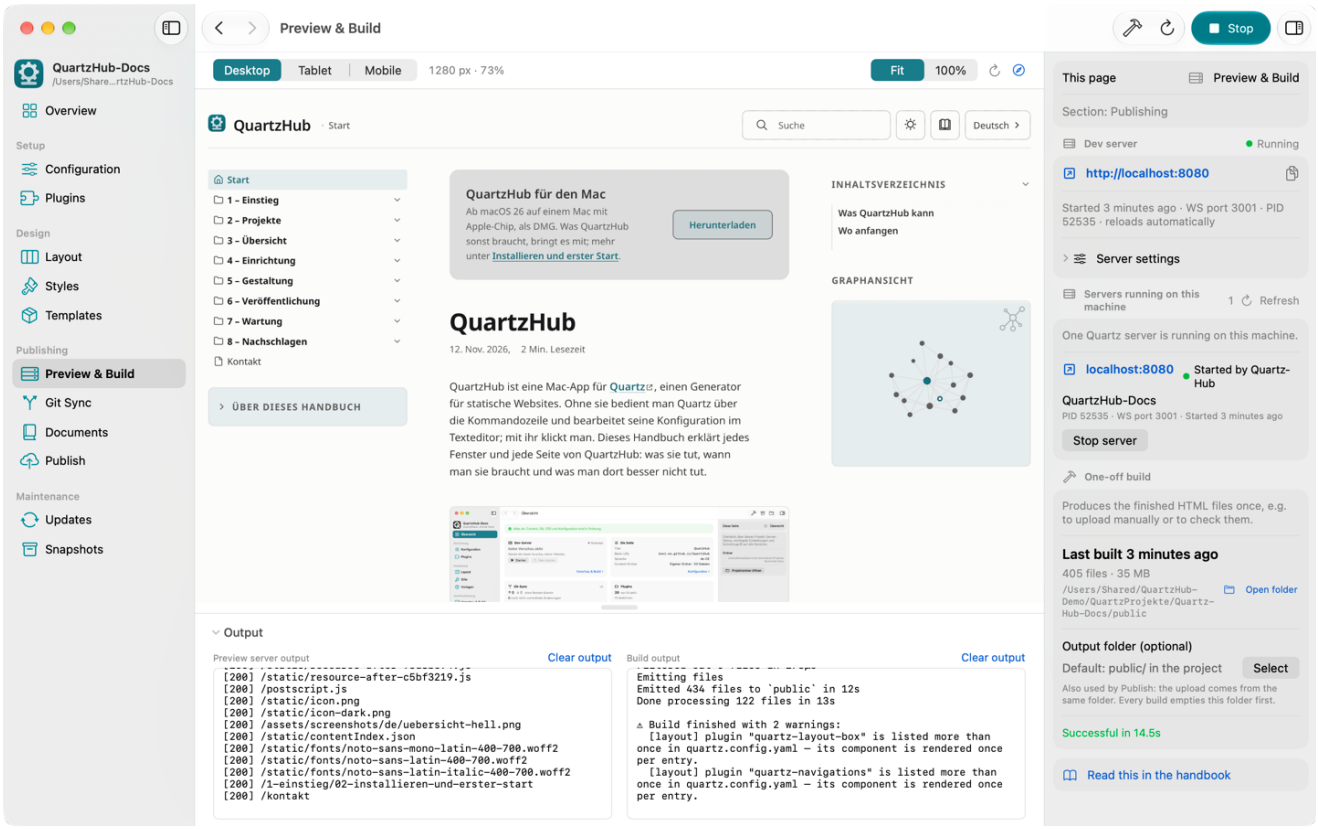
Every way to build uses the same saved output folder. When the field holds a new folder not yet confirmed with Return, **Build now** takes it first; a folder QuartzHub refuses is not built into.

When a build is needed

Before every publish that transfers files. **Publish** does not build by itself; it uploads what is in the output folder and says when there is nothing there yet (6.12).

6.3 – Reading the output

At the bottom of **Preview & Build** is the **Output**, in two consoles side by side: on the left the preview server's, on the right the last build's. The grip above can be dragged; the output can also be folded, and the preview then gets the room.



Preview stopped, a build's output at the bottom right.

Colours

- **Normal text**: what Quartz itself prints.
- **Orange**: a sentence from QuartzHub about a run that continues, such as "Port 8080 is in use, the server takes 8081 instead." or what it put in order before the start.
- **Red**: Quartz's error output, or an error from QuartzHub, such as a refused build.

Clear output empties a console. Text can be selected and copied with ⌘C, for an error message for instance. Each console keeps roughly the last thousand lines while QuartzHub runs, even when you leave the page; after a restart they are empty.

What Quartz says

Quartz's output comes unchanged. A few lines come up often:

Line	Meaning
`Parsing input files`	Quartz is reading the notes.
`Emitting files`	Quartz is writing the pages.
`Done processing … files in …`	The build is finished.
`Started a Quartz server listening at http://localhost:8080`	The preview is running.
`Detected change, rebuilding…`	A file changed; the preview rebuilds.
`[200] /path`	The preview served a page.
`[404] /path`	Something was requested that does not exist, a picture with a wrong path for instance.

Warnings in square brackets name the plugin they come from, such as `[links]` for links to pages that do not exist. They do not stop the build.

When the build fails

A build mostly fails at one of three places:

1. **At the CSS.** An error in a stylesheet. **Styles ▸ Custom CSS** shows it with file and line, as does the overview (5.8).
2. **At a note.** Broken frontmatter (a colon without quotes, say) or an embed that does not exist. Quartz names the file; fix it in the editor.
3. **At a plugin.** A plugin that is missing or does not fit this Quartz. Quartz names it; **Plugins ▸ Installed** shows what Quartz's own check says about it (4.5).

The most important line is usually at the end of the red output. The overview also reports a crashed server under **Needs attention**.

6.4 – Other servers

A Quartz server runs as a process of its own and can outlive QuartzHub: when you choose “Keep running” at quit, when QuartzHub crashes or when someone starts Quartz in the Terminal. QuartzHub finds such servers.

Servers running on this machine

This section in the inspector of **Preview & Build** lists every Quartz server on the Mac, whoever started it. **Refresh** searches again; otherwise QuartzHub searches when the window comes back to the front and when one of its own servers starts or ends.

Each row shows:

- the address; a click opens it in the browser;
- **Started by QuartzHub** (in this session) or **Started elsewhere** (everything else);
- the project, if QuartzHub knows it, otherwise the site’s title or the folder;
- process number, WebSocket port and since when it has been running;
- **Stop server**.

A server QuartzHub did not start is never stopped by it on its own. **Stop server** asks first, because whoever started it may be working with it right now.

Why this matters

Two servers for the same project would build into the same folder. QuartzHub therefore starts no second one and says which is already running. If a port is taken, but not by a recognisable Quartz server, the section says so; the preview then takes another port.

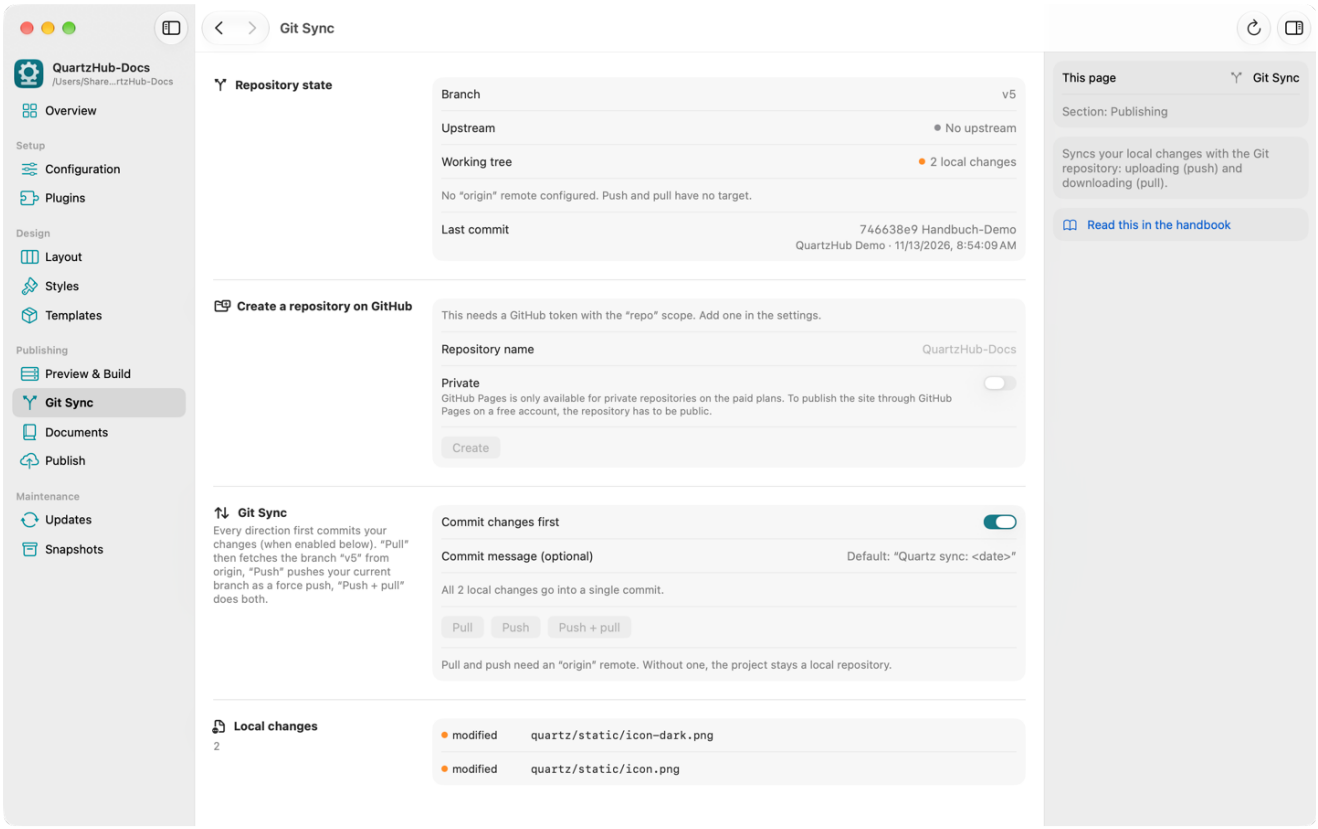
At start and at quit

At start QuartzHub asks about servers it started itself in an earlier session that are still running: **Keep running** or stop them.

At quit it asks about this session’s servers, depending on the setting (2.5). Before that it asks about unsaved changes and about running work such as an update or a publish.

6.5 – Git Sync

A Quartz project is a git repository. **Git Sync** uploads it to a repository on the net and fetches changes from there: as a backup, to carry on working on a second Mac, or as the basis for GitHub Pages (6.8). If you only want to publish, you do not need Git Sync.



A project without a repository on the net, the section for creating one below.

Repository state

- **Branch**: the branch checked out. Quartz projects stand on `v5`.
- **Upstream** and **Sync state**: whether the branch has a counterpart on the net and how many commits are still to be pushed or fetched; "Up to date with the remote" when nothing is missing. This is as of the last exchange; the page does not ask the net itself.
- **Working tree**: local changes and conflicts.
- **origin**: the address of the repository on the net, with **Open on GitHub** for GitHub.
- **Last commit**.

At the bottom, **Local changes** lists the changed files: new, modified, deleted, renamed, untracked, conflict. **Refresh** in the toolbar reads the state again.

Creating a repository on GitHub

A new project deliberately has no repository on the net, so nothing ends up at Quartz itself by accident. **Create a repository on GitHub** creates one under your account and enters it as `origin`:

- **Repository name**, prefilled with the project's name;
- **Private**: on a free account, GitHub Pages is only available for public repositories.

This needs a personal access token with the "repo" scope under **Settings ▸ Connections** (6.7). A repository elsewhere, on Codeberg say, is entered as `origin` in the Terminal (`git remote add origin <address>`).

Push and pull

- **Commit changes first** (on by default): puts all local changes into one commit, with the **Commit message** or "Quartz sync: ".
- **Pull** fetches the branch `v5` from `origin` and merges it.
- **Push** uploads the current branch, as a force push: what exists only on the net is lost.
- **Push + pull** does both.

QuartzHub uses Quartz's own `quartz sync` for this. Below, it then says "Successful." or "Failed." with the output.

Signing in: for a repository on github.com over https, QuartzHub uses the token from the settings. For ssh addresses and other providers, git's own keys and credential helpers apply, as in the Terminal.

> ⚠ **Force push means: this Mac wins**
>
> If two computers work on the same project: first **Pull**, then change, then **Push**. Whoever pushes first overwrites what the other computer uploaded.

When a pull fails

If both sides changed the same file, a conflict remains: the working tree shows it, and at the top it says "A merge has been started and is not finished." **Abort merge** goes back to the state before the pull; what was staged by hand since would be lost with it, and QuartzHub asks first then. QuartzHub cannot resolve a conflict; that is done in an editor or in the Terminal.

What does not come along

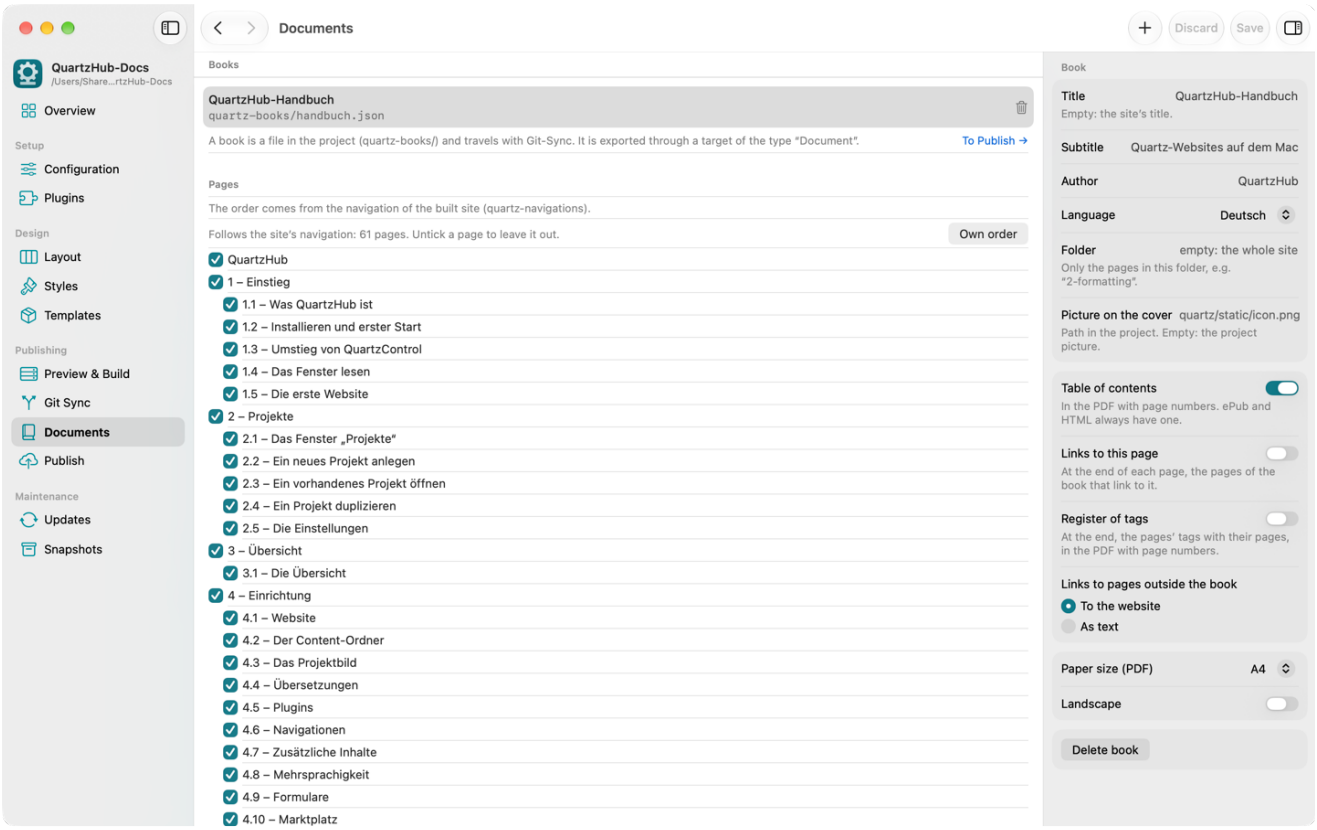
- `node_modules/` and `public/`: they are made afresh on the other computer (2.3).
- `.quartz-gui/`, meaning your frames, snapshots and targets: the folder is in `.gitignore`. Your own frames are therefore copied by hand or taken from

a snapshot (3.1).
- A linked vault: Quartz uploads the notes as files; on the other computer the content folder has to be linked again (4.2).

Books (`quartz-books/`) come along.

6.6 – Documents

A book is a selection of the site's pages in an order, with a cover and a table of contents. **Documents** creates books; they are put out through a target of the type "Document" under **Publish**, as PDF, ePub or a single HTML file. The PDF of this handbook was made that way.



A book following the site's navigation. Its settings on the right.

Creating books

➕ **New book** asks for the title and creates a book that follows the site's navigation at first. Each book is a file under `quartz-books/` in the project and comes along with Git Sync. The file is named after that title, "Garden notes" becomes `garden-notes.json`, and keeps the name when you change the title later; the exported files are named the same. The name is shown under the title while you create the book.

Save (⌘S) writes changes; **Discard** takes them back.

The pages

QuartzHub reads the list of pages from the last build. Without a build it says so and points to **Preview & Build**. The content index has to be on as well (4.1).

- **Following the navigation** (default): the book takes the order of the site's navigation. Untick a page to leave it out. Pages the navigation does not show are listed below.
- **Own order**: copies the navigation into a list of its own. Drag pages into place or move them with **Arrange ▸ Move Up/Move Down** (⌥⌘↑, ⌥⌘↓). **+ Heading** inserts a heading such as "Appendix"; **Take in** adds a page that is not in the navigation. **Follow the navigation again** drops the own list.

The start page of the chosen folder always comes first.

Settings in the inspector

- **Title** (empty: the site's title), **Subtitle**, **Author**.
- **Language**, if the site has several (4.8).
- **Folder**: only the pages in this folder; empty for the whole site.
- **Picture on the cover**: a path in the project; empty for the project image.
- **Table of contents** (in the PDF with page numbers), **Links to this page** at the end of each page, **Register of tags** at the end.
- **Links to pages outside the book**: **To the website** (needs a base URL) or **As text**.
- **Paper size (PDF)**: A4, A5 or Letter, plus **Landscape**.

Putting it out

Under **Publish**, create a target of the type **Document**: **Target folder**, **Book** and the **Formats** PDF, ePub, HTML (one file) (6.7). **Export now** writes the files, `<book>.pdf` and so on; a running export can be cancelled.

A PDF has a cover, the table of contents with page numbers, bookmarks by heading and a footer with title and page number. Formulas and diagrams are drawn as pictures; forms, videos and embedded pages are replaced by a sentence with their address.

If QuartzHub finds faults, such as pictures that do not load or links without a target, it writes nothing and says what it found. For formulas and diagrams the export loads stylesheets and scripts from the net; without a connection they are missing.

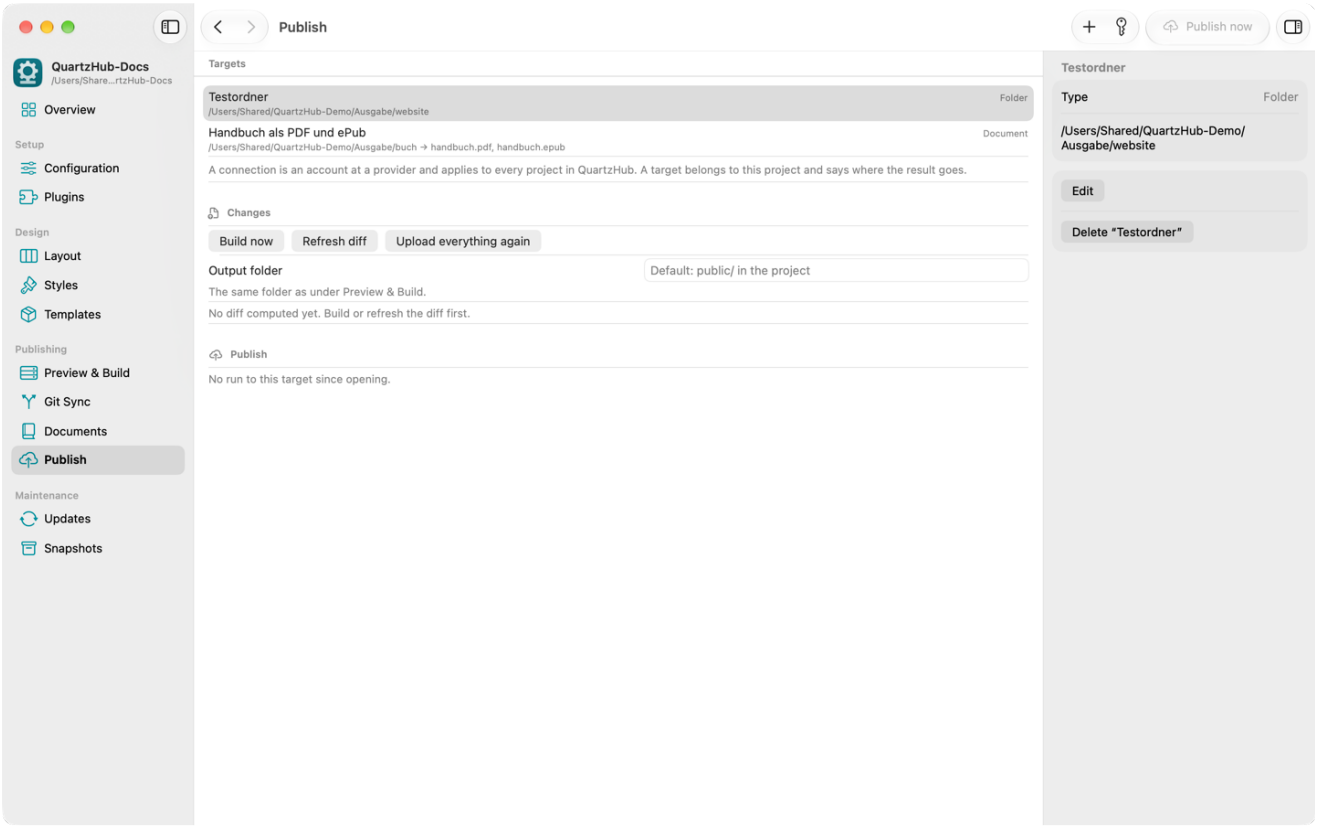
Upload documents too on another target uploads the last exported files with the site into the folder `downloads/`; QuartzHub shows a link for it you can put on a page.

6.7 – Connections and targets

Publishing takes two things:

- A **connection** is an account at a provider: a server login, a build hook, a token for GitHub. Connections apply to all projects and live under **Settings ▸ Connections**.
- A **target** belongs to one project and says where its site goes: which connection, which folder on the server, which branch, whether to delete. Targets live under **Publish**.

That way two projects use the same web space with one connection and two targets in different folders.



Two targets, a folder and a document. Below, the "Changes" section, with no diff computed yet.

Which target fits?

I have …	Target	Chapter
a GitHub account and want to pay nothing	**Git branch** with GitHub Pages	6.8
a web space at a hosting provider	**SFTP** (or rsync), FTP if nothing else	6.9

a server of my own or a NAS	**SFTP** with rsync	6.9
Netlify, Cloudflare Pages or Vercel, which build themselves	**Webhook**	6.11
a folder something else passes on (cloud folder, share)	**Folder**	6.10
a book as PDF or ePub	**Document**	6.6

Codeberg Pages and GitLab Pages also go through **Git branch**.

Creating connections

Settings ▸ **Connections** (⌘, or **Manage connections** under **Publish**):

- **GitHub**: a **Personal access token**. **Save & check** asks GitHub whether it is valid and shows **Valid** with the account name, **Rejected** or **Cannot check**. For the marketplace a token without permissions is enough; repositories and GitHub Pages need the “repo” scope.
- **SFTP / SSH**: **Host**, **Port** (22), **Username** and the **Authentication**: **Password**, **Private key** (as a file, usually from `~/.ssh`, or pasted) or **SSH agent**. Your own `~/.ssh/config` keeps applying.
- **FTP**: host, port (21), username, password and **FTPS (encrypted)**, on by default. Without FTPS, password and files cross the network unencrypted.
- **Webhook**: the provider’s **Webhook URL**, https only.

Passwords, keys, tokens and webhook URLs live in the macOS keychain, never in the project. A connection shows how many targets use it; deleting it warns.

Host key: at the first SFTP connection, QuartzHub shows the server’s fingerprint and asks whether it is right. The provider names the fingerprint. **Connect and remember** stores it for all projects. If it changes later, QuartzHub refuses the connection; only **Forget host key** makes it ask again, and only do that when the server was demonstrably set up anew.

Creating targets

➕ **New target** under **Publish**: **Name**, **Type** and, depending on the type, the fields the following chapters describe. For SFTP, FTP and Webhook you choose the **Connection**; **New … connection** creates one without leaving the dialog.

For all types except Git branch and Document there is:

- **Also delete removed files at the target** (on by default): what disappeared from the site disappears at the target too.
- **Never publish (one path per line)**: paths from the build that are never uploaded, such as `.htaccess` when the server has its own.

- **Upload documents too** (6.6).

Targets live in `.quartz-gui/publish-targets.json`, without passwords. A copy of the project does not get them (2.4).

> ⚠ **The base URL first**
>
> If the base URL still says `localhost`, **Publish** says so at the top. Otherwise links, RSS feed and preview images of a published site point nowhere (4.1).

6.8 – GitHub Pages

GitHub Pages serves a site from a branch of a repository, free and over https. QuartzHub puts the build as one commit onto a branch of its own, usually `gh-pages`, and points Pages at that branch. GitHub Actions are not needed.

Setting it up once

1. **Token**: under **Settings ▸ Connections**, enter a personal access token with the "repo" scope (6.7).
2. **Repository**: under **Git Sync ▸ Create a repository on GitHub**. On a free account it has to be public (6.5).
3. **Base URL**: under **Configuration ▸ Site**, enter the future address, usually `name.github.io/repository`, or your own domain.
4. **Target**: under **Publish**, a new target of the type **Git branch**, **Provider** GitHub Pages, **Branch** `gh-pages`. The branch must not be the one the project itself stands on.
5. **Switching Pages on**: in the **GitHub Pages** section below the target, **Point at "gh-pages"**.

Then build and **Publish now** (6.12). A few minutes later the site is at the address the section names.

The GitHub Pages section

It shows whether Pages is **Published**, **Building**, **Failed** or **Never built**, the site's address and which branch it comes from. **Reload status** asks GitHub again.

Custom domain: such as `wiki.example.org`. At the domain's provider, a DNS entry has to point at GitHub; GitHub describes how. **Save settings** writes the domain; GitHub puts it as a file `CNAME` into the branch, and QuartzHub keeps it at every publish. **Enforce HTTPS** only works once GitHub has issued the certificate, a few minutes after setting the domain.

What happens on publishing

The branch is replaced entirely by the current build every time (force push), in a single commit without history. That is why QuartzHub asks first. If nothing changed, nothing is uploaded. The project's hooks do not run.

Codeberg Pages serves the branch `pages` the same way. GitLab additionally needs a CI job that serves the files; the branch alone publishes nothing there.

6.9 – To a web space

For a web space at a hosting provider or a server of your own there are three ways. A connection under **Settings ▸ Connections** comes first (6.7).

Way	When
SFTP	the normal case; every host with SSH access can do it
SFTP with rsync	faster, compares against the server; needs a key and rsync on the server
FTP	only when the host offers nothing else, and then with FTPS

The target

A new target of the type **SFTP** or **FTP**, with the matching **Connection**, and:

- **Remote path**: the folder on the server the site goes into, such as `httpdocs` or `/var/www/example.org`. Without a leading slash it counts from the login directory; on a web space that is the normal case. Which folder serves the site, the host tells you.
- **Transfer** (SFTP only): **SFTP (file by file)** or **rsync (faster, compares against the server)**.
- **Also delete removed files at the target**, **Never publish**.

QuartzHub creates the site's folder, but not folders above it.

> ⚠ **Deleting and the login directory**
>
> With deleting on, QuartzHub refuses the login directory itself (`/`, `.`, `~`) as the path: it would be emptied. The site belongs in a subfolder.

SFTP or rsync

SFTP compares the build with a list of what QuartzHub last sent to this target and transfers only what is new or changed. It does not notice what someone changes or deletes directly on the server; for that there is **Upload everything again** (6.12).

rsync asks the server itself what is there; the comparison is right even after changes by hand. rsync needs:

- a connection with **Private key** as a file or **SSH agent**, no password;
- a confirmed host key: publish once via SFTP, then rsync can be chosen;
- rsync on the server.

The dialog says what is missing.

> ⚠ **rsync deletes everything that is not part of the build**
>
> With deleting on, rsync removes every file in the target folder the build does not have, including files QuartzHub never uploaded, such as an `.htaccess` or a folder of downloads. What should stay goes under **Never publish**, or the site gets a folder of its own.

FTP

FTP transfers file by file like SFTP, after the list of the last run. Without **FTPS** the password and the files cross the network in plain text; QuartzHub then warns.

When it does not work

QuartzHub says why, with the server's name: not reachable, port refused, login refused (with its own sentence for password, key and agent), no answer for 60 seconds. If the server is in your own network, a NAS for instance, QuartzHub needs permission for the local network (1.2).

6.10 – To a folder

A target of the type **Folder** copies the site into a folder: on the Mac, on a drive or a share. Useful when something else passes the files on (a cloud folder, a NAS that serves them itself), or to see what would be uploaded.

- **Target folder**: a full path, via **Select**. It must not lie inside the build folder or contain it, and must not be the project folder. The first time, QuartzHub creates it, along with missing folders above it.
- **Also delete removed files at the target** and **Never publish** as with the other targets.

As with SFTP, only what is new or changed is transferred. Each file is first written under a temporary name and then renamed, so a sync program never sees half a file.

If the target was used before and the folder is now gone or empty, QuartzHub stops: usually a drive is not connected. If the folder really should be created afresh, choose **Upload everything again** first (6.12).

6.11 – Webhook

Some providers build the site themselves, from the repository: Netlify, Cloudflare Pages, Vercel or a CI of your own. A **webhook** triggers that build. No files are transferred; the provider takes the state from the repository.

That also means: what is not in the repository does not arrive. So a **Push** under **Git Sync** comes before the webhook (6.5).

Setting it up

1. At the provider, create a **build hook**; it gives an address for it, such as `https://api.netlify.com/build_hooks/…`.
2. Under **Settings ▸ Connections**, create a **Webhook** connection with this address. The address is a secret: whoever knows it can trigger builds. QuartzHub keeps it in the keychain and only shows its beginning.
3. Under **Publish**, a target of the type **Webhook** with this connection.

Publish now asks briefly and then sends a request to the address. If the provider answers with success, the build has been triggered there; whether it succeeds, the provider shows.

6.12 – What gets uploaded

Before QuartzHub transfers anything, it shows what changed since the last run to this target. Under **Publish** this is in the **Changes** section below the selected target.

The changes

- **Build now** builds the site and then computes the changes again.
- **Refresh diff** computes them again without building. QuartzHub does not compute them by itself.
- **Output folder**: the same as under **Preview & Build** (6.2).

Below it says “Added: 3 · Changed: 12 · Removed: 1” and a list for each. For SFTP, FTP, folder and rsync, each file has a tick: without it the file is left out of this run, and next time it is listed again. Leaving it out for good is done with **Never publish** in the target.

How the comparison works:

Target	Compared with
SFTP, FTP, folder	the list of what QuartzHub last sent to this target, offline
SFTP with rsync	what is on the server
Git branch	the branch on the net
Document	whether the files are there already
Webhook	nothing; nothing is transferred

Upload everything again forgets the list of the last run; the preview then offers the whole build again. That is needed when something was deleted on the server by hand or a transfer broke off. A target that is pointed at another folder or connection also uploads everything next time. With rsync the button is not there: rsync compares with the server anyway.

The run

Publish now in the toolbar asks first, with the number of files to upload and delete; for a Git branch with the note that the branch is replaced. It does not build by itself; what is in the output folder is uploaded. No snapshot is taken.

During the run, a bar shows which file is being handled. You can leave the page; the run carries on, and at quit QuartzHub asks. Only an export can be cancelled.

Afterwards it says **Published successfully.**, **Publish failed.** or, if the server refused single deletions, how many files stayed; QuartzHub tries them again next time. The output below lists every file: `↑` uploaded, `×` deleted.

The usual order

Save, **Build now**, look at the list, **Publish now**. If the list holds more than expected, take a look before pressing.

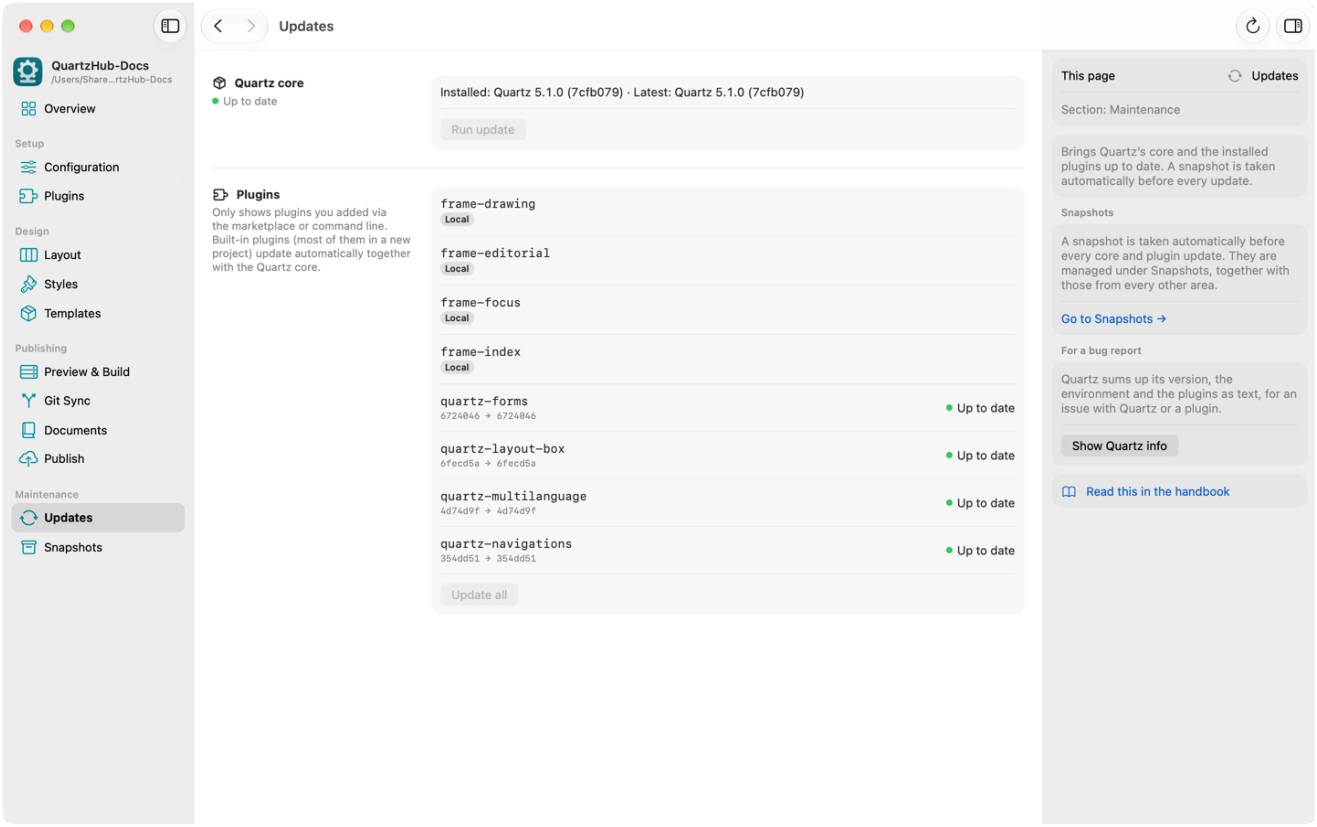
7 – Maintenance

A site that runs needs little care. Now and then there is something new from Quartz or a plugin, and sometimes something goes wrong. That is what the **Maintenance** group is for.

1. **Updates**: bringing Quartz and the plugins up to date.
2. **Snapshots**: what they are, when they are taken, what they contain.
3. **Restoring**: bringing back a snapshot or single files.
4. **Cleaning up**: what piles up and how to get rid of it.

7.1 – Updates

Quartz keeps evolving, and so do the plugins. **Updates** says whether there is something new and brings it into the project. QuartzHub takes a snapshot before every update.



Quartz is up to date, and so are the plugins from the marketplace.

Checking

When the page opens, QuartzHub asks GitHub whether there is a newer version of Quartz and whether the installed plugins have newer ones. **Check again** in the toolbar asks once more. Without a network it says **Cannot check**, not “Up to date”.

Quartz’s core

The line shows which version is installed and which is the latest, and how many changes are missing. Next to it **Up to date**, **Update available**, **Unfinished** or **Cannot check**. **New in Quartz …** unfolds the changes Quartz itself wrote down for them.

If the project’s remote `quartz-upstream` does not point to jackyzha0/quartz, to a copy of your own for instance, a note below says that **Run update** fetches from there while the line shows the state of jackyzha0/quartz.

Run update asks first and then:

1. takes a snapshot (“Before core update”);
2. fetches the new version of Quartz and merges it with the project. Your own packages in `package.json` are set aside briefly for this and put back afterwards;
3. reinstalls the npm packages;
4. builds the site once.

Afterwards it says “The core update is complete.”, with the output to unfold and **Open the snapshot from before the update**. From Quartz 5.1 on, QuartzHub then runs Quartz’s own check of the plugins and says whether entries should be moved (4.5).

> ⚠ **Commit first, then update**
>
> An update merges Quartz’s changes with the project. Uncommitted changes to Quartz’s files can get in the way; QuartzHub then names them. If you use **Git Sync**, commit first (6.5).

When it gets stuck

- **Conflicts**: if Quartz and the project changed the same place, the merge stops. The page names the files. **Abort merge** goes back to the state before the update; otherwise resolve the conflicts in an editor and run the update again. Your own texts in the language files are guarded by the update protection (4.4).
- **Unfinished**: if `npm install` failed or your own packages are still missing, the page says so. **Run update** catches up.
- **Back**: the snapshot from before the update leads to the old state. For the Updates page to report the old state too, switch on **Also reset the project commit** when restoring (7.3).

Plugins

These are the plugins added through the marketplace or the command line. Quartz’s built-in plugins come with the core update. Each row shows the installed and the latest version and **Up to date**, **Update** or **Cannot check**; **Local** means the plugin lives in the project itself (one of your frames, for instance) and has no source on the net.

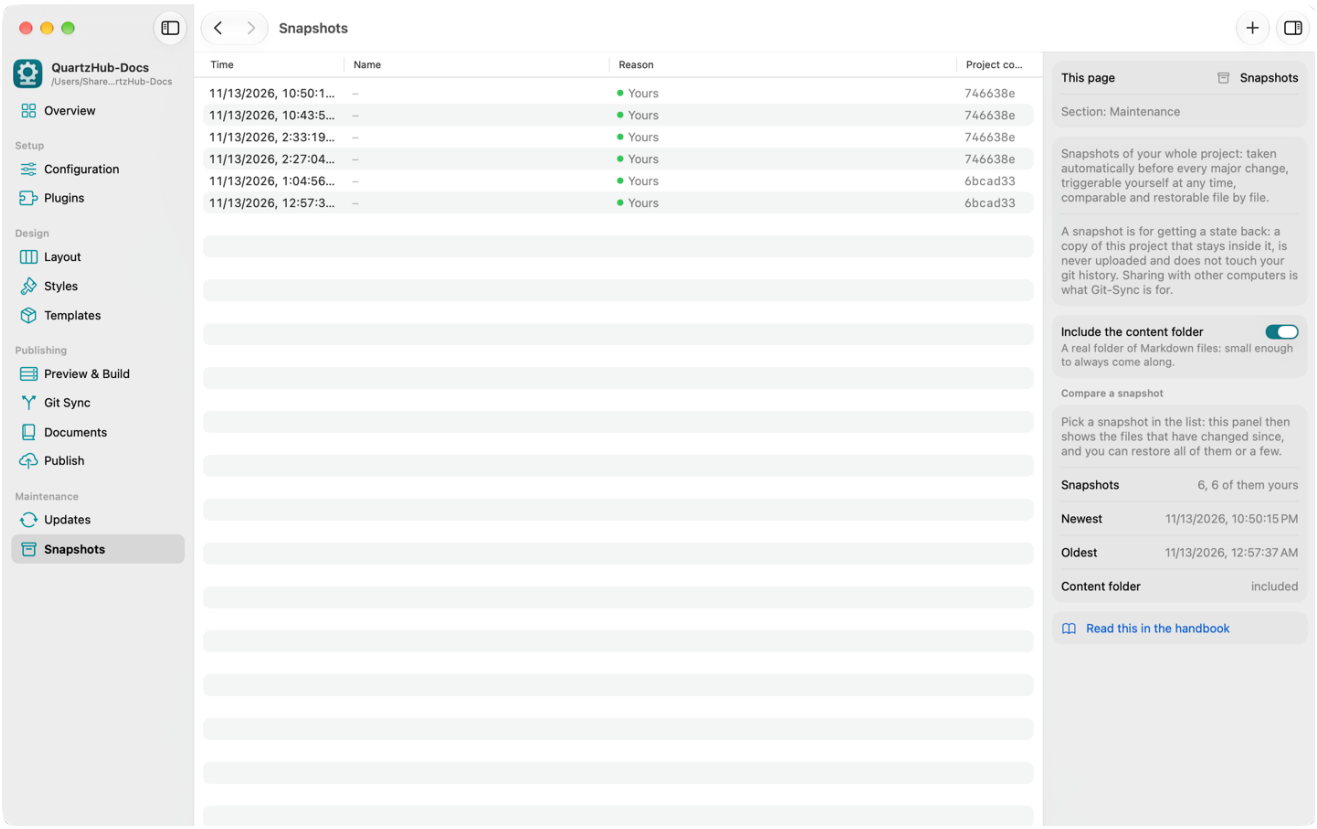
Update and **Update all** ask first, take a snapshot and have Quartz fetch the latest version. The configuration stays as it is.

For a bug report

From Quartz 5.1 on, the inspector has **Show Quartz info**: Quartz sums up its version, the environment and the plugins as text you can copy into a bug report for Quartz or a plugin.

7.2 – Snapshots

A snapshot is a copy of the whole project at one point in time. It stays in the project, is never uploaded and does not touch your git history. It is there for getting things back when something went wrong; sharing with other computers is what Git Sync is for.



A snapshot marked “Yours”. On the right, whether the content folder is included.

What is in it

Everything in the project except what a build or an install produces anew: `node_modules/`, `public/`, the downloaded plugins and caches. So configuration, stylesheets, your frames, translations, books, targets, and the notes if the content folder is a real folder.

Include the content folder (inspector, on by default) takes the notes along or leaves them out. A linked vault is never included; back that up with Obsidian or Time Machine.

The snapshots live in `.quartz-gui/snapshots.git/`, a git repository of their own.

When QuartzHub takes one

By itself, before every larger change. The **Reason** column says before what:

Reason	Before
Before config change	saving the configuration, under Configuration, Layout, Styles, Plugins; at most one every 15 minutes
Before plugin change	installing, removing, updating or moving a plugin; registering a new frame
Before core update	the update of Quartz
Before content switch	switching the content folder
Before template import	applying a template
Before the styles moved	moving old stylesheets
Before restore	every restore
Old config backup	configuration backups from early versions

If nothing changed since the last snapshot, QuartzHub takes no new one; before a core update it takes one anyway.

Taking one yourself: ➕ **Take snapshot** in the toolbar, with a name such as “before switching themes”, or **Take snapshot** on the overview, without a name. These carry the reason **Yours**.

How long they stay

Your own snapshots and those before a core update stay until you delete them. The other automatic ones are thinned out over time:

- younger than a day: all;
- up to a week: one per day;
- up to eight weeks: one per week;
- older: one per month.

The newest one of each stays. To keep a state for sure, take a snapshot yourself.

The list

Time, **Name**, **Reason** and, in a wide window, **Project commit**: the project's commit when the snapshot was taken (a dash without a git repository). The context menu has **Export as ZIP** and **Delete**. What a snapshot changes compared with today and how to get it back is in 7.3.

7.3 – Restoring

Comparing

A click on a snapshot under **Snapshots** compares it with the current state. The inspector shows **Compared with the current state**: how many files differ, and each with its state:

- **new since**: did not exist then; restoring deletes it;
- **deleted since**: existed then; restoring brings it back;
- **changed**: the symbol next to it shows the difference line by line.

If **Include the content folder** is off or the content folder is a link, the notes are left out of comparing and restoring.

Restoring

- **Restore everything** puts the whole project back to the snapshot. Files that did not exist then are deleted.
- **Restore selection** puts back only the ticked files.

Both ask first and take a snapshot of the current state beforehand ("Before restore"); a restore can itself be taken back. If `package.json` changed, QuartzHub then reinstalls the npm packages. While it runs, QuartzHub asks at quit.

Also reset the project commit appears when the snapshot was taken on a different git commit than today, before a core update for instance. It also resets the project's git history; only then does **Updates** report the old state of Quartz again.

The notes follow the **Include the content folder** switch: with it off they stay as they are, and git then shows them as changes against the old commit. With it on they come back from the snapshot; their state before is kept in the "Before restore" snapshot.

> ⚠ **A pushed commit**
>
> If the commit was already pushed, the local state differs from the repository on the net after the reset.

Exporting as ZIP

Export as ZIP in the context menu or the inspector writes a snapshot's state into a ZIP file, to get a single file out without resetting anything, for instance.

Set-aside content folders

Switching the content folder sets the old one aside (4.2). These folders are listed

under **Snapshots** in a list of their own, with time and size or the target of the link back then. **Restore** puts one back and sets the current one aside as well.

7.4 – Cleaning up

QuartzHub clears most things itself: it thins out old automatic snapshots, deletes the logs of earlier previews at the next start of the preview, and removes leftover work folders of export, relay and SSH at app start. For the rest there are these places:

What	Where	What it removes
Snapshots	**Snapshots**, context menu **Delete** or the Delete key	the snapshot; space is freed after a few minutes
Set-aside content folders	**Snapshots**, list below, trash	the folder for good; it is the only copy
Downloaded plugins nothing uses	**Plugins ▸ Installed ▸ Remove orphaned plugin folders**	folders under `.quartz/plugins/` the configuration does not point to
Unused fonts	**Styles ▸ Basics ▸ Unused fonts**	imported font files and their rules
Stylesheets	**Styles ▸ Custom CSS**, load order	the file; back only via git or a snapshot
Theme descriptions	**Settings ▸ Data & maintenance ▸ Clear cache**	the cache; reloaded when needed
What last went to a target	**Publish ▸ Upload everything again**	the list of the last run; the next uploads everything
A project from the list	**Projects ▸ Remove from list**	only the entry, not the folder (2.1)

`node_modules/` and `public/` can be deleted at any time: the overview then offers **Install**, and the next build creates `public/` anew.

8 – Reference

This part is not read front to back; it is there for looking things up.

1. **Menus and shortcuts**
2. **Where the data lives**: in the profile, in the project, in the keychain.
3. **Connections to the network**: when QuartzHub connects where.
4. **Known limits**: what QuartzHub cannot do or does differently.
5. **Glossary**

8.1 – Menus and shortcuts

On a German keyboard `[` and `]` sit on ⌥5 and ⌥6, so macOS offers ⌘Ö and ⌘Ä for ⌘[and ⌘]. In dialogs, Return always confirms and Esc cancels.

QuartzHub

Command	Shortcut
Settings…	⌘,

File

Command	Shortcut	
New Project…	⌘N	2.2
Open Existing Project…	⌘O	2.3
Save	⌘S	saves the page in front; grey when there is nothing to save

View

Command	Shortcut
Show/Hide Sidebar	^⌘S
Show Inspector / Hide Inspector	^⌘I

Go

Command	Shortcut
Back	⌘[
Forward	⌘]
Overview, Configuration, Plugins, Layout, Styles, Templates, Preview & Build, Git Sync, Documents, Publish, Updates, Snapshots	–

Project

Acts on the project in the frontmost window.

Command	Shortcut	
Start Server	⌘R	6.1
Stop Server	⌘.	
Run build	⌘B	6.2
Show Project Folder in Finder	–	

Arrange

Applies in lists that can be reordered: **Plugins ▸ Installed**, **Languages** and a book's own order.

Command	Shortcut
Move Up	⌥⌘↑
Move Down	⌥⌘↓

On the stage under **Layout ▸ Components**, **Earlier** and **Later** in the inspector move things.

Window

Command	Shortcut
Projects	⇧⌘0

Below are the open project windows, each with project and page.

Help

Command	What it does
Handbook	opens this handbook in the browser, in QuartzHub's language
This Page in the Handbook	opens the chapter for the page and tab of the front project window
Send feedback…	opens a new issue in QuartzHub's repository on GitHub, with version and macOS
Quartz documentation	opens quartz.jzhao.xyz
Plugin catalog	opens the quartz-community on GitHub
Open QuartzHub's data folder	shows the profile folder in the Finder (8.2)
Open licences	shows the licence texts of the bundled programs

8.2 – Where the data lives

In the profile

`~/Library/Application Support/QuartzHub`, opened via **Help ▸ Open QuartzHub's data folder** or **Settings ▸ Data & maintenance**.

File	Contents
`projects.json`	the project list: name, folder, when added and last opened
`settings.json`	the settings: language, theme, Node runtime, behaviour at quit, default project directory
`connections.json`	the connections, without passwords, keys and tokens
`relay-secrets.json`	per project, which recipients and captcha secrets the form relay has, without the values
`running-servers.json`	the preview servers QuartzHub started; for the question at the next start
`templates/`	the basic template last loaded
`theme-docs/`	the descriptions of the community themes

A file QuartzHub cannot read is set aside as `<name>.corrupt-<time>` instead of being overwritten, and **Settings ▸ Data & maintenance** says so.

Window positions and the interface language are kept by macOS in the app's defaults.

In the keychain

In the login keychain, service "QuartzHub":

- **"QuartzHub – "**: password, key, token or webhook URL of a connection.
- **"QuartzHub – Relay"**: per project, the form relay's recipient addresses and secrets.

The entries can be viewed and deleted with macOS's Keychain Access. They do not move to another Mac; enter them again there.

In the project

Place	Contents
`quartz.config.yaml`	the configuration: title, address, plugins, layout
`content/`	the notes, a folder or a link

`quartz/styles/custom.scss`	custom CSS; loads all other stylesheets
`quartz/styles/_quartzcontrol.scss`	load order, fonts and variables, written by QuartzHub
`quartz/styles/custom/`, `imported/`	your own stylesheets
`quartz/static/`	project image, fonts, pictures, snippets
`quartz/i18n/locales/`	the language files **Translations** edits
`quartz-books/`	the books; travel with git
`quartz-forms/keys/`	public keys for encrypting forms
`.quartz-gui/`	what QuartzHub keeps about the project, see below; listed in `.gitignore`

In `.quartz-gui/`:

Entry	Contents
`publish-targets.json`	the publishing targets, without passwords
`deploy-manifest-<target>.json`	what was last sent to a target
`snapshots.git/`	the snapshots, a git repository of their own
`snapshot-settings.json`	the snapshot settings
`content-backups/`	set-aside content folders
`authored-frames/`	your own frames
`layout-breakpoints.json`	the breakpoints of your own frames
`theme-presets.json`	the theme presets
`forms-relay.json`	the form relay's settings, without secrets
`project-prefs.json`	the output folder, for instance
`project-icon.json`, `icon-original.png`	the chosen project image and Quartz's original
`locale-baseline/`	the language files before your first change
`core-update.json`	a core update that did not run to the end
`logs/`	the output of the preview servers

QuartzHub only creates `.quartz-gui/` when it writes something there; reading creates nothing.

8.3 – Connections to the network

QuartzHub sends no usage data. It goes to the network only for things that cannot work without it, and then to the places in this table. It uses macOS's proxy and certificate settings.

By itself, when a page opens

When	Where	What
opening the **Projects** window	api.github.com (QuartzHub's repository)	asks whether a newer version of QuartzHub exists, at most every six hours, without signing in
opening **Overview** and **Updates**	github.com (Quartz's repository)	asks for the latest version of Quartz, without signing in
opening **Overview**, **Updates**, **Plugins**	the installed plugins' repositories, mostly github.com	asks for each plugin's latest version, without signing in

These requests send nothing but the question itself. Without a network the pages show that they cannot check.

When you do something

Action	Where
Creating a new project	github.com (Quartz), the npm registry
Using the basic template	raw.githubusercontent.com (once a day)
Duplicating a project, installing, restoring or updating plugins, core update	the npm registry, for plugins from git their repository
Plugins ▸ Marketplace	api.github.com, with the GitHub token if one is stored
Styles ▸ Community themes: catalogue and details	the npm registry, api.github.com, cdn.jsdelivr.net
Descriptions of theme settings	raw.githubusercontent.com (Obsidian's theme list and the theme), then from the cache
Choosing Google fonts or serving them locally	fonts.googleapis.com, fonts.gstatic.com
	stylesheets, fonts and pictures the site loads

Exporting a book	from outside; for formulas cdn.jsdelivr.net, for diagrams cdnjs.cloudflare.com
Checking the token, creating a repository, setting up GitHub Pages	api.github.com, with the token
Git Sync, Git branch targets	the project's repository, for github.com with the token
Publishing via SFTP, rsync, FTP	your own server, with the connection
Webhook	the provider's address
Checking the form relay	the `send.php` on your own web space
Links in the Help menu and in the preview	in the browser

The GitHub token only goes to GitHub, and to git only when the repository is on github.com; it never stands in an address or on the command line.

What works without a network

Editing, preview, build, snapshots, publishing into a folder. The site itself may load things from outside while building or in the preview, Google fonts for instance; that is up to Quartz and its plugins (5.5).

If a server is in your own network, macOS asks once whether QuartzHub may access the local network (1.2).

8.4 – Known limits

Computer and distribution

- **macOS 26 or later, Apple silicon only.** There is no version for Intel Macs, Linux or Windows.
- **Not notarised by Apple.** The first start needs “Open Anyway” once (1.2).
- **Updating is by hand.** When there is a newer version, the **Projects** window says so (2.1); downloading and replacing it is up to you (1.2).
- **A new project needs the network**: Quartz comes from GitHub, and so does the basic template.

Deliberately so

- No zooming of the interface; text size follows macOS.
- The desktop preview is built at 1280 pixels and fitted in.
- The export window for PDFs is always light, so diagrams are too.

Not there yet

- PDFs are not tagged for screen readers.

What to watch out for

- **Push in Git Sync is a force push** (6.5).
- **rsync with deleting on removes everything in the target folder that is not part of the build** (6.9).
- **Set-aside content folders are the only copy**; snapshots do not contain them (4.2).
- **Older automatic snapshots are thinned out** (7.2).

8.5 – Glossary

Base URL

The address the finished site is reachable at, without `https://`, such as `name.github.io/project`. Quartz builds links, the RSS feed and preview images from it (4.1).

Branch

A line of development in a git repository. Quartz projects stand on the branch `v5`; GitHub Pages usually serves the site from a branch of its own, `gh-pages` (6.8).

Breakpoint

The window width from which a site switches to the tablet or mobile layout. For your own frames 1200 and 800 pixels, adjustable (5.4).

Build

Producing the finished site from the notes: HTML, CSS, pictures, all in the output folder, usually `public/` (6.2).

Callout

A highlighted box in a note, written in Obsidian as `> [!note]`. Quartz shows it on the site the same way.

Commit

A saved state in a git repository. **Git Sync** puts local changes into one commit (6.5).

Community theme

A ready-made colour scheme from the `@quartz-themes` catalogue that lies on top of the base colours (5.6).

Component

A visible part of a page, such as search, table of contents or graph, with a position such as header or sidebar (5.3).

Connection and target

A **connection** is an account at a provider and applies to all projects; a **target** belongs

to one project and says where its site goes (6.7).

Content folder

The folder `content/` in the project with the notes the site is made from: a real folder or a link to a vault (4.2).

CSS variable

A named value in CSS, such as `--secondary` for the colour of links. Almost all colours and fonts of a Quartz site are variables (5.7).

Dev server

Quartz's own server for the preview. It builds the site, serves it on the Mac and rebuilds on every change (6.1).

Frame

The grid of a page: which areas there are and how they lie. Quartz brings three; your own are built under **Layout ▸ Frames** (5.4).

Frontmatter

The block between `---` at the start of a note, with entries such as `title`, `tags` or `draft`.

Marketplace

The catalogue of the quartz-community's plugins on GitHub, under **Plugins ▸ Marketplace** (4.10).

npm

Node's package manager. Quartz and many plugins are npm packages; they live in the project in `node_modules/`. QuartzHub brings npm along.

Page type

A kind of page Quartz builds: content pages, folder pages, tag pages, the 404 page and more depending on the plugins (5.2).

Plugin

An extension of Quartz. Almost everything Quartz does, a plugin does: reading Markdown, resolving links, search, the sidebar (4.5).

Preset

A saved combination of a community theme with its settings (5.6).

Project

A folder holding a Quartz site: configuration, stylesheets, plugins and content folder (2).

Quartz

A program that builds a website from Markdown notes, a “static site generator”. It understands Obsidian’s wikilinks, callouts and embedded pictures. QuartzHub works with Quartz 5 (**quartz.jzhao.xyz**).

Repository

A folder under version control with git, with its whole history. Every Quartz project is one; on the net it lives at GitHub, for instance (6.5).

Snapshot

A saved state of the whole project, which QuartzHub takes by itself before larger changes and which you can go back to (7.2).

Token

A key with which a program identifies itself to a service, here the personal access token for GitHub. It lives in the keychain (6.7).

Vault

A folder of notes managed by Obsidian. A project’s content folder can point to a vault or a folder in it (4.2).

Wikilink

A link in double square brackets, such as `[[Note]]`, as Obsidian writes it. Quartz understands it and turns it into a link on the site.